

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

К защите допустить:
Заведующий кафедрой ПОИТ
_____ О. Голда

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
дипломного проекта
на тему

**ПРОГРАММНОЕ СРЕДСТВО МНОГОПОЛЬЗОВАТЕЛЬСКОГО
ВЗАИМОДЕЙСТВИЯ В ОНЛАЙН ИГРЕ С ФИЗИЧЕСКОЙ МОДЕЛЬЮ
НА JAVASCRIPT-ПЛАТФОРМЕ**

БГУИР ДП 1-40 01 01 01 075 ПЗ

Студент	Е. С. Панкратьев
Руководитель	А. Г. Хмелев
Консультанты:	
<i>от кафедры ПОИТ</i>	А.Г. Хмелев
<i>по экономическому разделу</i>	А. А. Горюшкин
Нормоконтролёр	С. В. Болтак
Рецензент	

Минск 2026

РЕФЕРАТ

БГУИР ДП 1-40 01 01 075 ПЗ

ПРОГРАММНОЕ СРЕДСТВО МНОГОПОЛЬЗОВАТЕЛЬСКОГО ВЗАИМОДЕЙСТВИЯ В ОНЛАЙН ИГРЕ С ФИЗИЧЕСКОЙ МОДЕЛЬЮ НА JAVASCRIPT-ПЛАТФОРМЕ: дипломный проект / Е. С. Панкратьев. – Минск: БГУИР, 2026, – П.З. – 127 с., чертежей – 3 л. формата А1, плакатов – 3 л. формата А1.

МНОГОПОЛЬЗОВАТЕЛЬСКОЕ ВЗАИМОДЕЙСТВИЕ, ОНЛАЙН ИГРА, ФИЗИЧЕСКАЯ МОДЕЛЬ, JAVASCRIPT, TYPESCRIPT, NODE.JS, WEBSOCKET, REACT, СИНХРОНИЗАЦИЯ СОСТОЯНИЯ, ДЕТЕКЦИЯ КОЛЛИЗИЙ, ДЕРЕВО КВАДРАНТОВ, КЛИЕНТ-СЕРВЕРНАЯ АРХИТЕКТУРА.

Объектом исследования являются процессы взаимодействия пользователей в многопользовательских сетевых приложениях реального времени. Целью дипломного проекта является обеспечение согласованного многопользовательского игрового процесса в режиме реального времени за счёт реализации клиент-серверной архитектуры, механизмов синхронизации состояния игрового мира и физической симуляции. В ходе работы проведён анализ предметной области многопользовательских онлайн игр, архитектурных решений сетевых игровых систем и требований к программным средствам реального времени. Выполнено проектирование архитектуры, включающей клиентскую и серверную части, а также механизмы сетевого взаимодействия и синхронизации состояния. Разработана и реализована серверная часть в среде *Node.js* на языке *TypeScript* с использованием *WebSocket* для двустороннего обмена данными. Реализована клиентская часть на базе *React* и *TypeScript*, обеспечивающая визуализацию игрового состояния и обработку пользовательского ввода. В составе игрового движка реализованы модули физической симуляции и обнаружения столкновений. Проведено тестирование, подтвердившее корректность синхронизации состояния и работоспособность системы при многопользовательском взаимодействии. Выполнено технико-экономическое обоснование разработки. Рассчитаны показатели экономической эффективности: рентабельность затрат составила 23,68 %. Проект является экономически целесообразным и готовым к эксплуатации.

СОДЕРЖАНИЕ

Перечень условных обозначений	7
Введение.....	8
1 Анализ литературных источников, прототипов и формирование требований к проектируемому программному средству	10
1.1 Анализ архитектурных решений распределённых систем	10
1.2 Алгоритмы пространственной оптимизации	13
1.3 Математические методы детекции коллизий.....	15
1.4 Анализ сетевых технологий и серверных платформ.....	17
1.5 Обзор существующих аналогов.....	19
1.6 Постановка задачи	23
2 Анализ требований к ПС и разработка функциональных требований.....	27
2.1 Функциональная модель программного средства	27
2.2 Разработка информационной модели	30
2.3 Разработка спецификации функциональных требований	34
3 Проектирование программного средства	37
3.1 Разработка архитектуры программного средства	37
3.2 Разработка модели базы данных	39
3.3 Разработка алгоритмов программного средства.....	42
4 Конструирование программного средства	49
4.1 Разработка серверной части.....	49
4.2 Разработка клиентской части.....	54
4.3 Реализация хранения данных и системы повторов	58
4.4 Итоги реализации программного средства	60
5 Тестирование и проверка работоспособности программного средства.....	61
5.1 Цели, объект и методы тестирования	61
5.2 Автоматизированное тестирование серверной части	62
5.3 Ручное тестирование пользовательского интерфейса	63
6 Методика использования программного средства	71
6.1 Страница входа в аккаунт	71
6.2 Страница регистрации аккаунта.....	72
6.3 Страница главного раздела	72
6.4 Страница раздела «Игра»	73
6.5 Страница выбора танка	74
6.6 Страница комнаты	75
6.7 Страница игры.....	76
6.8 Страница окончания игры.....	77
6.9 Страница раздела «Повтор».....	77

6.10	Страница просмотра сохранённого повтора	78
6.11	Страница раздела «Друзья»	79
6.12	Страница настроек приложения	80
7	Технико-экономическое обоснование разработки программного средства многопользовательского взаимодействия в онлайн игре с физической моделью на javascript-платформе	81
7.1	Характеристика разработанного программного средства	81
7.2	Расчет затрат на разработку программного средства	82
7.3	Расчет результата от разработки и использования программного средства	86
	Заключение	90
	Список использованных источников	92
	Приложение А (обязательное) Листинг исходного кода серверной части	94
	Приложение Б (обязательное) Листинг исходного кода клиентской части	116
	Приложение В (обязательное) Результат проверки на заимствования	126

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ПС	– Программное средство. Совокупность программных компонентов, реализующих функции разрабатываемой системы.
СУБД	– Система управления базами данных. Программная система, предназначенная для создания, хранения и обработки данных в базе данных.
ФТ	– Функциональное требование. Требование, описывающее функцию или поведение программного средства.
API	– <i>Application Programming Interface</i> – интерфейс прикладного программирования, определяющий способы взаимодействия программных компонентов.
HTTP	– <i>HyperText Transfer Protocol</i> – протокол прикладного уровня для передачи данных в сети Интернет.
HTTPS	– <i>HyperText Transfer Protocol Secure</i> – защищённая версия протокола <i>HTTP</i> , использующая шифрование передаваемых данных.
JSON	– <i>JavaScript Object Notation</i> – текстовый формат обмена данными, основанный на синтаксисе <i>JavaScript</i> .
JWT	– <i>JSON Web Token</i> – формат токена, используемый для передачи данных аутентификации и авторизации пользователя.
SQL	– <i>Structured Query Language</i> – язык структурированных запросов, используемый для работы с реляционными базами данных.
TCP	– <i>Transmission Control Protocol</i> – транспортный протокол, обеспечивающий надёжную передачу данных.
UML	– <i>Unified Modeling Language</i> – язык графического моделирования для описания архитектуры программных систем.
URI	– <i>Uniform Resource Identifier</i> – унифицированный идентификатор ресурса, используемый для обозначения конечных точек приложения.
UUID	– <i>Universally Unique Identifier</i> – универсальный уникальный идентификатор, используемый для однозначной идентификации записей.

ВВЕДЕНИЕ

В последние годы наблюдается рост популярности сетевых интерактивных систем реального времени, включая многопользовательские онлайн игры. Развитие веб-технологий и современных браузеров позволило создавать игровые приложения, функционирующие непосредственно в веб-среде без установки дополнительного программного обеспечения. Такой подход обеспечивает доступность системы для широкого круга пользователей и упрощает процесс подключения к игровому процессу.

Особое место среди таких систем занимают многопользовательские игры с физической моделью, в которых состояние игрового мира непрерывно изменяется в зависимости от действий участников. Для подобных приложений критически важными являются задачи синхронизации состояния между клиентами, обработки пользовательских действий в реальном времени и обеспечения корректной физической симуляции. Нарушение согласованности между клиентом и сервером может приводить к ошибкам отображения и снижению качества игрового взаимодействия.

Современные веб-платформы предоставляют средства для реализации подобных систем, включая двусторонние сетевые протоколы обмена данными и серверные среды выполнения. Использование *JavaScript*-платформы позволяет реализовывать клиентскую и серверную части в рамках единой технологической экосистемы, что упрощает разработку и сопровождение программного средства.

Целью дипломного проекта является обеспечение согласованного многопользовательского игрового процесса в режиме реального времени за счёт реализации клиент-серверной архитектуры, механизмов синхронизации состояния игрового мира и физической симуляции.

Для достижения поставленной цели необходимо решить следующие задачи:

- исследовать предметную область многопользовательских онлайн игр и проанализировать существующие архитектурные решения для сетевых игровых систем;
- выполнить анализ требований к программному средству, обеспечивающему многопользовательское взаимодействие и синхронизацию игрового состояния;
- разработать архитектуру программного средства, включающую клиентскую и серверную части и механизмы сетевого взаимодействия;

- спроектировать структуру реляционной базы данных и реализовать её для обеспечения хранения учетных записей пользователей, игровой статистики и результатов матчей;
- выполнить программную реализацию игрового движка, включающего подсистемы физической симуляции, обнаружения столкновений и игрового цикла;
- реализовать серверную часть программного средства, обеспечивающую централизованное управление игровыми сессиями, обмен данными и синхронизацию состояния игрового мира;
- реализовать клиентскую часть программного средства с визуализацией игрового состояния и обработкой пользовательского ввода;
- провести тестирование разработанного программного средства и анализ полученных результатов;
- выполнить технико-экономическое обоснование разработки.

Объектом исследования являются процессы взаимодействия пользователей в многопользовательских сетевых приложениях реального времени.

Предметом исследования являются методы и средства реализации клиент-серверной архитектуры, обеспечивающей синхронизацию состояния игрового мира и обработку действий пользователей в многопользовательской онлайн игре на *JavaScript*-платформе.

Актуальность разработки программного средства обусловлена необходимостью обеспечения стабильного многопользовательского взаимодействия в режиме реального времени, синхронизации состояния игрового мира и корректной обработки физической симуляции в условиях клиент-серверной архитектуры.

Дипломный проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности составляет 94%. Цитирования обозначены ссылками на публикации, указанными в «Списке использованных источников».

Снимок экрана в приложении.

1 АНАЛИЗ ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ, ПРОТОТИПОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ

Перед началом проектирования высоконагруженной интерактивной системы необходимо провести детальный анализ предметной области, поскольку именно на этом этапе формируются фундаментальные решения, определяющие устойчивость, масштабируемость и корректность функционирования программного средства. Для веб-ориентированных интерактивных приложений существенное значение имеет понимание клиентских *Web API* браузера: с их помощью организуются ввод пользователя, отображение состояния сцены и обмен данными с серверной частью, то есть задаётся «контур» клиентской реализации интерактивной системы [1].

Анализ архитектурных подходов позволяет определить способ организации взаимодействия между компонентами системы, распределение вычислительной нагрузки между клиентом и сервером и механизм обеспечения согласованности отображаемого состояния. Для двумерной игровой сцены отдельное внимание уделяется методам детекции коллизий, поскольку от выбранной схемы (в том числе разделения широкой и узкой фазы проверок) зависят вычислительная сложность и корректность физического взаимодействия объектов [2]. В качестве платформы для серверной части целесообразно рассмотреть *Node.js*. Его архитектура, основанная на событийном цикле и неблокирующем вводе-выводе, позволяет эффективно обрабатывать множество одновременных соединений без существенных затрат ресурсов на управление потоками, что критично для интерактивных приложений [3].

Результатом проведённого анализа является обоснование архитектурных и технологических решений, которые будут использованы при разработке программного средства. Полученные выводы служат основой для дальнейшего проектирования системы, выбора алгоритмов и реализации серверной и клиентской частей приложения.

1.1 Анализ архитектурных решений распределённых систем

При проектировании распределённой интерактивной системы одним из ключевых этапов является выбор архитектурной модели взаимодействия между узлами. От данного решения зависят способы обмена данными, порядок обработки событий, уровень согласованности состояния системы и возможность её масштабирования. Важно различать сетевую архитектуру как

набор служб со стороны транспорта и архитектуру приложения, то есть то, как на конечных системах организованы роли участников и потоки запросов и ответов.

В современной практике разработки сетевых приложений обычно выбирают одну из двух доминирующих парадигм. В клиент-серверной модели выделяется сервер, который длительное время остается доступным и обслуживает запросы множества клиентов. При этом клиенты, как правило, не взаимодействуют друг с другом напрямую, а обмениваются данными с сервером по известному адресу сервиса. В одноранговой модели взаимодействие строится между равноправными узлами без обязательного выделенного сервера или с минимальным его участием. Для такой архитектуры типичны прямые соединения между узлами. Кроме того, в ряде приложений используется гибридная схема, где часть функций централизована, а часть трафика передается напрямую между участниками сети [4].

Принципиальные различия между подходами проиллюстрированы на рисунке 1.1.

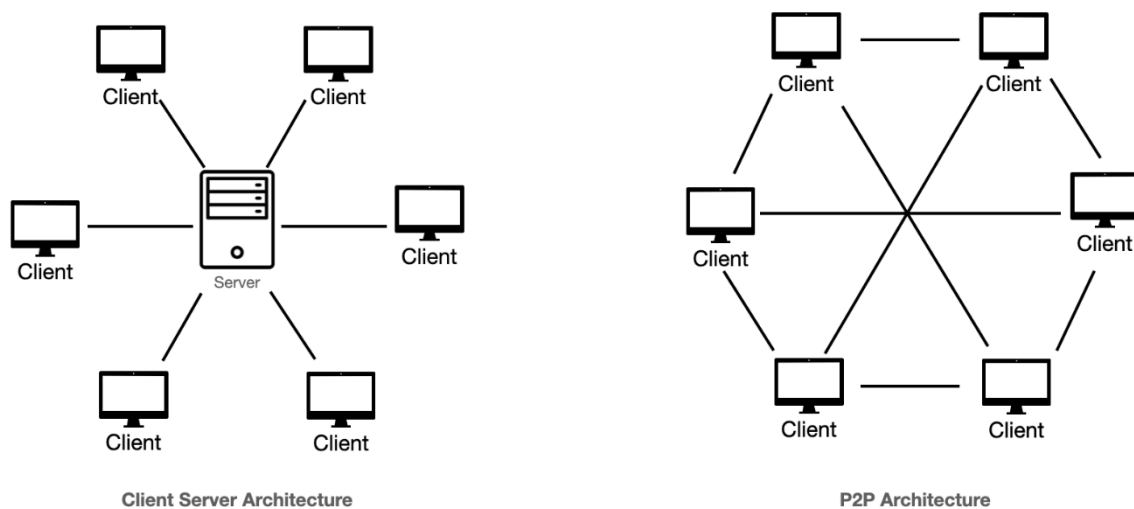


Рисунок 1.1 – Сравнение клиент-серверной и одноранговой архитектур распределённых систем

У одноранговой архитектуры есть практические преимущества: она может быть самомасштабируемой за счёт того, что узлы не только потребляют, но и предоставляют ресурсы другим участникам, а также может снижать требования к дорогостоящей централизованной серверной инфраструктуре в сценариях, где нагрузка распределена по множеству хостов.

Однако для интерактивных систем с общим изменяемым состоянием это не отменяет проблем согласования: при распределённом обновлении состояния узлы получают информацию в разное время, что приводит к расхождениям, различиям в порядке обработки событий и конфликтам интерпретации действий пользователей, особенно при нестабильной задержке.

Кроме того, у широкого класса одноранговых решений отмечаются отдельные системные ограничения, связанные с эксплуатацией в реальных сетях доступа и с повышенными требованиями к безопасности из-за открытой природы взаимодействия между множеством независимых узлов [4]. В интерактивных многопользовательских приложениях, где критична целостность мира и единый порядок применения событий, поэтому чаще выбирают клиент-серверную организацию и модель авторитарного сервера, при которой окончательное состояние определяется на сервере, а действия пользователя рассматриваются как входные команды, подлежащие проверке и упорядоченной обработке [5].

Клиент-серверная архитектура в рассматриваемом классе систем является более управляемой. Сервер принимает команды клиентов, выполняет обработку, обновляет глобальное состояние и рассылает результат участникам, что позволяет зафиксировать единый порядок применения событий и снизить риск логических противоречий. Клиентская часть ориентирована на сбор ввода, локальное отображение и обмен с сервером, серверная – на координацию, хранение авторитарного состояния и рассылку обновлений.

Даже при клиент-серверной организации существенное влияние на поведение системы оказывает транспортный уровень. Для интерактивного обмена часто используют *TCP* как протокол с установлением соединения: перед передачей данных стороны выполняют процедуру установления соединения, после чего данные передаются через созданные сокеты с привязкой к адресам и портам. Это обеспечивает надёжную упорядоченную доставку, но дополнительные этапы установления соединения и механизмы обеспечения надёжности могут увеличивать задержку в сценариях, чувствительных к времени отклика [6].

Наличие задержки делает недостаточным прямое отображение только подтверждённых сервером состояний, поскольку это формирует у пользователя ощущение инерционности управления. Для компенсации применяют интерполяцию между подтверждёнными состояниями и предсказание ввода на клиенте с последующей серверной коррекцией, что позволяет сочетать целостность глобального состояния и приемлемую отзывчивость интерфейса [7].

Таким образом, для высоконагруженных интерактивных систем с общим изменяемым состоянием наиболее обоснованным является клиент-серверный подход с централизованным вычислением и авторитарной ролью сервера. Одноранговая модель может быть эффективна в отдельных классах задач, однако в контексте требуемой согласованности и управляемости уступает по практической устойчивости. Использование клиент-серверной архитектуры в сочетании с методами компенсации сетевых задержек формирует техническую основу для дальнейшего проектирования серверной и клиентской подсистем программного средства.

1.2 Алгоритмы пространственной оптимизации

При моделировании взаимодействия большого количества объектов в интерактивной системе одной из ключевых задач является определение потенциально взаимодействующих пар. Прямой перебор всех возможных пар приводит к резкому росту числа проверок при увеличении количества объектов, поэтому на практике задачу обычно решают в два шага: сначала по простым правилам отсекают пары, а затем выполняют точные проверки только для уменьшенного набора пар [2]. Для интерактивных приложений важно, чтобы первый шаг сильно сокращал объём проверок и позволял учитывать объекты, расположенные рядом. Иначе при росте числа объектов вычислительная нагрузка на кадр становится неприемлемой.

Одним из наиболее эффективных подходов к решению данной задачи является использование дерева квадрантов. Данная структура данных представляет собой иерархическую модель разбиения двумерного пространства, при которой исходная область рекурсивно делится на четыре равные части. Каждый узел дерева соответствует определённой области пространства, а разбиение выполняется до тех пор, пока не будет достигнуто допустимое количество объектов в области или максимальная глубина структуры. При обновлении сцены дерево может перестраиваться или корректироваться, чтобы сохранить актуальное разбиение относительно новых положений объектов; глубина дерева в плотных участках возрастает сильнее, чем в разреженных, что позволяет тратить больше детализации там, где объектов больше, и тем самым снижать среднюю стоимость поиска соседей в каждом кадре [8].

Наглядная схема такого иерархического деления представлена на рисунке 1.2.

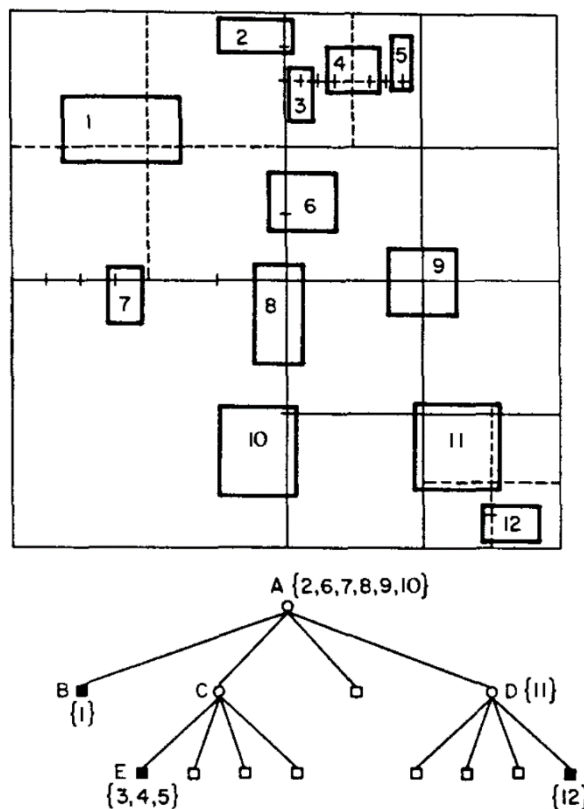


Рисунок 1.2 – Пример разбиения пространства с использованием дерева квадрантов

Принцип работы дерева квадрантов заключается в том, что объекты распределяются по узлам в зависимости от их положения. Если количество объектов в текущей области превышает установленный порог, она делится на четыре подобласти. Объекты, полностью принадлежащие одной из областей, помещаются в соответствующий дочерний узел, тогда как объекты, пересекающие границы нескольких областей, остаются на текущем уровне. Это позволяет сохранить корректность структуры и избежать избыточного дублирования данных.

При поиске потенциальных взаимодействий для заданного объекта рассматриваются только те области, которые пересекаются с областью его расположения. Если область не пересекается с рассматриваемым объектом, она исключается из дальнейшего анализа вместе со всеми вложенными узлами. Таким образом, алгоритм существенно сокращает количество проверяемых объектов и концентрирует вычисления только на локально значимых элементах.

Альтернативным методом пространственной оптимизации является регулярная сетка, в которой пространство делится на ячейки фиксированного

размера. Однако данный подход имеет ограничения, связанные с выбором размера ячейки: при неудачном подборе параметров эффективность снижается, поскольку либо увеличивается количество объектов в ячейке, либо возрастает сложность обработки объектов, пересекающих несколько ячеек [9].

В отличие от регулярной сетки, дерево квадрантов обладает адаптивной структурой. В областях с высокой плотностью объектов происходит более детальное разбиение пространства, тогда как в разреженных областях структура остаётся менее глубокой. Это обеспечивает более рациональное использование вычислительных ресурсов и делает метод особенно подходящим для динамических систем, в которых распределение объектов изменяется со временем.

Таким образом, применение дерева квадрантов позволяет существенно повысить производительность системы за счёт сокращения числа проверок взаимодействий и локализации вычислений. Это делает данный алгоритм одним из наиболее распространённых решений для пространственной оптимизации в интерактивных приложениях реального времени.

1.3 Математические методы детекции коллизий

После пространственной оптимизации, когда число проверяемых пар уже сокращено, необходимо выполнить точное определение факта пересечения объектов. На этом этапе важно не только установить факт касания, но и гарантировать корректность результата для типовых геометрических представлений, используемых в интерактивной сцене. Поэтому применяются методы, которые позволяют отличить реальное пересечение от ситуации, когда объекты находятся близко, но разделены, и при этом сохраняют предсказуемую вычислительную структуру проверки.

Для пары ориентированных ограничивающих прямоугольников точный тест пересечения строится на основе разделяющих осей. Идея состоит в том, что два таких прямоугольника разделены в пространстве, если существует ось, относительно которой проекции не перекрываются: сумма радиусов проекций меньше расстояния между проекциями центров. Для полного анализа пересечения достаточно проверить ограниченный набор направлений, что даёт возможность завершить вычисления сразу после обнаружения разделяющей оси и тем самым снизить среднюю стоимость теста в типовых сценариях [10].

Для выпуклых объектов широко применяется теорема о разделяющей оси. Она утверждает, что два выпуклых множества не пересекаются, если существует ось, на которую ортогональные проекции объектов не

перекрываются. Если же для полного набора проверяемых направлений перекрытие проекций сохраняется, делается вывод о пересечении. На практике набор осей выбирают из геометрии объектов и из взаимного расположения примитивов, а проверка выполняется последовательно с возможностью раннего выхода. Для невыпуклых фигур такой подход напрямую не всегда применим, поэтому используют разбиение на выпуклые части, выпуклые аппроксимации или альтернативные методы [11].

Визуально принцип разделяющей оси поясняется на рисунке 1.3. На нём показаны два выпуклых объекта и выбранное направление, относительно которого проекции разнесены. Это означает, что существует прямая, перпендикулярная данной оси, которая разделяет объекты, и следовательно они не имеют общих точек.

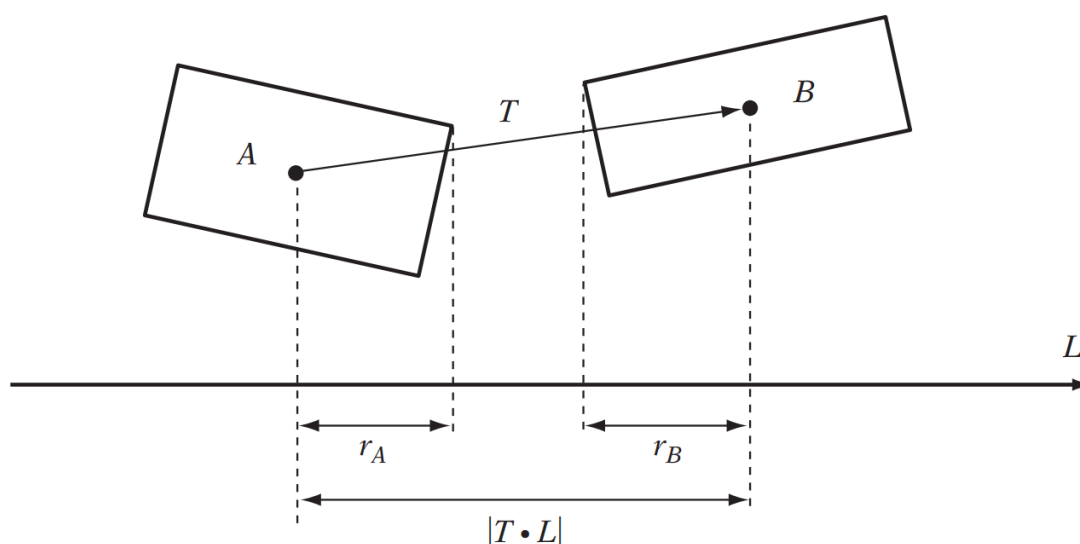


Рисунок 1.3 – Иллюстрация теоремы о разделяющих осях для двух выпуклых объектов

В двумерном случае для многоугольников набор направлений обычно строят из геометрии границ, как правило по нормальям к рёбрам. Для выбранной оси вершины проецируют через скалярное произведение с направлением оси, затем для каждого объекта находят интервал от минимальной до максимальной проекции. Если хотя бы для одной оси интервалы не пересекаются, пересечения нет, и дальнейшие проверки можно прекратить [12].

В двумерном случае для многоугольников набор направлений обычно строят из геометрии границ, как правило по нормальям к рёбрам. Для

выбранной оси каждая вершина проецируется через скалярное произведение с единичным направлением оси, после чего для каждого объекта определяется интервал от минимальной до максимальной проекции. Далее проверяется пересечение интервалов на оси. Если хотя бы для одной оси интервалы не пересекаются, пересечения объектов нет, и дальнейший перебор направлений для данной пары можно прекратить. Если пересечение интервалов сохраняется для всех требуемых направлений, фиксируется факт коллизии [12].

Таким образом, проверка пересечения сводится к последовательности сравнений одномерных интервалов на проекциях, что упрощает реализацию и отладку алгоритма в интерактивной системе. Указанный подход хорошо сочетается с практическими тестами для типовых ограничивающих объёмов, в том числе с тестом пересечения пары ориентированных ограничивающих прямоугольников, где важны как корректность результата, так и возможность раннего завершения вычислений.

1.4 Анализ сетевых технологий и серверных платформ

При разработке распределённой интерактивной системы важно согласованно выбрать механизм прикладного обмена данными между клиентом и сервером и определить серверную платформу, на которой будут обрабатываться соединения, сообщения и прикладная логика. От этого зависят задержки доставки, доля служебных данных, устойчивость к пиковым нагрузкам и сложность реализации серверной части. Для сценариев с большим числом одновременных подключений и преобладанием операций ввода-вывода распространённым выбором является платформа *Node.js* как среда выполнения *JavaScript* на стороне сервера с событийно-ориентированной моделью обработки [3].

В интерактивных системах с частым обновлением состояния классический *HTTP* становится неудобной основой, если имитировать непрерывный поток событий только серией отдельных запросов. Модель «запрос-ответ» предполагает инициацию обмена со стороны клиента, получение ответа сервера и типичный цикл с заголовками и метаданными на каждое обращение. При высокой частоте обновлений это приводит к росту числа обращений, увеличению накладных расходов и дополнительным задержкам, связанным с регулярным опросом, в том числе когда изменения возникают редко [13].

Чтобы уменьшить нагрузку от частых опросов, применяют *long polling*: запрос удерживается на сервере до появления данных, после чего клиент сразу

открывает следующий запрос. Подход уменьшает число бесполезных ответов по сравнению с коротким периодическим опросом, однако сохраняет ограничения модели *HTTP* и не даёт полноценного симметричного канала, в котором сервер может инициировать передачу в произвольный момент без нового *HTTP*-запроса со стороны клиента [14]. Принцип работы проиллюстрирован на рисунке 1.4.

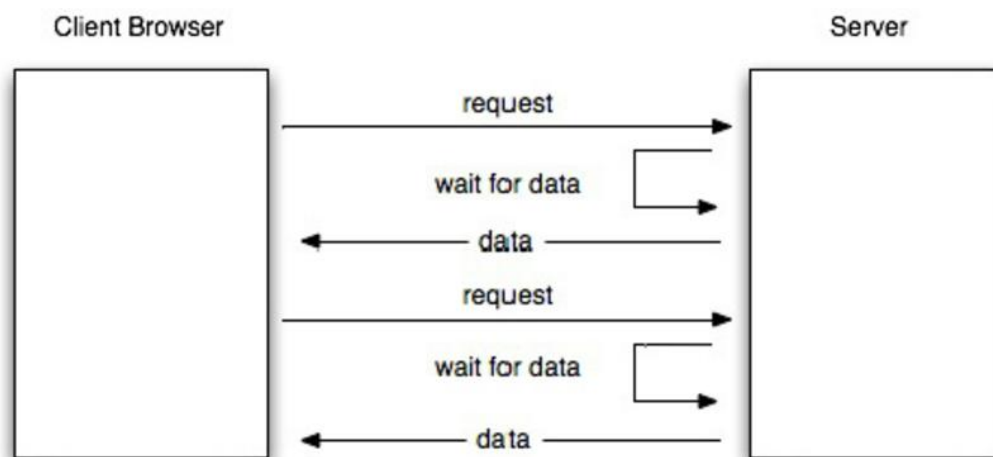


Рисунок 1.4 – Схема взаимодействия при использовании технологии *long polling*

Для постоянного двунаправленного обмена в типичных веб-клиентах целесообразнее использовать *WebSocket*. Инициализация выполняется поверх *HTTP* с последующим переключением протокола, после чего устанавливается долгоживущее соединение, в рамках которого сообщения передаются кадрами с относительно компактной служебной частью. Канал остаётся открытым, что снижает накладные расходы, связанные с повторным установлением соединения при каждом обновлении, и делает задержку доставки более предсказуемой при высокой частоте событий. Сервер также может инициировать отправку данных клиенту без ожидания нового *HTTP*-запроса, что важно для оперативного распространения изменений состояния [15].

Сопоставляя подходы, можно отметить следующее. *HTTP* остаётся естественной базой для нерегулярных запросов и выдачи документов и статических ресурсов. *Long polling* частично улучшает сценарий опроса, но сохраняет ограничения, вытекающие из модели запрос/ответ. *WebSocket* обеспечивает устойчивый двунаправленный канал и лучше соответствует требованиям интерактивных систем с высокой частотой сообщений при

сохранении совместимости инициализации с существующей *HTTP*-инфраструктурой.

Таким образом, для распределённой интерактивной системы с постоянным обменом состоянием разумной базой является связка *Node.js* как серверной платформы для обработки множества соединений и *WebSocket* как основного транспорта прикладного обмена между клиентом и сервером. Такое сочетание снижает избыточность сетевого обмена по сравнению с частым *HTTP*-опросом и упрощает реализацию серверной части при высокой интенсивности событий.

1.5 Обзор существующих аналогов

1.5.1 *Dier.io* – браузерная многопользовательская игра-шутер с видом сверху, действие которой разворачивается на общей карте. Игра запускается в веб-браузере и не требует установки отдельного клиента.

Игрок управляет танком, перемещается по двумерной сцене, ведёт огонь по другим игрокам и нейтральным объектам и уничтожает цели; за уничтожение начисляется опыт, который затрачивается на улучшение характеристик танка. Поддерживается режим реального времени, в котором состояние игрового мира согласуется между участниками посредством серверной обработки [16].

Интерфейс игры представлен на рисунке 1.5.

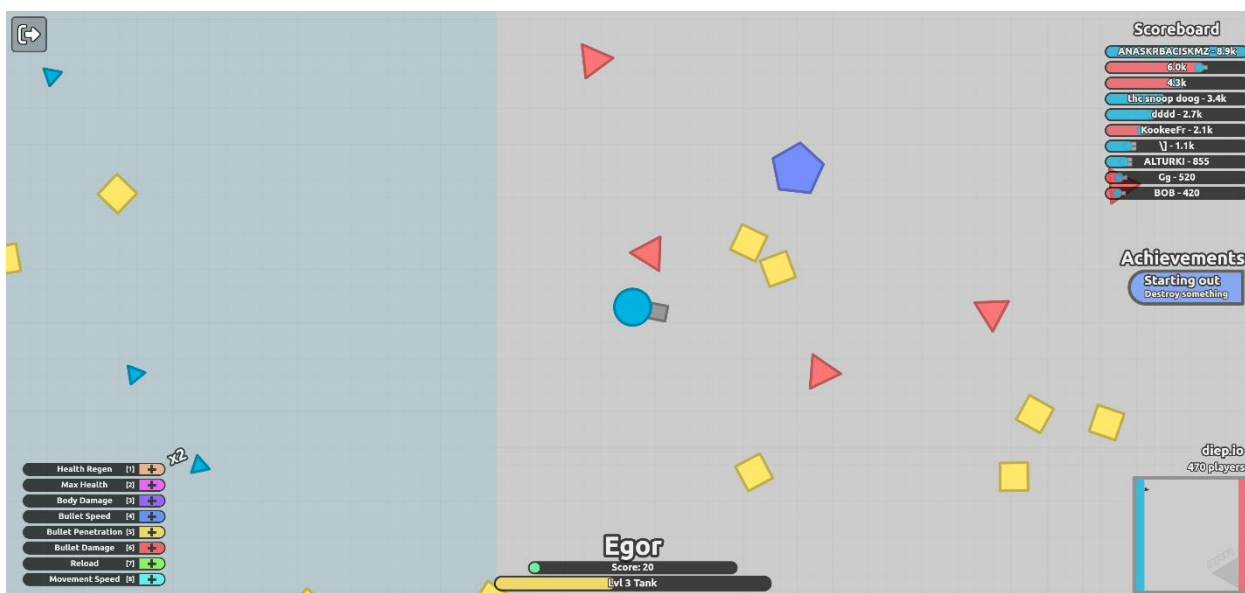


Рисунок 1.5 – Интерфейс игры *Dier.io*

К преимуществам *Dier.io* относится простая и понятная игровая механика: управление объектом и базовые правила развития осваиваются практически сразу. Постоянное взаимодействие с другими игроками делает игровой процесс динамичным, а система опыта и улучшений формирует наглядный цикл обратной связи. Минималистичная графика снижает нагрузку на клиентскую часть и позволяет поддерживать стабильную производительность даже при большом количестве объектов на сцене. Дополнительно следует отметить ориентацию игры на серверную синхронизацию состояния, благодаря чему игровой мир воспринимается участниками как единое согласованное пространство.

К недостаткам можно отнести закрытую архитектуру решения: по доступным материалам невозможно детально восстановить внутренние механизмы синхронизации, порядок обработки пользовательских действий и методы компенсации сетевых задержек. Физическая модель также реализована в упрощённом виде и не ориентирована на сложные взаимодействия объектов с окружением, расширяемые типы коллизий и глубокую настройку параметров среды в условиях многопользовательской игровой симуляции реального времени. Это ограничивает применение игры как полноценного инженерного эталона при разработке собственного программного средства.

Таким образом, *Dier.io* можно рассматривать как пример браузерной многопользовательской 2D-игры с базовой физикой движения и синхронизацией состояния в реальном времени. Для проектируемого программного средства данный аналог полезен прежде всего как ориентир по организации игрового цикла и взаимодействию большого количества игроков в едином игровом пространстве, позволяя проанализировать баланс между простотой реализации и динамикой сетевого игрового процесса в браузерной среде.

1.5.2 Tank Trouble – двумерная браузерная игра с видом сверху, в которой пользователи управляют танками на ограниченной арене с лабиринтом стен. Игра запускается в веб-браузере и не требует установки отдельного клиента.

В ходе матча необходимо перемещаться по карте, уклоняться от снарядов противника и уничтожать соперников. Столкновения танков и снарядов со стенами и другими элементами арены реализуют простую физику движения и отскоков. Поддерживается многопользовательский режим, состояние боя ведётся в реальном времени с участием нескольких игроков [17].

Интерфейс игры представлен на рисунке 1.6.

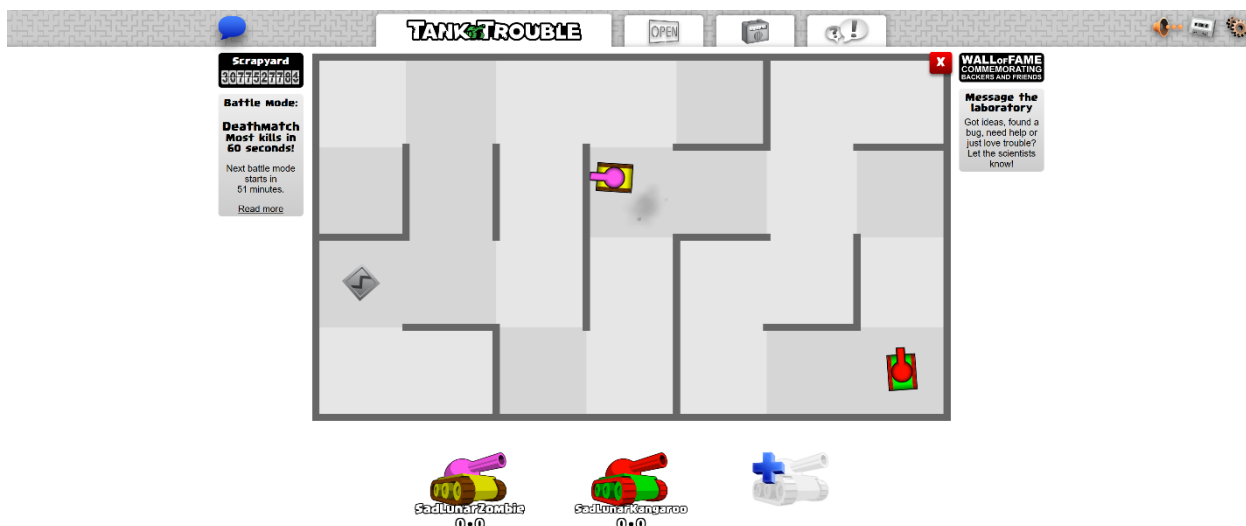


Рисунок 1.6 – Интерфейс игры *Tank Trouble*

К преимуществам *Tank Trouble* относится понятная механика и быстрый старт: правила ясны с первых минут, управление танком осваивается без длительного обучения. Наличие препятствий на арене делает бой более стратегическим по сравнению с открытой плоскостью: игрок вынужден учитывать геометрию лабиринта и направление отскоков снарядов. Многопользовательский режим позволяет организовать соревнование на одной сцене, а браузерный запуск упрощает доступ к игре.

К недостаткам можно отнести ограниченный размер арены и сравнительно узкий набор игровых ситуаций: при длительных сессиях это снижает разнообразие сценариев. Физическая модель остаётся базовой и не ориентирована на сложные взаимодействия между множеством типов объектов и расширяемые правила симуляции. Кроме того, решение закрытое: по открытым материалам нельзя детально восстановить сетевую архитектуру, порядок синхронизации состояния и обработку команд, что ограничивает использование игры как инженерного эталона при проектировании собственной системы.

Таким образом, *Tank Trouble* удобно рассматривать как пример простой многопользовательской 2D-игры с лабиринтом, базовой физикой столкновений и браузерным доступом; для проектируемого программного средства она полезна как ориентир по игровому циклу и по взаимодействию геометрии лабиринта и баллистики, позволяя эффективно проанализировать баланс между лаконичностью дизайна и динамикой игровых сессий.

1.5.3 ShellShock Live представляет собой многопользовательскую артиллерийскую игру, в которой участники поочерёдно производят выстрелы,

задавая угол орудия и силу заряда. Клиентская часть распространяется как отдельное приложение через цифровые площадки и не относится к классу браузерных решений.

Снаряд движется по баллистической траектории, взаимодействует с рельефом и препятствиями и при попадании наносит урон. Геометрия арены определяет необходимость учёта направления выстрела, дальности и возможных отражений. Поддерживается сетевой режим на нескольких игроках [18].

Интерфейс игры представлен на рисунке 1.7.

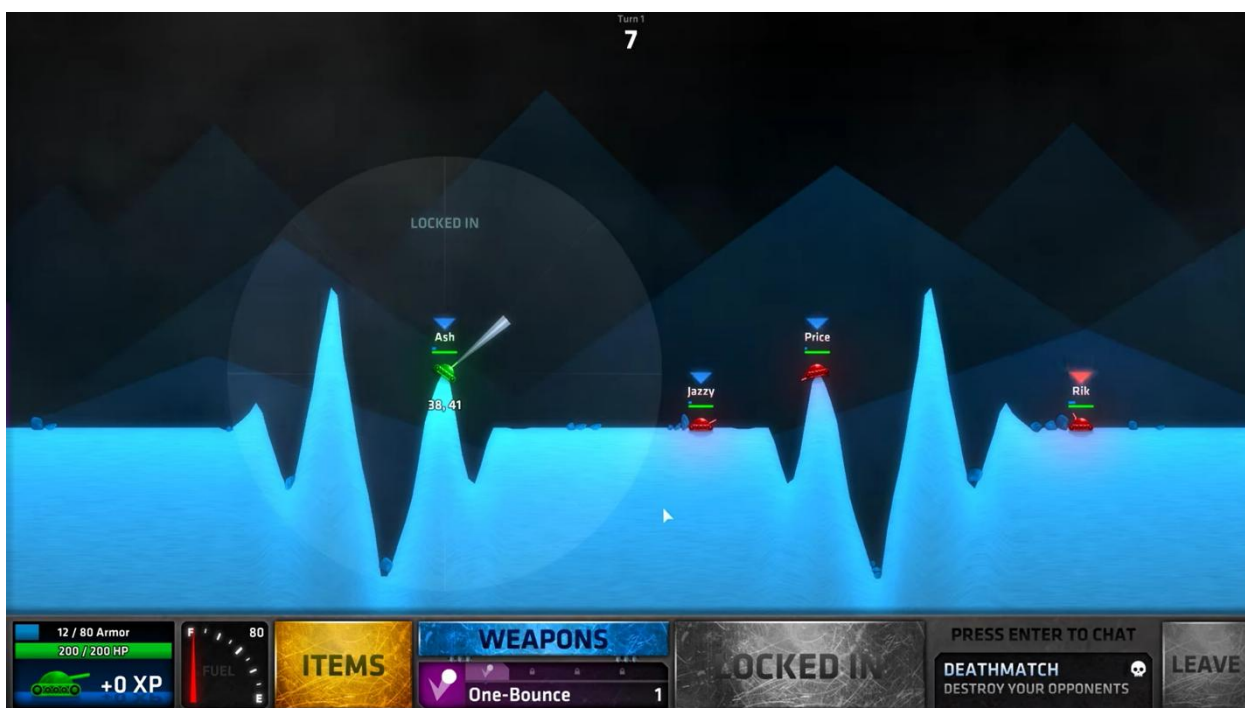


Рисунок 1.7 – Интерфейс игры *ShellShock Live*

К преимуществам *ShellShock Live* относится явная формализация параметров выстрела, обеспечивающая прослеживаемую связь между углом, силой заряда и формой траектории. Наличие препятствий и отражений снаряда от элементов окружения повышает значимость геометрии карты и корректной обработки столкновений. Многопользовательский режим демонстрирует соревновательное взаимодействие при общем состоянии боя и поочерёдном порядке ходов.

К недостаткам при сопоставлении с проектируемым браузерным программным средством относится отличие канала поставки клиента: основной сценарий запуска не предполагает работу в веб-браузере, что ограничивает сопоставимость по клиентскому стеку и модели доставки

обновлений интерфейса. Кроме того, архитектура и сетевой протокол являются закрытыми, что не позволяет на уровне пояснительной записки привести воспроизводимое описание механизмов синхронизации и правил симуляции. Также темп игры определяется пошаговой моделью артиллерийского боя и отличается от непрерывного режима реального времени, характерного для рассмотренных ранее браузерных примеров.

Таким образом, *ShellShock Live* целесообразно рассматривать как дополнительный аналог по разделу баллистики снаряда, геометрии сцены и обработке столкновений снаряда с элементами окружения при условии, что в требованиях к проектируемой системе отдельно фиксируются траектории и влияние рельефа. При этом ограничения по платформе клиента и по доступности внутренней реализации должны быть явно отражены в сравнительных выводах раздела.

1.6 Постановка задачи

На основе проведённого анализа архитектурных решений распределённых систем, методов пространственной оптимизации и детекции коллизий, сетевых технологий и существующих аналогов сформулированы назначение, функциональный состав, состав входных и выходных данных, а также требования к программно-техническим средствам и совместимости разрабатываемого программного средства.

1.6.1 Проектируемое программное средство предназначено для организации многопользовательского взаимодействия в онлайн игре с физической моделью, функционирующей в веб-среде.

Основное назначение разработки заключается в обеспечении согласованного взаимодействия пользователей в режиме реального времени на основе клиент-серверной архитектуры. Сервер выполняет централизованную обработку игровых событий и поддерживает единое состояние игрового мира; клиентская часть обеспечивает приём пользовательского ввода, отображение состояния и сетевой обмен с сервером.

Решаемый комплекс задач включает синхронизацию состояния игрового пространства между участниками, обработку пользовательских действий, моделирование перемещений и столкновений объектов в двумерной сцене, а также передачу данных по сети с учётом требований к задержке и устойчивости соединения.

Для практической реализации указанного назначения программное средство должно поддерживать полный цикл сетевой игровой сессии: от

установления соединения и регистрации участников до выполнения симуляции, рассылки состояния и завершения матча. Ниже приведён перечень функций, достаточный для описания границ функциональности на уровне пояснительной записки.

1.6.2 Программное средство должно обеспечивать выполнение следующих функций:

- установление соединения между клиентским приложением и сервером и инициализацию пользовательской игровой сессии;
- управление игровыми сессиями, включая создание, подключение пользователей, поддержку и завершение сессии;
- приём, передача и обработку пользовательских управляющих воздействий;
- моделирование движения игровых объектов в двумерном пространстве с учётом параметров физического взаимодействия;
- обнаружение и обработку столкновений игровых объектов;
- выполнение игрового цикла с фиксированным шагом времени и последовательной обработкой событий;
- формирование актуального состояния игрового мира на сервере;
- синхронизацию состояния игрового мира между всеми подключёнными клиентами;
- передачу данных между клиентом и сервером по постоянному сетевому соединению;
- визуализацию текущего состояния игрового мира на стороне клиента;
- обработку событий подключения, отключения и повторного подключения пользователей;
- определение условий завершения игровой сессии и формирование итогового результата.

1.6.3 В процессе работы система непрерывно получает от клиентов и пользователей сообщения, по которым определяются действия в игровом мире, параметры сессии и состояние соединений. Для однозначного описания программного средства входной поток целесообразно классифицировать по типам данных, перечисленным ниже.

К входным данным относятся:

- 1) данные, характеризующие пользователя и его подключение:
 - а) идентификатор пользователя или игровой сессии;
 - б) параметры сетевого подключения;
- 2) управляющие воздействия пользователя:

- а) команды перемещения игрового объекта;
 - б) команды выполнения игровых действий;
 - в) команды взаимодействия с игровыми объектами и игровым пространством;
- 3) данные, определяющие параметры игровой сессии:
- а) идентификатор игровой комнаты;
 - б) состав участников игровой сессии;
 - в) начальные параметры игрового мира;
- 4) сетевые сообщения, поступающие от клиента:
- а) события управления игровым объектом;
 - б) события подключения, отключения и восстановления соединения;
 - в) служебные сигналы, характеризующие состояние соединения.

1.6.4 Результатом работы серверной логики и симуляции является согласованное состояние игрового мира и сопутствующие сведения, которые должны быть доставлены клиентам для отображения, а также зафиксированы для завершения сессии. Перечень выходных данных приведён ниже и используется для согласования интерфейса обмена между подсистемами.

К выходным данным относятся:

- актуальное состояние игрового мира, включающее координаты, параметры и состояния игровых объектов;
- данные, передаваемые клиентским приложениям для визуализации игрового процесса;
- результаты игровой сессии, включая информацию о завершении матча и его исходе;
- уведомления о состоянии соединения и результатах обработки пользовательских действий;
- события игрового процесса, возникающие в ходе выполнения симуляции.

1.6.5 Устойчивое функционирование распределённой игры в веб-среде определяется не только алгоритмами, но и минимально допустимым составом средств на стороне клиента и сервера. Ниже зафиксированы требования к технической базе и к стеку разработки, выбранному для реализации.

Для функционирования программного средства необходимо наличие следующих технических и программных средств.

Клиентская часть должна выполняться на пользовательском устройстве, оснащённом современным веб-браузером, поддерживающим технологии,

необходимые для работы интерактивных веб-приложений, включая двусторонний сетевой обмен данными и средства графической отрисовки.

Серверная часть должна функционировать на вычислительной системе, обеспечивающей выполнение среды *Node.js* и способной обрабатывать множественные сетевые соединения в режиме реального времени.

Программная реализация должна включать:

- использование языка программирования *TypeScript* для разработки клиентской и серверной частей;
- применение библиотеки *React* для построения пользовательского интерфейса;
- использование платформы *Node.js* для реализации серверной логики;
- применение протокола *WebSocket* для организации постоянного двустороннего обмена данными между клиентом и сервером.

1.6.6 При эксплуатации в различных средах необходимо исключить противоречия в интерпретации сообщений и обеспечить возможность развития системы без полной переработки базовых механизмов. Этому соответствуют требования к информационной и программной совместимости, сформулированные ниже.

Программное средство должно обеспечивать совместимость и корректное взаимодействие всех компонентов системы.

Система должна функционировать в различных операционных системах при использовании современных веб-браузеров, поддерживающих необходимые стандарты веб-технологий.

Передача данных между клиентской и серверной частями должна осуществляться с использованием унифицированного формата, обеспечивающего однозначную интерпретацию сообщений обеими сторонами.

Архитектура программного средства должна обеспечивать согласованность взаимодействия компонентов, независимость клиентской и серверной реализации, а также возможность расширения функциональности без изменения базовых принципов работы системы.

2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Функциональная модель программного средства

Для представления функциональной модели разрабатываемого программного средства используется диаграмма вариантов использования *UML*. Она применяется для описания системы на концептуальном уровне, позволяет определить доступные пользователю функции и отразить взаимодействие с внешними пользователями. Диаграмма включает акторов, прецеденты и связи между ними, что наглядно демонстрирует функциональность системы и границы её взаимодействия с внешней средой [19].

Основными элементами модели являются актёры и варианты использования. Актёр представляет внешнюю по отношению к системе роль, инициирующую или потребляющую результат работы системы; вариант использования описывает цельное действие, приносящее наблюдаемый результат актёру. Отношения между вариантами использования, в частности включение и расширение, задают повторное использование описаний и развитие базового сценария без дублирования текста. Диаграмма вариантов использования *UML* отображает актёров, варианты использования и отношения, включая участие актёров и включение прецедентов [19].

В разрабатываемом программном средстве выделяется один актор – пользователь. Пользователь взаимодействует с системой посредством клиентского приложения: проходит аутентификацию, подключается к игровому серверу, организует и участвует в игровых сессиях, управляет игровым объектом, при необходимости наблюдает за чужими матчами, работает со списком друзей и приглашениями, просматривает историю матчей и повторы, а также получает информацию об итогах игры.

Функциональные возможности системы на диаграмме сгруппированы в семь логических блоков:

1 Повторы и история матчей объединяют просмотр истории матчей и просмотр сохранённого повтора. Просмотр повтора на диаграмме связан с прецедентом истории: повтор рассматривается как уточнение сценария работы с прошедшими матчами после ознакомления со списком или карточкой матча в истории. Блок отражает анализ завершённых игр вне текущей активной сессии.

2 Доступ к системе и управление профилем включают регистрацию, вход в систему и выход из системы, а также просмотр и изменение профиля.

Изменение профиля опирается на сценарий просмотра как на предварительное ознакомление с данными. Выход из системы сопровождается завершением сетевого взаимодействия с игровым сервером; на диаграмме это связано с прецедентом отключения от игрового сервера.

3 Организация игровой сессии описывает подключение к игровому серверу, создание и покидание игровой комнаты, подключение к комнате и отключение от сервера. Прецедент подключения к игровому серверу не связан с актором напрямую: он выступает общим шагом для сценариев, которым нужно установленное соединение (создание комнаты, подключение к комнате, принятие приглашения, наблюдение за игрой). Создание комнаты и подключение к комнате предполагают этот шаг. Покидание комнаты не означает автоматического разрыва соединения с сервером. Отключение от игрового сервера выделено отдельно и соответствует полному завершению работы с приложением.

4 Игровое взаимодействие включает в себя визуализацию процесса, получение состояния игрового мира, выполнение игровых действий и управление игровым объектом. Визуализация предполагает отображение состояния мира, а игровые действия связаны с управлением объектом. Так показана связь ввода, отображения и состояния мира на сервере.

5 Итоги матча включают просмотр результата матча и просмотр статистики. Статистика может дополнять сценарий ознакомления с результатом как необязательное углубление после завершения матча.

6 Друзья и приглашение в игру описывают управление списком друзей, поиск пользователя, добавление в друзья, отправку приглашения в комнату, принятие и отклонение приглашения. Добавление в друзья связано со сценарием поиска пользователя как с логическим продолжением после нахождения нужной учётной записи. Отправка приглашения в комнату на диаграмме опирается на прецедент подключения к игровой комнате: приглашение исходит от участника, который уже связан с комнатой. Принятие приглашения опирается на подключение к игровому серверу как на необходимое условие сетевого ответа на приглашение. Отклонение приглашения задаёт альтернативный сценарий без входа в комнату по приглашению.

7 Наблюдение за игрой включает в себя просмотр списка комнат для наблюдения и наблюдение за игрой. Наблюдение связано с просмотром списка и с подключением к игровому серверу для получения игровых обновлений в роли зрителя.

Диаграмма вариантов использования представлена на рисунке 2.1:

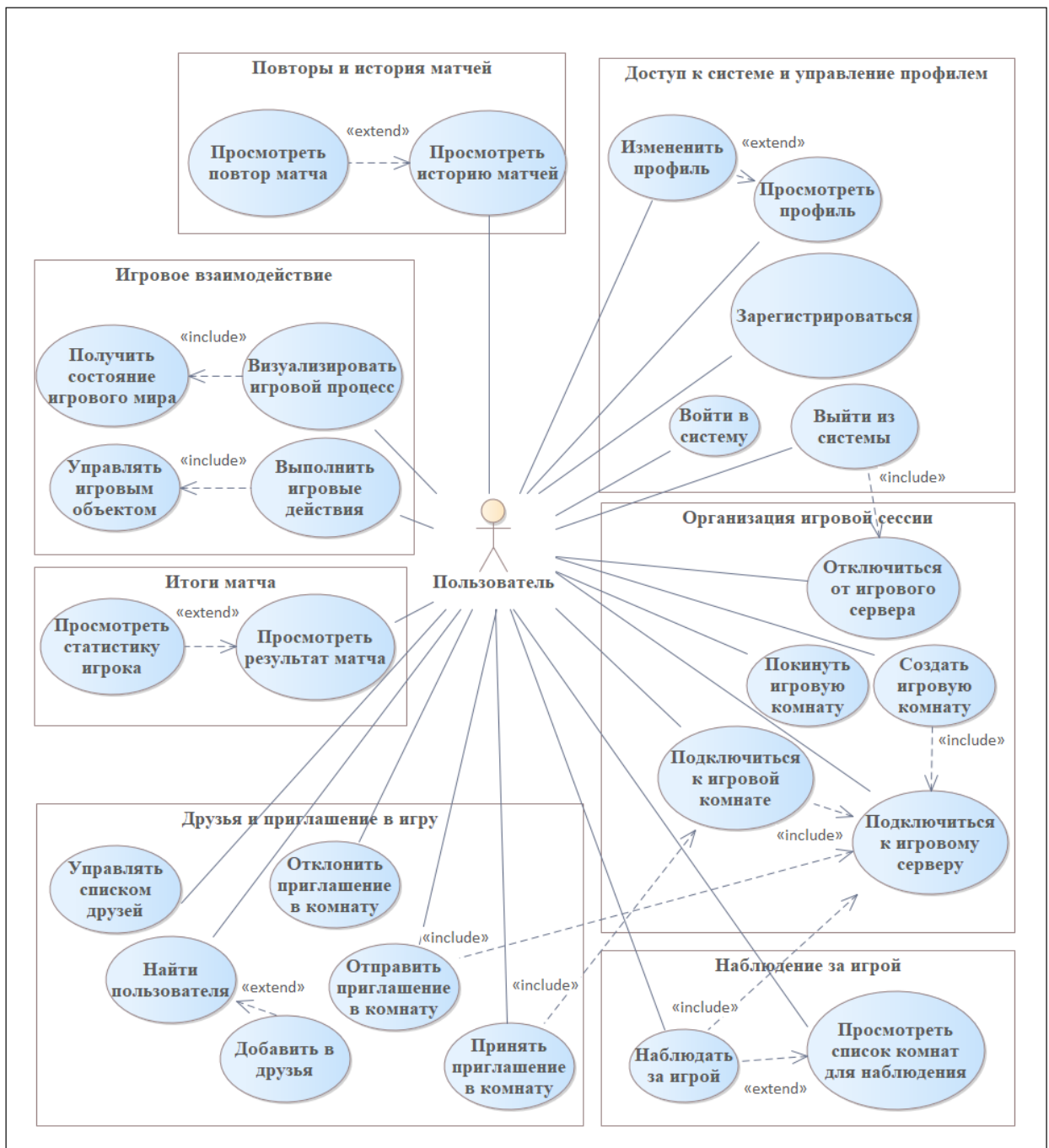


Рисунок 2.1 – Диаграмма вариантов использования *Use-Case*

Таким образом, обновлённая диаграмма вариантов использования отражает основной набор ключевых функций программного средства: доступ и профиль, организация сессии и игровое взаимодействие, социальные функции и приглашения, наблюдение, работа с историей и повторами, а также получение итогов матча. Она задаёт целостное представление о границах системы и типовых пользовательских целях, что облегчает проверку полноты заявленного функционала и согласование ожиданий с проектируемыми

возможностями продукта. Полученная функциональная модель служит основой для дальнейшей детализации функциональных требований и проектирования архитектуры программного средства, в том числе при декомпозиции системы на клиентские и серверные подсистемы и выделении каналов взаимодействия между ними.

2.2 Разработка информационной модели

При проектировании программного средства для многопользовательского взаимодействия в онлайн игре с физической моделью выбрана реляционная модель данных, обеспечивающая структурированное хранение сведений о пользователях, игровых сессиях, участниках матчей, рейтингах и повторах. Данные представляются в виде взаимосвязанных таблиц, каждая из которых соответствует отдельной сущности предметной области. Такой подход упрощает сопровождение данных, повышает согласованность при обновлениях и облегчает выполнение типовых операций выборки и анализа.

Логическое проектирование предполагает построение модели, отражающей используемые в приложении данные, независимо от конкретной СУБД и физических особенностей хранения. На этом этапе определяются сущности, атрибуты и связи, задаются ключи, обеспечивающие уникальную идентификацию записей, а также выполняется анализ избыточности и нормализация отношений с целью снижения дублирования и противоречивости. Дополнительно проверяется соответствие модели ожидаемым пользовательским операциям и ограничениям целостности, что снижает риск появления неполных или несогласованных состояний при эксплуатации [20].

Указанные принципы задают основу для последующего перехода к физической схеме базы данных в выбранной СУБД и для реализации доступа к данным на стороне сервера приложения. Разработанная информационная модель отражает основные сущности предметной области и связи между ними, обеспечивая хранение данных. Использование реляционного подхода позволяет поддерживать целостность и согласованность данных, уменьшать избыточность хранения информации и обеспечивать возможность дальнейшего расширения функциональности программного средства без изменения базовой структуры модели.

Таким образом, с учётом функциональной модели программного средства была разработана инфологическая модель базы данных, представленная на рисунке 2.2.

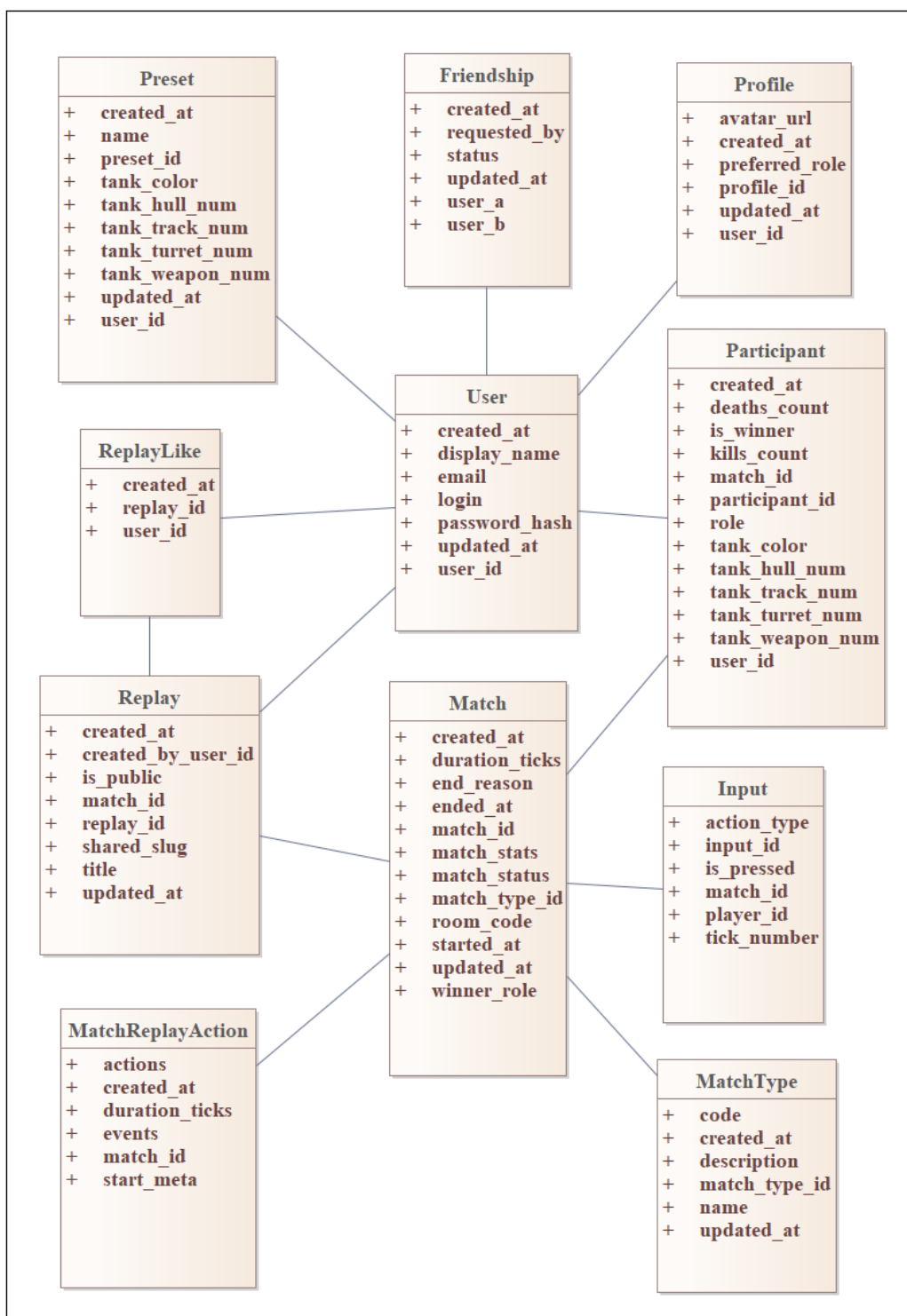


Рисунок 2.2 – Инфологическая модель базы данных

В процессе анализа предметной области были выделены следующие типы сущностей:

1) *User* – сущность, представляющая учётную запись пользователя в системе:

а) *user_id* – идентификатор пользователя;

- б) *login* – логин пользователя;
 - в) *email* – адрес электронной почты;
 - г) *password_hash* – хеш пароля;
 - д) *display_name* – отображаемое имя игрока;
 - е) *created_at* – дата и время регистрации;
 - ж) *updated_at* – дата и время последнего изменения записи;
- 2) *Profile* – сущность, представляющая расширенные данные профиля пользователя:
- а) *profile_id* – идентификатор профиля;
 - б) *user_id* – идентификатор пользователя;
 - в) *avatar_url* – ссылка на изображение, отображаемое в профиле пользователя;
 - г)
 - д) *preferred_role* – предпочитаемая игровая роль;
 - е) *created_at* – дата создания профиля;
 - ж) *updated_at* – дата последнего обновления профиля;
- 3) *MatchType* – сущность, представляющая тип матча:
- а) *match_type_id* – идентификатор типа матча;
 - б) *code* – код типа;
 - в) *name* – название типа;
 - г) *description* – описание;
 - д) *created_at* – дата создания записи;
 - е) *updated_at* – дата последнего обновления записи;
- 4) *Match* – сущность, представляющая игровую сессию:
- а) *match_id* – идентификатор матча;
 - б) *match_type_id* – идентификатор типа матча;
 - в) *room_code* – код игровой комнаты;
 - г) *match_status* – статус матча;
 - д) *winner_role* – победившая сторона;
 - е) *end_reason* – причина завершения;
 - ж) *duration_ticks* – длительность матча;
 - з) *match_stats* – статистика матча;
 - и) *started_at* – время начала;
 - к) *ended_at* – время окончания;
 - л) *created_at* – дата создания записи;
 - м) *updated_at* – дата последнего обновления записи;
- 5) *Participant* – сущность, представляющая участие пользователя в матче:
- а) *participant_id* – идентификатор участника;

- б) *match_id* – идентификатор матча;
 - в) *user_id* – идентификатор пользователя;
 - г) *role* – роль участника;
 - д) *tank_color* – цвет танка;
 - е) *tank_hull_num* – номер корпуса;
 - ж) *tank_track_num* – номер гусениц;
 - з) *tank_turret_num* – номер башни;
 - и) *tank_weapon_num* – номер оружия;
 - к) *kills_count* – количество уничтожений;
 - л) *deaths_count* – количество смертей;
 - м) *is_winner* – признак победителя;
 - н) *created_at* – дата создания записи;
- 6) *Input* – сущность, представляющая пользовательские действия:
- а) *input_id* – идентификатор записи;
 - б) *match_id* – идентификатор матча;
 - в) *tick_number* – номер игрового тика;
 - г) *player_id* – идентификатор игрока в рамках матча;
 - д) *action_type* – тип действия;
 - е) *is_pressed* – состояние действия;
- 7) *Replay* – сущность, представляющая сохранённый матч:
- а) *replay_id* – идентификатор записи;
 - б) *match_id* – идентификатор матча;
 - в) *created_by_user_id* – пользователь, создавший запись;
 - г) *title* – название;
 - д) *is_public* – признак доступности;
 - е) *shared_slug* – публичный идентификатор для ссылки;
 - ж) *created_at* – дата создания;
 - з) *updated_at* – дата обновления;
- 8) *MatchReplayActions* – сущность, представляющая данные повтора в виде действий и событий матча:
- а) *match_id* – идентификатор матча;
 - б) *start_meta* – метаданные старта;
 - в) *actions* – набор действий игроков по тикам;
 - г) *duration_ticks* – длительность матча в тиках;
 - д) *events* – журнал событий мира матча;
 - е) *created_at* – дата создания записи;
- 9) *Preset* – сущность, представляющая сохранённый пользователем пресет сборки танка:
- а) *preset_id* – идентификатор пресета;

- б) *user_id* – идентификатор пользователя;
- в) *name* – название пресета;
- г) *tank_color* – цвет танка;
- д) *tank_hull_num* – номер корпуса;
- е) *tank_track_num* – номер гусениц;
- ж) *tank_turret_num* – номер башни;
- з) *tank_weapon_num* – номер оружия;
- и) *created_at* – дата создания;
- к) *updated_at* – дата обновления;

10) *Friendship* – сущность, представляющая социальные связи между пользователями:

- а) *user_a* – идентификатор первого пользователя в паре;
- б) *user_b* – идентификатор второго пользователя в паре;
- в) *status* – состояние связи;
- г) *requested_by* – идентификатор пользователя, инициировавшего действие;
- д) *created_at* – дата создания записи;
- е) *updated_at* – дата последнего изменения записи;

11) *ReplayLike* – сущность, представляющая лайк пользователя на повтор:

- а) *replay_id* – идентификатор повтора;
- б) *user_id* – идентификатор пользователя;
- в) *created_at* – дата постановки лайка;

Разработанная совокупность сущностей задаёт структуру предметной области многопользовательской онлайн игры и охватывает ключевые элементы учёта пользователей, организации матчей, фиксации участия и сохранения данных повторов. Связи между сущностями определяют взаимозависимости данных и обеспечивают согласованное описание жизненного цикла матча на логическом уровне.

Таким образом, информационная модель обеспечивает логическую целостность и снижает избыточность данных. Она служит основой для дальнейшего уточнения требований к данным в последующих этапах проектирования.

2.3 Разработка спецификации функциональных требований

На основе укрупнённых требований технического задания и разработанной функциональной модели сформирована спецификация функциональных требований к программному средству. Спецификация задаёт

ожидаемое поведение системы в терминах пользовательских целей и поддерживает единый язык описания между заказчиком, разработчиком и проверкой результатов. Функциональные требования отражают основные возможности системы и определяют поведение программного средства при взаимодействии с пользователем и обработке игровых процессов.

В соответствии с принятой классификацией функциональные требования формулируют предоставляемые системой услуги, ожидаемую реакцию на входные воздействия и поведение в типовых ситуациях. Отдельные требования могут явно ограничивать недопустимое поведение. На уровне системных требований функциональность уточняется до описания функций, их входов и выходов, а также исключительных ситуаций. Это обеспечивает проверяемость формулировок и снижает риск неоднозначной интерпретации при реализации [21].

Для структурирования спецификации используется нумерованный перечень функциональных требований. Каждое требование формулируется как обязательство программного средства и охватывает логически завершённую группу возможностей. Такой формат упрощает сопоставление требований с элементами функциональной модели и последующую проверку полноты реализации в процессе разработки и испытаний.

ФТ-1. Программное средство должно обеспечивать регистрацию пользователя, аутентификацию и завершение пользовательской сессии.

ФТ-2. Программное средство должно обеспечивать управление данными игрового профиля пользователя, включая изменение отображаемого имени и параметров профиля.

ФТ-3. Программное средство должно обеспечивать установление соединения между клиентским приложением и сервером и инициализацию пользовательской игровой сессии.

ФТ-4. Программное средство должно обеспечивать создание игровой комнаты, подключение пользователей к существующей комнате по коду и выход пользователя из комнаты.

ФТ-5. Программное средство должно обеспечивать выбор режима игры при создании комнаты и корректную обработку ограничений, связанных с выбранным режимом.

ФТ-6. Программное средство должно обеспечивать передачу и обработку пользовательских команд управления игровым объектом.

ФТ-7. Программное средство должно обеспечивать выполнение игровой логики, включая моделирование движения объектов и обработку столкновений в двумерном пространстве.

ФТ-8. Программное средство должно обеспечивать формирование и поддержание актуального состояния игрового мира на сервере в процессе выполнения игровой сессии.

ФТ-9. Программное средство должно обеспечивать синхронизацию состояния игрового мира между серверной и клиентской частями в режиме реального времени.

ФТ-10. Программное средство должно обеспечивать обработку событий подключения, отключения и повторного подключения пользователей в рамках игровой сессии.

ФТ-11. Программное средство должно обеспечивать визуализацию игрового мира и текущего состояния игровых объектов на стороне клиента.

ФТ-12. Программное средство должно обеспечивать определение условий завершения матча и формирование итогового результата игровой сессии.

ФТ-13. Программное средство должно обеспечивать сохранение результатов завершённых матчей и связанных с ними данных.

ФТ-14. Программное средство должно обеспечивать формирование и предоставление пользователю истории матчей и сводной игровой статистики.

ФТ-15. Программное средство должно обеспечивать запись игровых сессий в виде повторов и последующее воспроизведение их пользователем.

ФТ-16. Программное средство должно обеспечивать управление параметрами повторов, включая изменение названия и режима доступа.

ФТ-17. Программное средство должно обеспечивать публикацию повторов и предоставление доступа к ним по ссылке без обязательной аутентификации пользователя.

ФТ-18. Программное средство должно обеспечивать работу со списком друзей, включая отправку запросов, принятие и отклонение запросов, удаление из списка и блокировку пользователей.

ФТ-19. Программное средство должно обеспечивать поиск пользователей по заданным критериям для выполнения социальных действий.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Разработка архитектуры программного средства

Разрабатываемое программное средство реализуется как распределённая система с клиент-серверной архитектурой, ориентированной на многопользовательское взаимодействие и согласованное обновление общего игрового состояния. Выбор клиент-серверной схемы для сетевой игры связан с тем, что состояние мира удобно централизовать на сервере, а клиентам рассылать согласованные обновления, что снижает риск расхождения состояния на клиентах и упрощает контроль правил игры [22].

Архитектура включает клиентскую подсистему и серверную подсистему. Между ними поддерживается сетевой обмен на уровне прикладных протоколов. Передаваемые сообщения содержат команды управления, события игрового процесса и данные о состоянии игровых объектов, необходимые для синхронизации игрового мира между участниками сеанса.

Клиентская часть реализована в виде веб-приложения, включающего отображение интерфейса и игровой сцены, обработку ввода, управление состоянием экрана и передачу действий пользователя в серверную среду. Реализация интерфейса выполняется на *TypeScript* с использованием библиотеки *React*, которая задаёт декларативное описание интерфейса в виде иерархии компонентов и поддерживает обновление пользовательского интерфейса при изменении данных и состояния приложения [23]. Статическая типизация *TypeScript* задаёт контракты для компонентов и структур данных и снижает вероятность ошибок при росте объёма кода [24].

Для пользователя эта подсистема проявляется как одностраничное приложение, которое после сборки размещается на узле веб-сервера и загружается в браузер при обращении по сетевому адресу. Для выдачи собранной клиентской части в серверной подсистеме используется программный комплекс *Nginx*. Он обслуживает запросы по протоколам *HTTP* и *HTTPS*, отдаёт статические ресурсы приложения, а также задаёт правила кэширования и маршрутизации запросов, применяемые при публикации одностраничных веб-приложений [25]. Указанный компонент входит в состав серверной подсистемы общей и ограничивается доставкой клиентского приложения в браузер. На нём не выполняется прикладная логика игры и не организуется постоянный игровой обмен с участниками сеанса.

Обмен игровыми сообщениями в течение сеанса в проектируемой конфигурации выполняется по протоколу *WebSocket* непосредственно между

браузером пользователя и игровым сервером, а первоначальная загрузка статических ресурсов клиентской части осуществляется через компонент на базе *Nginx*, отвечающий за выдачу этих ресурсов. С прикладной точки зрения взаимодействие выстраивается как последовательность этапов, в которой клиент передаёт команды управления, сервер применяет их при обновлении состояния мира и направляет участникам сообщения с актуальными данными, а клиент отображает полученное состояние. Тем самым обеспечивается единый порядок применения событий и согласованность игровой логики у всех пользователей.

Серверная подсистема проектируемой конфигурации состоит из программного комплекса *Nginx* для публикации клиентской части, игрового сервера на *Node.js* с использованием *TypeScript*, отвечающего за обработку игровой логики и управление игровыми сеансами, а также сервера *PostgreSQL* для хранения данных между сеансами.

Прикладную обработку входящих *HTTP*-запросов на игровом узле организует веб-фреймворк *Express*. Маршрутизация сопоставляет *URI* конечных точек приложения с клиентскими запросами и методами протокола *HTTP* и при совпадении вызывает заданный обработчик. Предусмотрено подключение цепочки промежуточных функций с передачей управления следующему обработчику [26]. Для доступа к СУБД *PostgreSQL* применяется модуль *pg* проекта *node-postgres* – неблокирующий клиент *PostgreSQL* для среды *Node.js*, поддерживающий пул соединений и параметризованные запросы к серверу баз данных [27].

В основе игрового сервера лежит дискретная симуляция с фиксированным шагом времени. На каждом шаге выполняются обработка действий пользователей, обновление состояния объектов, расчёт последствий физической модели и детекция столкновений, после чего формируется актуальное состояние игрового мира. Сформированные обновления рассылаются подключённым клиентам по открытым соединениям *WebSocket* в соответствии с описанным выше порядком обмена данными. Такой подход позволяет поддерживать актуальность состояния игрового мира у всех подключённых участников.

Со стороны пользователя выделяется устройство с операционной системой и браузером, в котором выполняется клиентская часть приложения. Связь с серверной инфраструктурой выполняется через Интернет и маршрутизатор, направляющий трафик к узлам веб-сервера, игрового сервера и сервера СУБД.

Состав узлов, направления связей и протоколы представлены на рисунке 3.1.

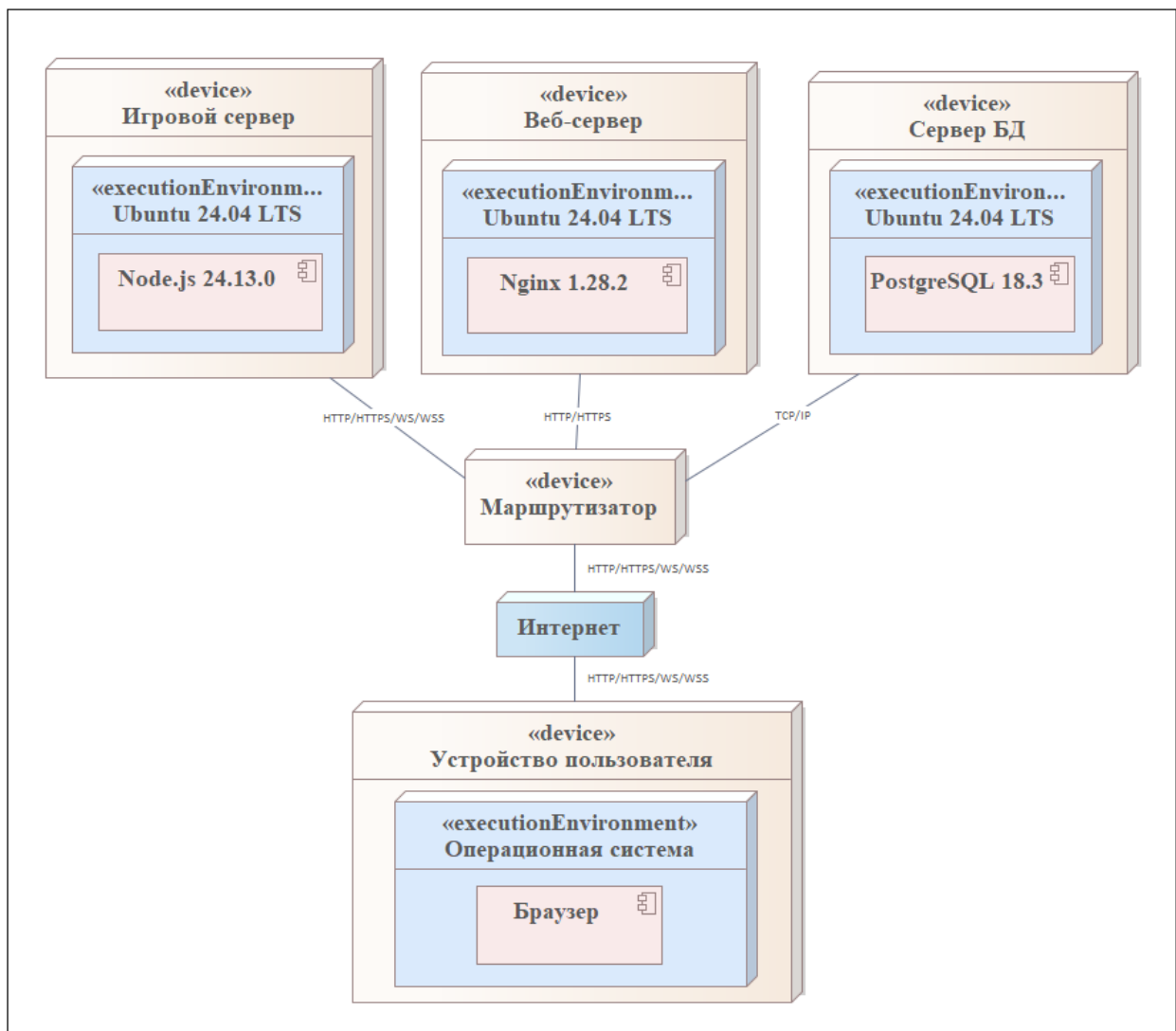


Рисунок 3.1 – Архитектура программного средства

Таким образом, архитектура программного средства задаёт разделение клиента и сервера, прямой игровой канал *WebSocket* к игровому серверу *Node.js*, публикацию интерфейса через отдельный узел *Nginx* и изолированное хранение данных в *PostgreSQL*. Такое разнесение узлов согласуется с различием характера нагрузки и создаёт основу для масштабирования и развития системы без смешения игрового трафика и выдачи статического клиента.

3.2 Разработка модели базы данных

На основе разработанной инфологической модели выполнено проектирование физической модели базы данных с ориентацией на СУБД *PostgreSQL*. Преимущества этой системы для серверного хранения между

сеансами связаны с тем, что *PostgreSQL* сочетает строгую реляционную модель и развитый язык *SQL*, позволяет задавать на уровне сервера первичные и внешние ключи, ограничения уникальности и обязательности, проверки допустимых значений и тем самым поддерживать целостность без дублирования всех правил только в прикладном коде. Параллельные обращения нескольких клиентов обрабатываются в транзакционном режиме, что важно для согласованного доступа к учёту пользователей, матчам и связанным данным. Развитые средства индексирования сокращают время выполнения типовых выборок, расширяемость ядра используется для генерации идентификаторов *UUID* средствами расширения *pgcrypto*. Краткая характеристика системы и обзор её возможностей приведены в электронном ресурсе [28].

Физический уровень моделирования продолжает детализацию концептуальной структуры и позволяет построить физическую схему, на которой учитываются технические возможности выбранной СУБД – типы столбцов, правила ссылочной целостности, индексы и дополнительные ограничения, обеспечивающие корректность и предсказуемость поведения базы в эксплуатации.

Разработанная модель является физическим отображением инфологической модели и реализует сущности предметной области в виде таблиц, связанных первичными и внешними ключами. При проектировании учитывались требования нормализации, снижение избыточности и обеспечение согласованности информации между учётом пользователей, проведением матчей, хранением повторов и вспомогательными подсистемами в условиях многопользовательского сетевого взаимодействия в реальном времени.

Первичные ключи пользовательских и игровых таблиц и справочника типов матчей имеют тип *UUID* и генерируются в СУБД функцией *gen_random_uuid()* при подключённом расширении *pgcrypto*. Текстовые реквизиты хранятся в *VARCHAR* и *TEXT*, моменты событий – в *TIMESTAMPTZ* для однозначной интерпретации времени. Целостность поддерживается ключами и ограничениями *NOT NULL*, *UNIQUE* и *CHECK* по допустимым значениям соответствующих столбцов схемы, для внешних ключей заданы *ON DELETE CASCADE*, *RESTRICT* и *SET NULL* по смыслу связей. Так, при удалении матча удаляются связанные строки ввода и повторов, удаление типа матча при наличии матчей блокируется, необязательная ссылка участника на пользователя может обнуляться при удалении учётной записи.

Схема физической модели базы данных представлена на рисунке 3.2.



Рисунок 3.2 – Физическая модель базы данных

Таким образом, разработанная физическая модель базы данных обеспечивает хранение и согласованную обработку данных программного средства, а использование *PostgreSQL* и ограничений целостности позволяет поддерживать устойчивую работу системы.

3.3 Разработка алгоритмов программного средства

Для обеспечения корректной работы программного средства разработаны алгоритмы, определяющие последовательность выполнения основных процессов системы. Использование алгоритмов позволяет формализовать логику работы приложения и обеспечить согласованное выполнение операций в режиме реального времени.

3.3.1 Обобщённый алгоритм работы программного средства отражает основные этапы взаимодействия клиентской и серверной частей после запуска системы. Серверное приложение инициализирует сетевое взаимодействие, игровые модули и доступ к базе данных, после чего переходит к определению типа поступившего события.

В системе выделяются три основных направления обработки. При запросе интерфейса пользователю передаются статические файлы клиентского приложения, после чего в браузере выполняется инициализация пользовательского интерфейса. При запросе данных сервер проверяет авторизацию пользователя, определяет состав запроса, обращается к базе данных и возвращает результат клиентскому приложению. Такие запросы используются для работы с профилем, статистикой, историей матчей, повторами и другими разделами приложения.

Отдельное направление связано с игровыми событиями. Сервер проверяет участника и игровую сессию, подготавливает или обновляет матч, рассчитывает текущее состояние игрового процесса, обрабатывает столкновения, попадания, подбор предметов и другие игровые взаимодействия. После этого обновлённое состояние передаётся всем участникам сессии.

Клиентская часть при этом выполняет роль средства ввода и отображения: она передаёт серверу действия пользователя и отображает подтверждённое состояние игрового мира. Такой подход позволяет централизовать игровую логику на сервере и уменьшить риск расхождения состояния между участниками многопользовательской сессии.

После обработки выбранного направления система возвращается к ожиданию следующего события. Такой алгоритм соответствует клиент-серверной архитектуре программного средства и обеспечивает согласованную работу интерфейса, операций с данными и многопользовательского игрового процесса.

Обобщённая схема алгоритма работы системы представлена на рисунке 3.3.

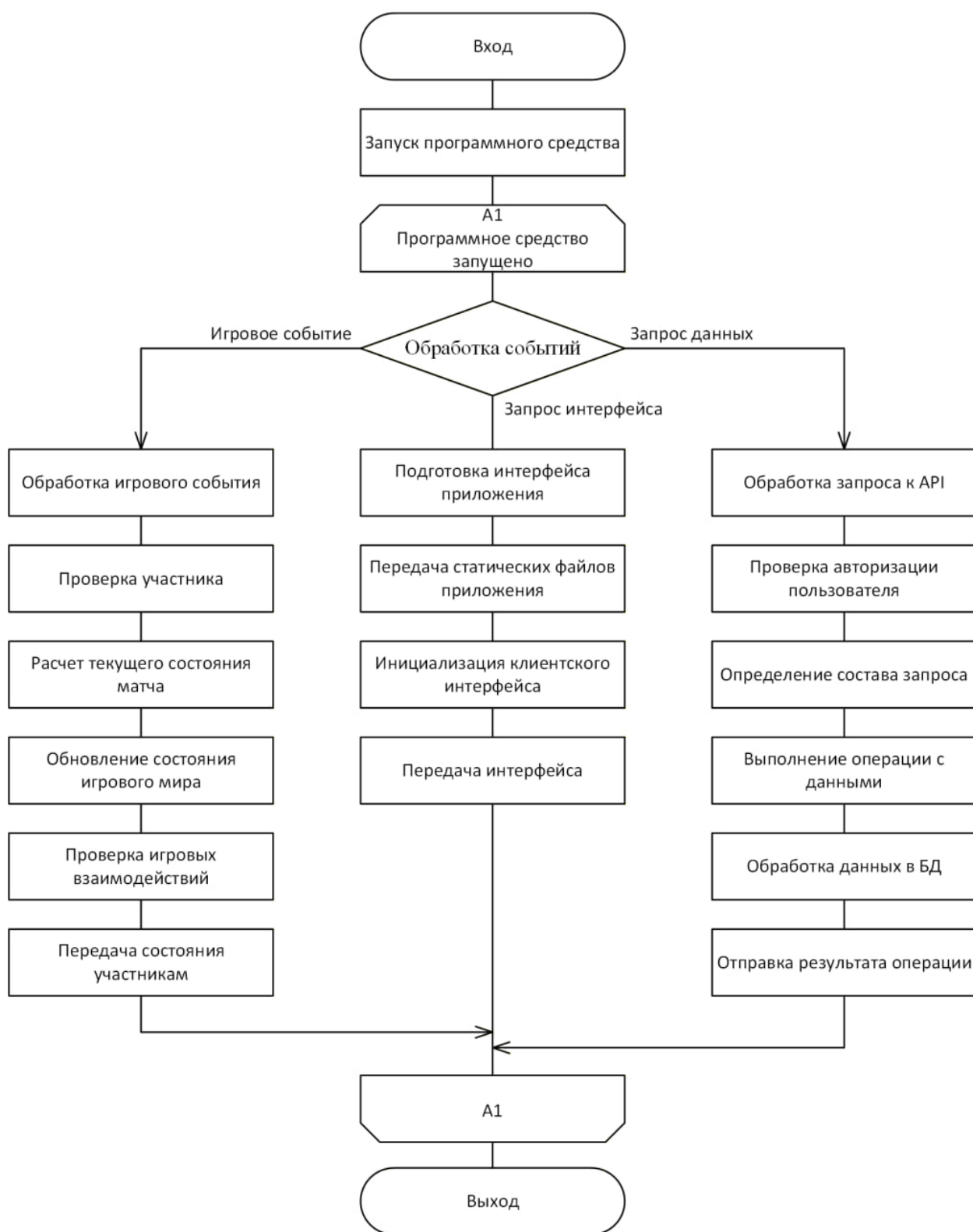


Рисунок 3.3 – Обобщённый алгоритм работы программного средства

Таким образом, представленная схема отражает общий порядок взаимодействия клиентской и серверной частей, включая загрузку интерфейса, обработку запросов данных и выполнение игровых событий. Она

показывает основные направления работы системы и переход к обработке следующего события после завершения выбранного сценария.

3.3.2 Алгоритм создания лабиринта предназначен для формирования структуры игрового уровня на основе сетки ячеек. Каждая ячейка рассматривается как отдельный участок будущего лабиринта, а стены между соседними ячейками определяют возможные направления перемещения. Для построения лабиринта используется пошаговый обход с возвратом, при котором путь обхода сохраняется в стеке.

В начале работы алгоритма инициализируется двумерный массив, предназначенный для хранения информации о посещённых ячейках. Это позволяет исключить повторную обработку уже добавленных в лабиринт участков. Дополнительно создаётся стек, в котором хранится последовательность ячеек текущего пути. После подготовки структур данных выбирается начальная ячейка, помечается как посещённая и помещается в стек.

Основная часть алгоритма выполняется до тех пор, пока стек не станет пустым. На каждой итерации текущей считается ячейка, расположенная на вершине стека. Для неё формируется набор возможных направлений движения: вверх, вниз, влево и вправо. По каждому направлению вычисляются координаты соседней ячейки, после чего проверяется корректность этих координат относительно границ лабиринта.

Для каждой соседней ячейки дополнительно проверяется признак посещения. Если ячейка находится в пределах допустимой области и ранее не была обработана, она добавляется в список доступных соседей. После проверки всех направлений из этого списка случайным образом выбирается одна ячейка, которая становится продолжением текущего пути обхода.

Между текущей и выбранной соседней ячейкой удаляется стена, за счёт чего формируется проход в лабиринте. Затем выбранная ячейка помечается как посещённая и добавляется в стек. Благодаря этому алгоритм продолжает движение вглубь лабиринта, постепенно расширяя построенную область.

Если для текущей ячейки не найдено ни одной допустимой непосещённой соседней ячейки, выполняется возврат назад. Для этого текущая ячейка удаляется из стека, а дальнейшая обработка продолжается для предыдущей ячейки пути. Такой механизм позволяет алгоритму выходить из тупиков и продолжать построение лабиринта в ещё не обработанных участках сетки.

Работа алгоритма завершается после опустошения стека. Это означает, что все достижимые ячейки были обработаны, а между ними сформированы

проходы. В результате получается связная структура лабиринта, пригодная для размещения препятствий на игровом поле.

Схема алгоритма создания лабиринта представлена на рисунке 3.4.



Рисунок 3.4 – Алгоритм создания лабиринта

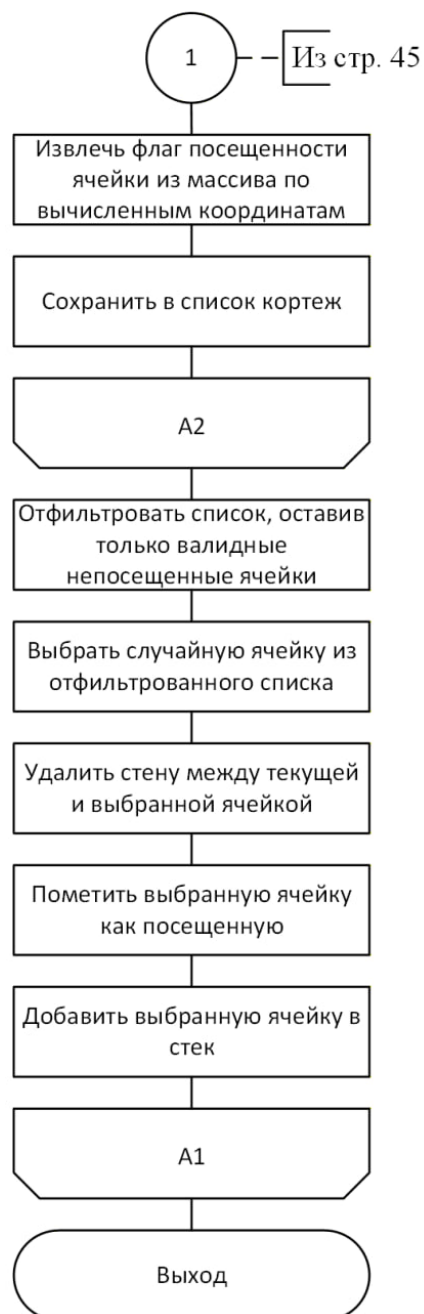


Рисунок 3.4, лист 2 – Алгоритм создания лабиринта

3.3.3 Алгоритм обработки действия игрока предназначен для преобразования команды управления в изменение состояния танка участника матча. На вход алгоритма поступает команда, содержащая признаки движения, поворота корпуса, поворота башни и выполнения выстрела. После получения команды определяется игровая комната, участник матча и соответствующая ему модель танка.

Далее выполняется разбор флагов движения и определяется режим перемещения танка. На основе выбранного режима рассчитываются действующие силы, обновляется скорость танка и вычисляется новая позиция

корпуса. Если обнаружено столкновение с объектами игрового мира, выполняется разрешение коллизий.

После обработки перемещения обновляется поворот корпуса танка и положение башни. Башня синхронизируется с корпусом, при этом её поворот может дополнительно изменяться в зависимости от команды игрока. При наличии команды выстрела проверяется готовность оружия; если оружие готово, создаётся снаряд и обновляются параметры стрельбы.

В завершение результат обработки команды сохраняется в состоянии матча, после чего система переходит к дальнейшему обновлению игрового процесса. Схема алгоритма обработки действия игрока представлена на рисунке 3.5.



Рисунок 3.5 – Алгоритм обработки действия игрока

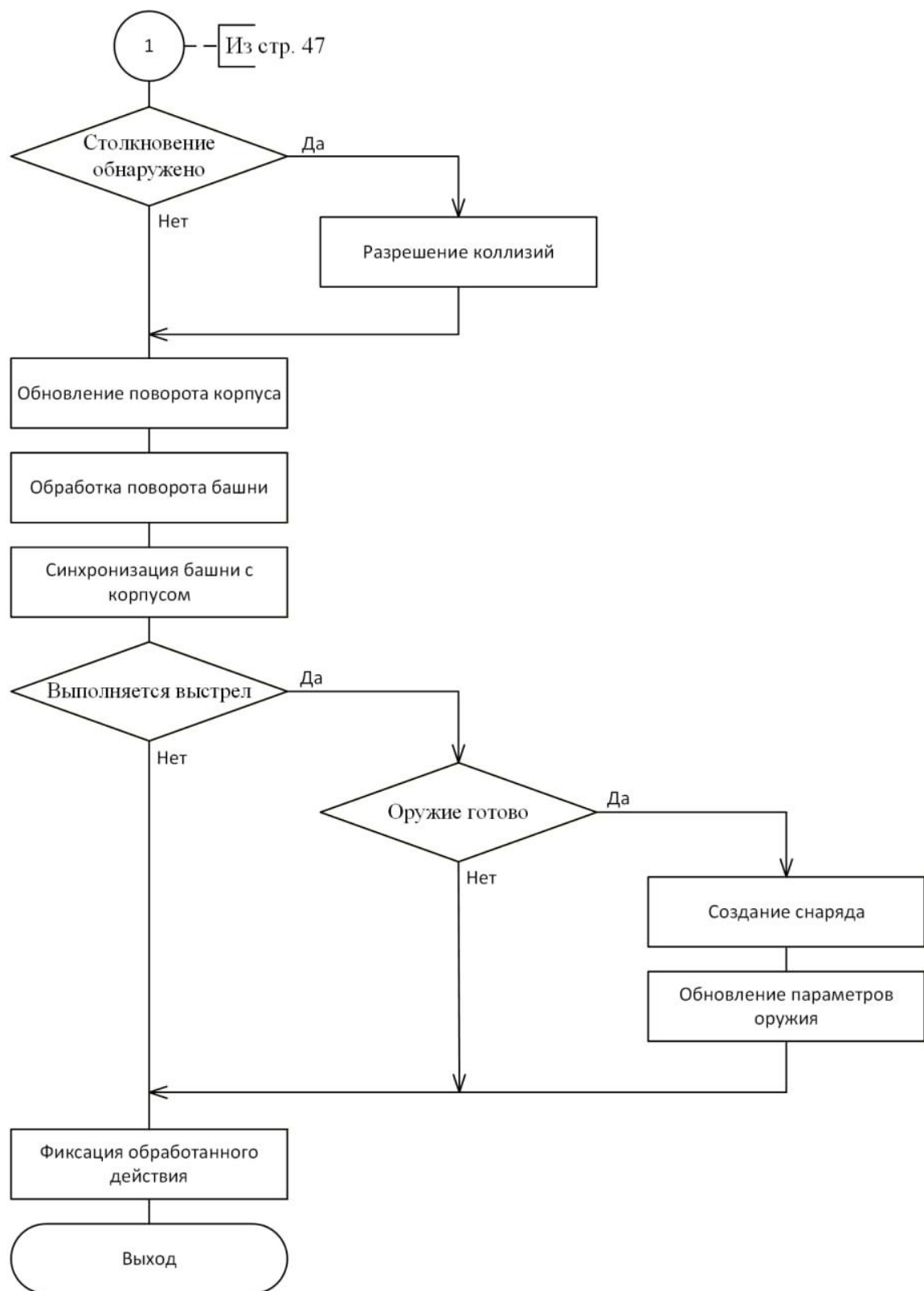


Рисунок 3.5, лист 2 –Алгоритм обработки действия игрока

Таким образом, определена алгоритмическая основа программного средства. Разработанные алгоритмы обеспечивают согласованную работу игровых подсистем и обработку взаимодействий.

4 КОНСТРУИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

В данном разделе рассматривается реализация программного средства многопользовательского взаимодействия в онлайн игре с физической моделью. Описание приведено на уровне архитектурных решений, принципов организации подсистем, используемых технологий, алгоритмов игровой симуляции и механизмов взаимодействия клиентской и серверной частей.

Программное средство построено по клиент-серверной архитектуре. Серверная часть выполняет функции авторитарного узла, определяющего корректное состояние игрового мира, порядок обработки действий пользователей, результаты столкновений, завершение матчей и сохранение игровых данных. Клиентская часть отвечает за пользовательский интерфейс, передачу команд управления, отображение актуального состояния игрового мира и воспроизведение сохранённых матчей. Обмен данными в реальном времени выполняется по протоколу *WebSocket*, а вспомогательные операции учётной записи, профиля, истории матчей и повторов реализованы через *HTTP API*.

4.1 Разработка серверной части

В данном подразделе рассматривается реализация серверной части программного средства как центрального узла управления многопользовательским игровым процессом. Описываются организация сетевого обмена, управление игровыми сессиями, выполнение серверной симуляции, обработка физической модели и реализация правил игровых режимов. Отдельное внимание уделяется тому, как сервер формирует единое подтверждённое состояние игрового мира для всех подключённых клиентов.

4.1.1 Серверная часть программного средства реализована на платформе *Node.js* на языке *TypeScript* и обеспечивает централизованное управление игровыми сессиями, сетевым взаимодействием, физической симуляцией и хранением данных.

Серверная часть поддерживает работу *HTTP API* и обмен данными с клиентами через *WebSocket*. *TypeScript* используется при описании игровых сущностей, сетевых структур данных, параметров матчей, пользовательских действий и результатов симуляции.

Для организации *HTTP API* применяется *Express*, для постоянного двустороннего обмена данными используется библиотека *WebSocket*. Доступ к реляционной базе данных реализован через *PostgreSQL*-драйвер, а

криптографические операции аутентификации выполняются средствами библиотек хеширования паролей и формирования *JWT*-токенов. Конфигурация приложения задаётся через переменные окружения.

Серверная часть разделена на несколько логических подсистем: аутентификация и управление пользователями, игровой *HTTP API*, публичный доступ к опубликованным повторам, управление игровыми комнатами, игровая симуляция, геометрические вычисления, физическая модель, обработка столкновений и доступ к данным. Каждая подсистема выполняет ограниченный набор функций и взаимодействует с остальными через прикладные структуры данных.

4.1.2 Серверное приложение в рамках единого процесса обрабатывает *HTTP*-запросы и *WebSocket*-соединения пользователей.

При запуске сервер инициализирует *HTTP*-приложение, подключает обработку запросов в формате *JSON*, настраивает правила кросс-доменных обращений и регистрирует группы маршрутов для аутентификации, игровой подсистемы и публичного доступа к данным повторов. Дополнительно предусмотрена служебная проверка работоспособности приложения, используемая при локальном запуске и контейнерном развёртывании.

На том же сетевом порту поднимается *WebSocket*-сервер. При контейнерной эксплуатации *nginx* проксирует *HTTP*-запросы и *WebSocket*-соединения. Пользовательский интерфейс, *HTTP API* и игровой канал доступны браузеру через единый адрес.

Перед допуском клиента к игровому обмену выполняется проверка авторизации. Пользовательский токен передаётся при установлении *WebSocket*-соединения и проверяется сервером до регистрации соединения в игровой подсистеме. Неавторизованный клиент не может создать комнату, присоединиться к матчу или отправлять игровые команды.

4.1.3 Управление игровыми сессиями реализовано как серверный реестр комнат, в котором каждая комната содержит участников, наблюдателей, выбранный режим, состояние готовности и активную симуляцию матча.

При создании комнаты сервер формирует короткий код подключения, удобный для передачи другому пользователю. По этому коду второй игрок или группа игроков могут присоединиться к сессии. Сервер назначает роли участников в зависимости от выбранного режима: в стандартном режиме используются роли атакующего и защитника, а в режиме свободного боя все участники выступают как бойцы.

Перед запуском матча каждый участник выбирает конфигурацию танка. Сервер проверяет корректность выбранных параметров и исключает конфликт цветов внутри одной комнаты. Матч начинается только после того, как все необходимые участники выбрали конфигурацию и подтвердили готовность. До этого игровая симуляция для комнаты не запускается.

Система комнат поддерживает переподключение пользователя к активному матчу. Если соединение временно прерывается, сервер сохраняет принадлежность пользователя к комнате и при повторном подключении восстанавливает его участие в текущей игре. Также предусмотрен режим наблюдения, при котором пользователь получает обновления состояния активного матча, но не влияет на игровую симуляцию и не отправляет управляющие действия.

4.1.4 Сетевое взаимодействие в реальном времени построено на обмене типизированными сообщениями, в которых клиент передаёт намерения игрока, а сервер возвращает подтверждённое состояние игрового мира.

Клиент не передаёт серверу координаты танка, результаты попаданий или сведения о победе. Вместо этого он отправляет команды управления: движение, поворот корпуса, поворот башни и выстрел. Сервер принимает эти команды как входные воздействия, применяет их к физической модели и самостоятельно рассчитывает новое состояние мира.

Сервер регулярно передаёт клиентам снимки игрового мира. Снимок содержит состояние танков, снарядов, препятствий, предметов, визуальных событий, текущего уровня, игрового времени и параметров режима. Клиент использует этот снимок только для отображения и не выполняет авторитарное изменение состояния матча.

Кроме игровых снимков, *WebSocket* используется для служебных событий комнаты, уведомлений о начале и завершении матча, приглашений друзей, сообщений чата и отметок на карте. Для социальных и коммуникационных сообщений предусмотрены ограничения частоты, предотвращающие чрезмерную нагрузку и спам в игровой сессии.

4.1.5 Игровой цикл выполняется на сервере с фиксированным шагом времени и используется как для текущего матча, так и для последующего воспроизведения сохранённых повторов.

После запуска матча сервер создаёт игровую модель мира и начинает периодически выполнять шаги симуляции. Частота обновления соответствует интерактивному режиму реального времени. Для компенсации небольших отклонений системного таймера применяется накопление прошедшего

времени: если между итерациями проходит больше времени, сервер выполняет несколько фиксированных шагов подряд, но ограничивает их количество, чтобы не допустить чрезмерной вычислительной нагрузки.

На каждом шаге выполняется последовательная обработка игровых процессов: применение действий игроков, расчёт остаточного движения объектов, перемещение снарядов, проверка столкновений, нанесение урона, удаление неактуальных объектов, подбор предметов, появление бонусов и проверка условий завершения матча. Результат вычисляется сервером независимо от частоты отрисовки кадров на клиентской стороне.

В стандартном режиме реализована логика прохождения уровней с лабиринтами. Атакующий должен собирать ключи и переходить между уровнями, а защитник препятствует выполнению этой задачи до истечения времени. В тренировочном режиме временное ограничение отключается. В режиме свободного боя несколько игроков сражаются на арене, а результат определяется количеством уничтожений за установленное время.

4.1.6 Физическая модель игрового мира учитывает геометрию объектов, массу, поступательную и угловую скорость, сопротивление поверхности, сопротивление воздуха и параметры составных частей танка.

Танк в игровой модели рассматривается как составной объект, включающий корпус, гусеницы, башню и оружие. Корпус определяет запас прочности, бронирование и массу. Гусеницы влияют на ускорение, предельную скорость и манёвренность. Башня задаёт скорость наведения и вместимость боекомплекта. Оружие влияет на урон, скорость снаряда, пробитие и время перезарядки.

Движение танка рассчитывается на основе сил тяги, сил сопротивления и массы объекта. Для разных типов поверхности используются различные коэффициенты сопротивления и бокового скольжения, задающие поведение танка на траве, грунте и песчанике. После применения сил выполняется обновление положения и угла поворота корпуса, а положение башни синхронизируется с изменением ориентации танка.

Снаряды имеют собственные параметры скорости, урона, массы, пробития и времени существования. Для них учитывается сопротивление воздуха и проверяется столкновение с препятствиями и танками. Отдельно реализованы боеприпасы с различными игровыми характеристиками.

4.1.7 Обработка столкновений реализована двухэтапно: сначала выполняется пространственная выборка потенциальных пар объектов, затем для найденных пар применяется точный геометрический алгоритм.

Для предварительного отбора объектов используется дерево квадрантов. Игровое поле рекурсивно разделяется на области, после чего объект проверяется с сущностями, расположенными в потенциально пересекающихся зонах. Полный перебор всех объектов мира в основной процедуре столкновений не выполняется.

Точная проверка пересечения выполняется методом разделяющих осей. Для выпуклых многоугольников вычисляются оси, соответствующие их сторонам, после чего проекции объектов сравниваются на каждой оси. Если существует хотя бы одна ось, на которой проекции не пересекаются, столкновение отсутствует. Если пересечение имеется на всех осях, вычисляются нормаль контакта и глубина проникновения.

Разрешение столкновения выполняется с учётом масс объектов, относительной скорости, нормального импульса, коэффициента восстановления и силы трения. Статические стены не смещаются, а динамические объекты корректируют положение и скорость. Для столкновений танков с препятствиями дополнительно формируются события, используемые клиентом для звуковых и визуальных эффектов.

Для снарядов предусмотрено деление перемещения на подшаги с проверкой контактов после каждого подшага. При попадании по танку урон применяется к броне и прочности цели, при попадании по разрушаемому объекту уменьшается его запас прочности, а при уничтожении объекта формируется событие разрушения.

4.1.8 Игровые правила и режимы реализованы на сервере и включают режимы «Классика», «Тренировка» и «Арена».

В режиме «Классика» игровой процесс строится вокруг сбора ключей атакующим. После выполнения цели на текущем уровне сервер перестраивает игровую сцену, размещает новые препятствия и переносит участников на следующий уровень. Победа атакующего наступает после завершения последнего уровня, а победа защитника - при истечении лимита времени.

Режим «Тренировка» использует механику «Классики», но не ограничивает матч по времени и не предназначен для полноценной записи соревновательного результата.

В режиме «Арена» от двух до пяти игроков участвуют в раунде на отдельной боевой карте. Сервер ведёт счёт уничтожений, выполняет респавн уничтоженных танков и по истечении времени определяет победителя или группу победителей. Для этого режима предусмотрены разрушаемые и подвижные препятствия, влияющие на тактическое поведение игроков.

Во всех режимах сервер собирает статистику участника: количество уничтожений, смертей, выстрелов, попаданий, нанесённого и полученного урона, подобранных ключей и боеприпасов. Статистика используется при отображении результатов и сохраняется для последующего анализа матчей.

4.2 Разработка клиентской части

В данном подразделе рассматривается реализация клиентской части программного средства, предназначенной для взаимодействия пользователя с системой. Описываются организация экранов приложения, обработка пользовательского ввода, поддержка *WebSocket*-соединения, отображение игрового состояния и работа с графическими и звуковыми ресурсами. Клиентская часть рассматривается как подсистема представления и управления, получающая подтверждённое состояние игрового мира от сервера.

4.2.1 Клиентская часть программного средства реализована как одностраничное веб-приложение на *React* и *TypeScript*, обеспечивающее пользовательские сценарии, управление матчем и визуализацию игрового состояния.

Клиентская часть реализует интерфейс как набор взаимосвязанных экранов. *TypeScript* применяется для описания пользовательских данных, сетевых ответов и внутренних состояний интерфейса. Сборка выполняется средствами *Webpack*, который формирует набор статических файлов для размещения на веб-сервере.

Клиентская часть включает экраны регистрации, входа, главной панели, выбора режима игры, настройки танка, игрового лобби, самого матча, итогов, профиля, настроек, друзей, статистики, истории повторов, публичной галереи и просмотра повторов. Маршрутизация реализована на стороне браузера, поэтому переход между основными разделами не требует полной перезагрузки страницы.

Доступ к защищённым разделам ограничивается состоянием авторизации. После загрузки приложения клиент пытается восстановить сохранённую сессию, запрашивает сведения о текущем пользователе и при успешной проверке открывает защищённую часть интерфейса. При отсутствии действительной сессии пользователь перенаправляется на экран входа или регистрации.

4.2.2 Управление состоянием клиентского приложения организовано через глобальные контексты и локальные состояния экранов, используемые для авторизации, настроек и игрового соединения.

Глобальное состояние содержит сведения о текущем пользователе, токене доступа, пользовательских настройках, параметрах управления, настройках звука и активном *WebSocket*-соединении. Такие данные используются в разных частях приложения, поэтому они вынесены на уровень общих контекстов. Локальные состояния применяются для временных данных конкретных экранов: текущего кода комнаты, состава лобби, выбранной конфигурации танка, сообщений об ошибках и состояния загрузки ресурсов.

HTTP-взаимодействие с сервером сосредоточено в клиентской подсистеме доступа к *API*. Она формирует адреса запросов, передаёт токен доступа и обрабатывает ответы сервера. Через *HTTP* выполняются операции, не требующие постоянной синхронизации: вход и регистрация, получение профиля, работа с повторами, историей матчей, пресетами танка, друзьями и публичной галереей.

WebSocket-соединение создаётся один раз для авторизованного пользователя и используется как общий канал игровых и социальных уведомлений. При выходе из учётной записи соединение корректно закрывается, а при повторном входе создаётся с новым токеном доступа.

4.2.3 Клиентская подсистема *WebSocket* поддерживает установление соединения, передачу токена авторизации, обработку входящих событий, буферизацию исходящих команд и автоматическое переподключение при кратковременных сбоях.

При подключении клиент определяет адрес игрового сервера в зависимости от режима развёртывания. В локальной разработке соединение может выполняться напрямую с серверной частью, а в контейнерной конфигурации - через проксирующий веб-сервер. Прикладная логика соединения при этом остаётся одинаковой.

Если пользователь отправляет команду до фактического открытия соединения, команда помещается во внутреннюю очередь и передаётся после успешного подключения. При обрыве соединения клиент выполняет ограниченное количество повторных попыток с увеличивающейся задержкой. Ошибки авторизации обрабатываются отдельно, поскольку в этом случае повторное подключение без нового входа пользователя не имеет смысла.

Входящие сообщения распределяются по обработчикам соответствующих пользовательских сценариев. Игровой экран реагирует на снимки мира и завершение матча, экран лобби - на изменения состава комнаты

и готовности участников, глобальный интерфейс – на приглашения друзей и социальные уведомления.

4.2.4 Пользовательский сценарий участия в матче включает выбор режима, создание или присоединение к комнате, настройку танка, ожидание готовности участников, игровой процесс и просмотр результатов.

На начальном игровом экране пользователь выбирает режим матча и создаёт комнату либо вводит код существующей комнаты. После успешного подключения сервер назначает пользователю роль, а клиент переводит его на экран выбора танка. Конфигурация танка формируется из доступных частей и цвета, после чего передаётся серверу для проверки.

После выбора танка пользователь попадает в лобби. В лобби отображается состав участников, их роли, выбранные конфигурации и состояние готовности. Когда все необходимые участники подтвердили готовность, сервер запускает матч и клиент автоматически переходит к игровому экрану.

После завершения матча клиент отображает итоговую информацию: победителя или победителей, причину завершения, очки в режиме «Арена» и статистику участников. Пользователь может вернуться к выбору режима, просмотреть историю матчей или перейти к сохранённым повторам.

4.2.5 Обработка пользовательского ввода преобразует действия клавиатуры и сенсорных элементов управления в унифицированные команды, передаваемые серверу как намерения игрока.

На игровом экране клиент отслеживает нажатые клавиши и сопоставляет их с действиями движения, поворота корпуса, вращения башни и стрельбы. Раскладка управления хранится в пользовательских настройках, поэтому она может изменяться без переработки игровой логики. При удержании клавиш движения команды отправляются с частотой, близкой к частоте серверного игрового цикла.

Стрельба обрабатывается отдельно от непрерывного движения. Клиент передаёт намерение выстрела, а сервер самостоятельно проверяет возможность выстрела с учётом текущего боекомплекта и времени перезарядки. Это исключает зависимость боевой логики от частоты отправки клиентских сообщений.

Для устройств с сенсорным вводом предусмотрено виртуальное управление. Виртуальный джойстик преобразует направление отклонения в команды движения и поворота, отдельные элементы интерфейса управляют

башней, огнём и отметками на карте. На сервер передаются те же типы управляющих команд, что и при клавиатурном управлении.

4.2.6 Визуализация игрового мира выполняется средствами *HTML Canvas* на основе последнего подтверждённого сервером снимка состояния.

Клиентский рендеринг не рассчитывает авторитарную физику и не принимает решений о столкновениях, уроне или победе. Он получает снимок состояния и обновляет отображаемые объекты: танки, снаряды, стены, разрушаемые препятствия, ключи, бонусы и визуальные эффекты. Такой подход соответствует модели тонкого клиента и предотвращает расхождение игровой логики между участниками.

Игровой мир имеет базовую координатную систему, которая масштабируется под фактический размер окна браузера. При изменении размеров окна клиент пересчитывает область отображения и сохраняет пропорции сцены без изменения серверной модели координат.

Танки отображаются как составные графические объекты, соответствующие выбранным частям. Дополнительно отображаются полосы здоровья и брони, имя игрока, следы движения, защитный эффект и одноразовые анимации попаданий или уничтожения. События взрыва, попадания и столкновения поступают от сервера вместе со снимками мира и используются для синхронного запуска визуальных и звуковых эффектов.

4.2.7 Подсистема ресурсов выполняет предварительную загрузку графики и звука до начала активной отрисовки матча.

Перед началом активной отрисовки клиент загружает изображения фонов, стен, танковых частей, снарядов, предметов и анимационных эффектов. На время загрузки отображается состояние подготовки игрового экрана.

Звуковая подсистема регистрирует аудиоресурсы интерфейса и игрового процесса. В игре воспроизводятся звуки выстрелов, столкновений, попаданий, взрывов, получения урона, уничтожения и действий интерфейса. Громкость регулируется пользовательскими настройками, а игровые звуки могут учитывать расстояние от танка пользователя до события в игровом мире.

Дополнительно предусмотрен механизм обеспечения минимального набора графических ресурсов при развёртывании. Если часть необязательных изображений отсутствует, приложение может использовать технические заглушки, что сохраняет работоспособность интерфейса, хотя полноценное визуальное качество требует полного набора ресурсов.

4.3 Реализация хранения данных и системы повторов

В данном подразделе рассматриваются механизмы долговременного хранения данных и восстановления хода матчей. Описываются основные сущности предметной области, порядок авторизации пользователей, операции *HTTP API* и способ сохранения данных, необходимых для воспроизведения повторов. Хранение данных рассматривается как часть серверной реализации, обеспечивающая сохранение пользовательского прогресса, истории матчей и публичных игровых материалов.

4.3.1 Хранение данных реализовано на основе *PostgreSQL*, а доступ к данным организован через слой репозитория, отделяющий прикладную логику от *SQL*-запросов.

В базе данных сохраняются сведения об учётных записях, профилях, матчах, участниках, результатах, повторах, пользовательских конфигурациях танков, социальных связях и оценках публичных повторов. Связи между пользователями, матчами и сохранёнными игровыми данными описаны средствами реляционной модели.

Серверная часть обращается к базе через пул соединений. Все запросы выполняются параметризованно и используют значения пользовательского ввода как параметры *SQL*-операций.

Слой доступа к данным скрывает детали *SQL*-операций от игровой и сетевой логики. Прикладные подсистемы работают с предметными операциями: создание пользователя, сохранение матча, получение истории, публикация повтора, изменение профиля или обработка дружеской связи. Такое разделение повышает сопровождаемость программного средства.

4.3.2 Модель данных включает сущности пользователей, профилей, матчей, участников, повторов, действий повтора, сохранённых конфигураций танка, социальных связей и оценок публичных записей.

Сущность пользователя хранит данные, необходимые для входа в систему и отображения имени в интерфейсе. Расширенный профиль содержит дополнительные параметры, такие как аватар и предпочитаемая игровая роль. Для логина и электронной почты предусмотрены ограничения уникальности, исключающие создание дублирующих учётных записей.

Сущности матча и участника фиксируют историю игровых сессий. Для матча сохраняются тип режима, код комнаты, статус, причина завершения, длительность и временные метки. Для участника сохраняются пользователь,

роль, выбранная конфигурация танка, индивидуальные показатели и признак победителя.

Сущность повтора связывается с матчем и владельцем записи. Для повтора хранятся название, признак публичности, данные для воспроизведения и служебная информация публикации. Отдельно предусмотрено хранение сохранённых пользователем конфигураций танка для повторного выбора наборов частей.

Социальная часть модели содержит связи между пользователями, включая заявки в друзья, принятые дружеские связи и блокировки. Для публичных повторов предусмотрена возможность оценки, при которой один пользователь может поставить не более одной оценки одной записи.

4.3.3 Аутентификация и авторизация реализованы на основе защищённого хранения паролей, *JWT*-токенов и проверки прав доступа при обращении к *HTTP API* и подключении через *WebSocket*.

При регистрации сервер проверяет корректность логина, электронной почты, пароля и отображаемого имени. Пароль не сохраняется в открытом виде: перед записью выполняется криптографическое хеширование с солью. При входе в систему сервер сравнивает введённый пароль с сохранённым хешем.

После успешной регистрации или входа пользователю выдаётся подписанный токен доступа. Токен содержит сведения, необходимые для идентификации пользователя при последующих обращениях, и проверяется сервером перед выполнением защищённых операций. После завершения входа клиент передаёт серверу токен, а не пароль пользователя.

HTTP API использует проверку токена в заголовке запроса. *WebSocket*-подключение проверяется при установлении соединения. Один и тот же механизм идентификации применяется для операций профиля, участия в матчах, получения уведомлений друзей и восстановления активной игровой сессии.

4.3.4 *HTTP API* обеспечивает операции учётной записи, профиля, истории матчей, повторов, пресетов танка, друзей, публичной галереи и главной панели пользователя.

Операции учётной записи включают регистрацию, вход, получение сведений о текущем пользователе и изменение профиля. Игровое *API* предоставляет пользователю доступ к истории матчей, сохранённым повторам, данным воспроизведения, публикации и отзыву публичной ссылки, а также к сохранённым конфигурациям танка.

Социальные функции включают поиск пользователей, отправку и обработку заявок в друзья, удаление связей, блокировку и разблокировку пользователей. При изменении социальных данных сервер отправляет уведомления активным клиентам через *WebSocket*, после чего клиент обновляет соответствующие элементы интерфейса.

Публичный доступ к повторам отделён от защищённых операций пользователя. Опубликованные записи доступны по общей ссылке или через галерею, при этом приватные повторы остаются доступными только владельцам и участникам соответствующих матчей.

4.3.5 Система повторов реализована как детерминированное воспроизведение матча по начальному состоянию, зерну генератора случайных чисел, событиям мира и журналу действий игроков.

При запуске записываемого матча сервер сохраняет метаданные, необходимые для восстановления исходных условий: режим, частоту симуляции, конфигурации участников, материал арены или уровня и зерно псевдослучайного генератора. При воспроизведении эти данные используются для повторного формирования начального состояния матча.

В процессе игры сохраняются не все кадры состояния, а компактный журнал событий. В него входят начальное состояние мира, события появления случайных предметов и действия игроков на конкретных шагах симуляции. Такой подход значительно уменьшает объём сохраняемых данных по сравнению с покадровой записью.

При воспроизведении система повторно запускает симуляцию с тем же фиксированным шагом времени и последовательно применяет сохранённые действия. Поскольку серверная физика детерминирована относительно начальных условий и входных команд, восстановленная последовательность состояний соответствует исходному матчу. Клиент получает подготовленные данные воспроизведения и отображает повтор как последовательность кадров игрового мира.

4.4 Итоги реализации программного средства

Таким образом, разработанное программное средство представляет собой клиент-серверную систему для организации многопользовательского игрового взаимодействия в реальном времени. Серверная часть выполняет расчёт игрового состояния, обработку игровой логики и хранение данных. Клиентская часть обеспечивает управление игровым процессом, отображение состояния игрового мира и работу с результатами матчей и повторами.

5 ТЕСТИРОВАНИЕ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОГРАММНОГО СРЕДСТВА

Проверка качества разработанного программного средства организована в два взаимодополняющих контура. Для серверной части используется автоматизированное тестирование на базе среды *Jest*. Такой подход обеспечивает регрессионную проверку алгоритмов, прикладного интерфейса *HTTP* и слоя доступа к данным при изменении исходного кода, а также гарантирует, что критические маршруты обработки запросов сохраняют ожидаемое поведение после каждой итерации разработки.

Ручное тестирование выполняется только для клиентской части и только с позиции конечного пользователя: проверяется поведение веб-интерфейса в браузере по заранее описанным сценариям без необходимости опираться на внутреннюю реализацию.

Оценка качества опирается на совокупность автоматических и ручных проверок: серверная логика и *API* подтверждаются тестами *Jest*, работоспособность интерфейса – прохождением сценариев в браузере. Поэтапная отладка в среде разработки дополняет эти виды проверок; выявленные дефекты устранялись итеративно. Успешное прохождение полного набора проверок служит основанием для фиксации стабильной версии, готовой к демонстрации и дальнейшей эксплуатации.

5.1 Цели, объект и методы тестирования

Объектом тестирования является разработанное программное средство в целом, включающее серверную и клиентскую части и обеспечивающее многопользовательское. Серверная часть принимает и обрабатывает запросы прикладного интерфейса *HTTP*, ведёт учётные данные и игровую статистику во внешней реляционной базе данных *PostgreSQL*, поддерживает постоянное двустороннее соединение по протоколу *WebSocket* для передачи игровых событий. Клиентская часть реализована как одностраничное веб-приложение и обеспечивает ввод пользовательских действий, отображение интерфейса и игровой сцены, а также взаимодействие с сервером.

Целью тестирования является подтверждение работоспособности программного средства в типовых и граничных условиях эксплуатации: корректность реализации физической модели и игровой логики на сервере, согласованность ответов прикладного интерфейса с правилами доступа и целостности данных, устойчивость к некорректному вводу, а также

возможность пользователя пройти полный цикл от регистрации до завершения матча, работы с повторами и социальными функциями без сбоев интерфейса.

В качестве методов проверки применяются автоматизированное тестирование серверной части на платформе *Jest* с фиксацией ожидаемых результатов в исходном коде тестов и ручное функциональное тестирование клиентской части по заранее составленным сценариям с занесением фактических результатов в сводную таблицу. Такое разделение обусловлено тем, что алгоритмические и протокольные аспекты сервера допускают формализацию в виде повторяемых автоматических проверок, тогда как оценка удобства навигации, наглядности сообщений об ошибках и визуальной согласованности состояния игры удобнее выполняется человеком при непосредственной работе с браузером.

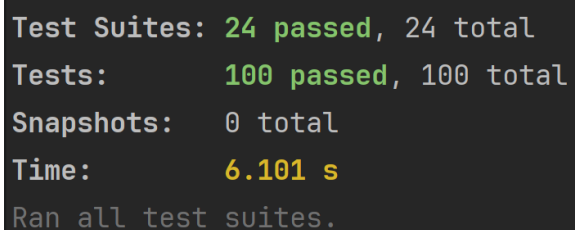
5.2 Автоматизированное тестирование серверной части

Для автоматизированной проверки серверной части используется среда *Jest*, что позволяет выполнять тесты по исходным модулям проекта и получать воспроизводимый результат на каждом прогоне. Исходные тексты тестов разделены на два набора: изолированные модульные тесты и интеграционные сценарии, в которых участвуют несколько слоёв приложения.

5.2.1 Модульное тестирование направлено на изолированную проверку отдельных частей серверного приложения, исключая взаимодействие с базой данных и полный сетевой цикл игры.

В проект включен набор проверок для ключевых модулей, охватывающий корректность вычислений, поведение при различных ветвях игровой логики и реакцию на недопустимые ситуации, такие как повторная регистрация с тем же логином или ввод неверных данных для входа.

Результаты успешного выполнения набора модульных тестов представлены на рисунке 5.1.



```
Test Suites: 24 passed, 24 total
Tests:      100 passed, 100 total
Snapshots:  0 total
Time:       6.101 s
Ran all test suites.
```

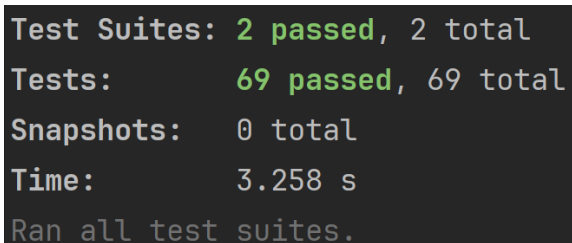
Рисунок 5.1 – Результаты успешного выполнения набора модульных тестов

Полученные результаты подтверждают корректность реализации ключевых алгоритмов серверной части: физической модели, обработки столкновений и реакции системы на некорректные входные данные. Отсутствие ошибок в модульных тестах создаёт уверенность в надёжности отдельных компонентов перед их интеграцией.

5.2.2 Интеграционное тестирование дополняет модульные проверки, воспроизводя обращение к серверу через стандартные *HTTP*-запросы от клиентского приложения к тестовому экземпляру программы.

В ходе тестирования проверяется вся цепочка обработки данных: от приема запроса и валидации входных параметров до взаимодействия с базой данных и формирования итогового ответа.

Результаты успешного выполнения набора интеграционных тестов представлены на рисунке 5.2.



```
Test Suites: 2 passed, 2 total
Tests:      69 passed, 69 total
Snapshots:  0 total
Time:       3.258 s
Ran all test suites.
```

Рисунок 5.2 – Результаты успешного выполнения набора интеграционных тестов

Успешное прохождение интеграционных тестов подтверждает корректность взаимодействия между слоями приложения: маршрутизация *HTTP*-запросов, валидация параметров, обращение к базе данных и формирование ответов работают согласованно. Результаты демонстрируют ожидаемое поведение системы в сценариях, приближенных к реальной эксплуатации.

5.3 Ручное тестирование пользовательского интерфейса

Для комплексной проверки качества программного продукта, наряду с автоматизированным тестированием серверной части, было проведено ручное тестирование клиента на собранной версии системы. Оно позволило подтвердить корректность работы программного средства в типовых пользовательских сценариях и оценить удобство и логику интерфейса с точки зрения конечного пользователя.

Ручное тестирование выполнялось на персональном компьютере под управлением *Microsoft Windows 11* в браузере *Google Chrome* версии 148.0.7778.96. Проверка проводилась на собранной версии клиентского приложения с подключением к серверной части через локальную сеть разработки.

Для сценариев с несколькими одновременными участниками применялся второй экземпляр *Chrome* в режиме инкогнито, что позволило изолировать сессии (учетные записи, токены, кэш) и исключить влияние общего локального состояния браузера при проверке многопользовательских сценариев. Дополнительно контролировалась корректность синхронизации состояния между клиентами при одновременном выполнении игровых действий.

В ходе проверки последовательно воспроизводились основные пользовательские сценарии, предусмотренные техническим заданием и функциональной моделью системы. Ручное тестирование позволило оценить обработку действий пользователя, корректность отображения игрового процесса и синхронизацию состояния между игроками. Также проверялась обработка типовых пользовательских действий и игровых событий в многопользовательском режиме.

В процессе тестирования проверяются:

- регистрация и авторизация пользователя;
- создание игровой комнаты, настройка танка и подключение второго игрока по коду комнаты;
- запуск матча из комнаты ожидания, управление танком и отображение итогов раунда;
- просмотр сохраненных повторов и истории матчей;

Фактические результаты выполнения тест-кейсов логически распределены по таблицам.

Таблица 5.1 – Тест-кейсы регистрации и авторизации

Тест	Ожидаемый результат	Результат
Регистрация пользователя 1 Открыть веб-интерфейс будучи неавторизованным. 2 Перейти на страницу регистрации. 3 Ввести допустимый логин.	1 Отображается экран входа. 2 Открыта форма регистрации пользователя. 3 Логин отображается в соответствующем поле.	Успех

Продолжение таблицы 5.1

Тест	Ожидаемый результат	Результат
4 Ввести корректный адрес электронной почты. 5 Ввести имя в игре. 6 Ввести допустимый пароль. 7 Нажать кнопку подтверждения регистрации.	4 Адрес электронной почты отображается в соответствующем поле. 5 Имя в игре отображается в соответствующем поле. 6 Пароль введен с маскированием символов. 7 Учетная запись создана; выполнен переход к основному интерфейсу.	
Конфликт учетных данных 1 Открыть веб-интерфейс будучи неавторизованным. 2 Перейти на страницу регистрации. 3 Ввести новый допустимый логин. 4 Ввести адрес электронной почты, уже зарегистрированный в системе. 5 Заполнить остальные поля корректными значениями. 6 Нажать кнопку подтверждения регистрации.	1 Отображается экран входа. 2 Открыта форма регистрации пользователя. 3 Логин отображается в поле ввода. 4 Адрес электронной почты отображается в поле ввода. 5 Остальные поля содержат введенные значения. 6 Регистрация отклонена; отображается сообщение об ошибке; новая учетная запись не создана.	Успех
Успешная авторизация 1 Открыть веб-интерфейс будучи неавторизованным. 2 Открыть страницу входа в аккаунт. 3 Ввести логин зарегистрированного пользователя.	1 Отображается экран входа. 2 Отображается форма входа с полями логина и пароля. 3 Логин отображается в поле ввода.	Успех

Продолжение таблицы 5.1

Тест	Ожидаемый результат	Результат
4 Ввести корректный пароль пользователя. 5 Нажать кнопку подтверждения входа. 6 Проверить состояние верхнего меню системы.	4 Пароль введен с маскированием символов. 5 Выполнен переход к основному интерфейсу; сессия установлена. 6 В верхнем меню отображаются разделы программного средства, имя пользователя и кнопка выхода.	
Отказ входа при неверном пароле 1 Открыть веб-интерфейс будучи неавторизованным. 2 Открыть страницу входа в аккаунт. 3 Ввести логин зарегистрированного пользователя. 4 Ввести заведомо неверный пароль. 5 Нажать кнопку подтверждения входа. 6 Проверить текущий экран системы.	1 Отображается экран входа. 2 Отображается форма входа с полями логина и пароля. 3 Логин отображается в поле ввода. 4 Пароль введен с маскированием символов. 5 Вход отклонен; отображается сообщение об ошибке учетных данных. 6 Пользователь остается на странице входа; защищенные разделы не открываются.	Успех

Таблица 5.2 – Тест-кейсы игрового раздела

Тест	Ожидаемый результат	Результат
Создание игровой комнаты и подключение второго игрока 1 Открыть веб-интерфейс будучи авторизованным первым пользователем. 2 Перейти в раздел «Игра».	1 Отображается основной интерфейс авторизованного первого пользователя. 2 Открыт раздел выбора режима игры.	Успех

Продолжение таблицы 5.2

Тест	Ожидаемый результат	Результат
3 Оставить активной вкладку «Создать комнату». 4 Выбрать режим матча «Арена». 5 Нажать кнопку создания комнаты. 6 На экране выбора танка подтвердить конфигурацию первого пользователя. 7 Зафиксировать код комнаты на экране комнаты ожидания. 8 Открыть второе окно браузера в режиме инкогнито. 9 Во втором окне открыть веб-интерфейс и войти под учетной записью второго пользователя. 10 Перейти во втором окне в раздел «Игра». 11 Открыть вкладку «Войти по коду». 12 Ввести код комнаты первого пользователя и нажать кнопку «Войти». 13 На экране выбора танка подтвердить конфигурацию второго пользователя. 14 Проверить список участников в комнате ожидания.	3 Отображаются элементы выбора режима и кнопка создания комнаты. 4 Режим «Арена» визуально выбран. 5 Открыт экран настройки танка перед входом в комнату. 6 Открыта комната ожидания; конфигурация танка первого пользователя сохранена. 7 Код комнаты отображается на экране комнаты ожидания. 8 Запущено отдельное окно без общей сессии с первым пользователем. 9 Выполнен вход второго пользователя. 10 Открыт раздел выбора режима игры второго пользователя. 11 Отображается поле ввода кода комнаты. 12 Выполнен переход на экран настройки танка второго пользователя. 13 Открыта комната ожидания второго пользователя; конфигурация танка сохранена. 14 В комнате ожидания отображаются оба участника.	
Подключение по неверному коду комнаты 1 Открыть веб-интерфейс будучи авторизованным.	1 Отображается основной интерфейс авторизованного пользователя.	Успех

Продолжение таблицы 5.2

Тест	Ожидаемый результат	Результат
2 Перейти в раздел «Игра». 3 Открыть вкладку «Войти по коду». 4 Ввести шестизначный код, не соответствующий активной комнате. 5 Нажать кнопку «Войти». 6 Проверить доступность элементов раздела «Игра».	2 Открыт раздел выбора режима игры. 3 Отображается форма ввода кода комнаты. 4 Код отображается в поле ввода. 5 Подключение отклонено; отображается сообщение об ошибке. 6 Раздел остается доступным; пользователь может повторить ввод или создать комнату.	
Запуск матча, игровой процесс и отображение итогов 1 Открыть комнату ожидания с двумя подключенными пользователями. 2 Проверить список участников в первом окне. 3 Проверить список участников во втором окне. 4 Нажать кнопку «Готов» первым пользователем. 5 Нажать кнопку «Готов» вторым пользователем. 6 Дождаться перехода к игровой сцене. 7 Нажать клавишу движения вперед. 8 Нажать клавишу поворота корпуса. 9 Нажать клавишу поворота башни. 10 Нажать клавишу выстрела.	1 Отображается комната ожидания с кодом комнаты. 2 В первом окне отображаются оба участника и статусы готовности. 3 Во втором окне отображаются оба участника и статусы готовности. 4 Статус первого пользователя изменяется на «Готов». 5 Статус второго пользователя изменяется на «Готов»; отображается сообщение о начале матча. 6 В обоих окнах открывается игровая сцена. 7 Танк пользователя начинает движение; состояние синхронизируется у второго участника. 8 Корпус танка изменяет направление движения.	Успех

Продолжение таблицы 5.2

Тест	Ожидаемый результат	Результат
11 Дождаться окончания времени раунда. 12 Проверить экран итогов матча. 13 Нажать кнопку «Вернуться в меню».	9 Башня танка поворачивается независимо от корпуса. 10 На сцене появляется снаряд; воспроизводится визуальный и звуковой эффект выстрела. 11 После завершения раунда выполняется переход к экрану итогов. 12 Отображаются итоговый статус матча, список результатов участников и личная статистика пользователя. 13 Пользователь возвращается в раздел выбора режима игры.	

Таблица 5.3 – Тест-кейсы повторов и истории матчей

Тест	Ожидаемый результат	Результат
Просмотр сохраненного повтора 1 Открыть веб-интерфейс будучи авторизованным. 2 Перейти в раздел «Повторы». 3 Открыть вкладку «Мои повторы». 4 Выбрать сохраненный повтор и нажать «Смотреть». 5 Нажать кнопку паузы или воспроизведения. 6 Переместить ползунок позиции воспроизведения.	1 Отображается основной интерфейс авторизованного пользователя. 2 Открыт раздел повторов. 3 Отображается список сохраненных записей матчей. 4 Открывается экран воспроизведения выбранного повтора. 5 Воспроизведение начинается. 6 Кадр повтора изменяется в соответствии с положением ползунка.	Успех

Продолжение таблицы 5.3

Тест	Ожидаемый результат	Результат
7 Изменить скорость воспроизведения. 8 Нажать кнопку выхода из просмотра.	7 Скорость воспроизведения изменяется; выбранное значение визуально выделено. 8 Пользователь возвращается к списку повторов.	
Просмотр истории матчей 1 Открыть веб-интерфейс будучи авторизованным. 2 Перейти в раздел «Повторы». 3 Открыть вкладку «История матчей». 4 Найти запись завершеного матча. 5 Нажать кнопку «Показать полную статистику». 6 Нажать кнопку «Скрыть полную статистику».	1 Отображается основной интерфейс авторизованного пользователя. 2 Открыт раздел повторов. 3 Отображается список матчей пользователя. 4 В записи матча отображаются команда, роль, результат, причина завершения и длительность. 5 Раскрывается таблица статистики участников. 6 Таблица статистики скрывается, запись матча остается в списке.	Успех

Результаты ручного тестирования показали корректную работу пользовательского интерфейса и основных игровых сценариев. В ходе проверки были подтверждены работоспособность многопользовательского взаимодействия, корректность отображения игрового состояния и обработка пользовательских действий в режиме реального времени.

6 МЕТОДИКА ИСПОЛЬЗОВАНИЯ ПРОГРАММНОГО СРЕДСТВА

Разработанное программное средство представляет собой веб-приложение для многопользовательского взаимодействия в онлайн игре с физической моделью. Работа с приложением выполняется через браузер. Основные действия пользователя связаны с регистрацией и авторизацией, созданием или подключением к игровой комнате, участием в матче, просмотром сохранённых повторов, наблюдением за активными играми, управлением профилем, друзьями и локальными настройками интерфейса.

6.1 Страница входа в аккаунт

При первом обращении к программному средству без активной пользовательской сессии открывается страница входа, представленная на рисунке 6.1. На странице отображается форма авторизации с полями «Логин» и «Пароль», кнопкой «Войти», а также ссылка для перехода к регистрации.

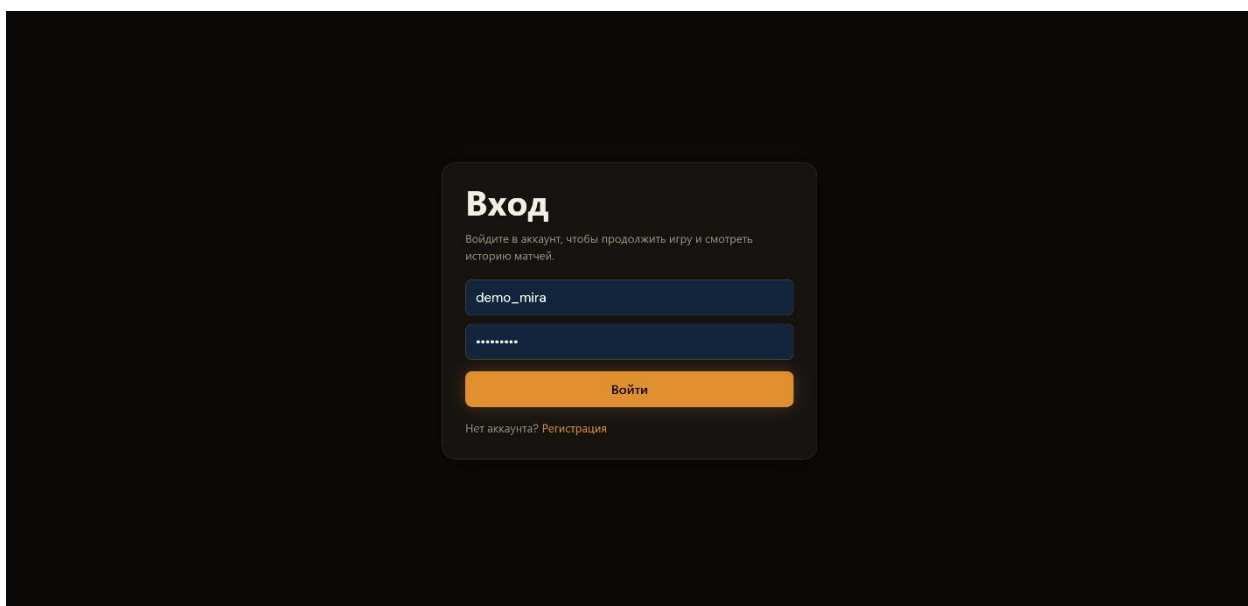


Рисунок 6.1 – Страница входа

Для входа в систему необходимо ввести логин и пароль зарегистрированного пользователя, после чего нажать кнопку «Войти». При успешной авторизации выполняется переход на главную страницу приложения. Если введены неверные учётные данные или сервер возвращает ошибку авторизации, под формой отображается соответствующее сообщение.

6.2 Страница регистрации аккаунта

Если пользователь не имеет учётной записи, необходимо перейти по ссылке «Регистрация», расположенной под формой входа. На открывшейся странице регистрации, показанной на рисунке 6.2, пользователь заполняет поля «Логин», «Email», «Имя в игре» и «Пароль».

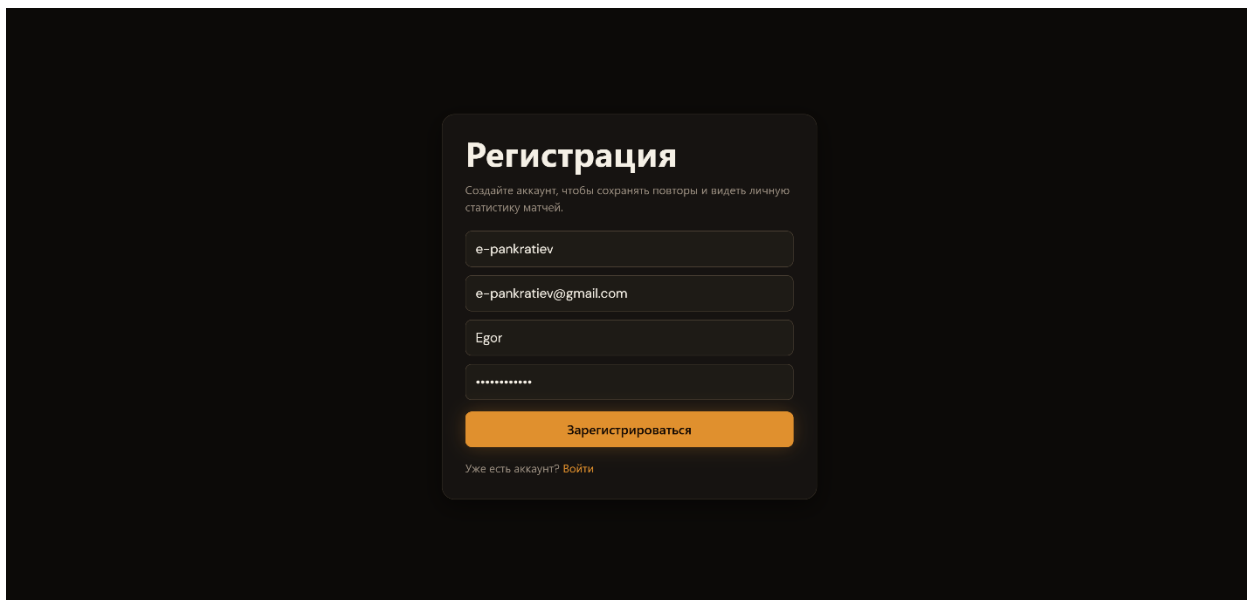
The image shows a registration form titled "Регистрация" (Registration) on a dark background. Below the title is a subtitle: "Создайте аккаунт, чтобы сохранять повторы и видеть личную статистику матчей." (Create an account to save repeats and see your personal match statistics). The form contains four input fields: "e-pankratiev" for the login, "e-pankratiev@gmail.com" for the email, "Egor" for the in-game name, and a password field with masked characters. Below these fields is an orange button labeled "Зарегистрироваться" (Register). At the bottom, there is a link that says "Уже есть аккаунт? Войти" (Already have an account? Log in).

Рисунок 6.2 – Страница регистрации

После заполнения формы необходимо нажать кнопку «Зарегистрироваться». При успешном выполнении операции создаётся новая учётная запись, после чего выполняется переход на главную страницу приложения. Если указанный логин или адрес электронной почты уже используются, регистрация не выполняется, а на странице отображается сообщение об ошибке.

6.3 Страница главного раздела

После авторизации пользователь попадает на главную страницу приложения, представленную на рисунке 6.3. На ней отображается приветствие с именем пользователя, сводка последней активности, блок продолжения игры, последний выбранный режим, список последних повторов и сведения о друзьях, находящихся в сети. Главная страница используется как начальная точка для перехода к основным разделам приложения и быстрого запуска игры.

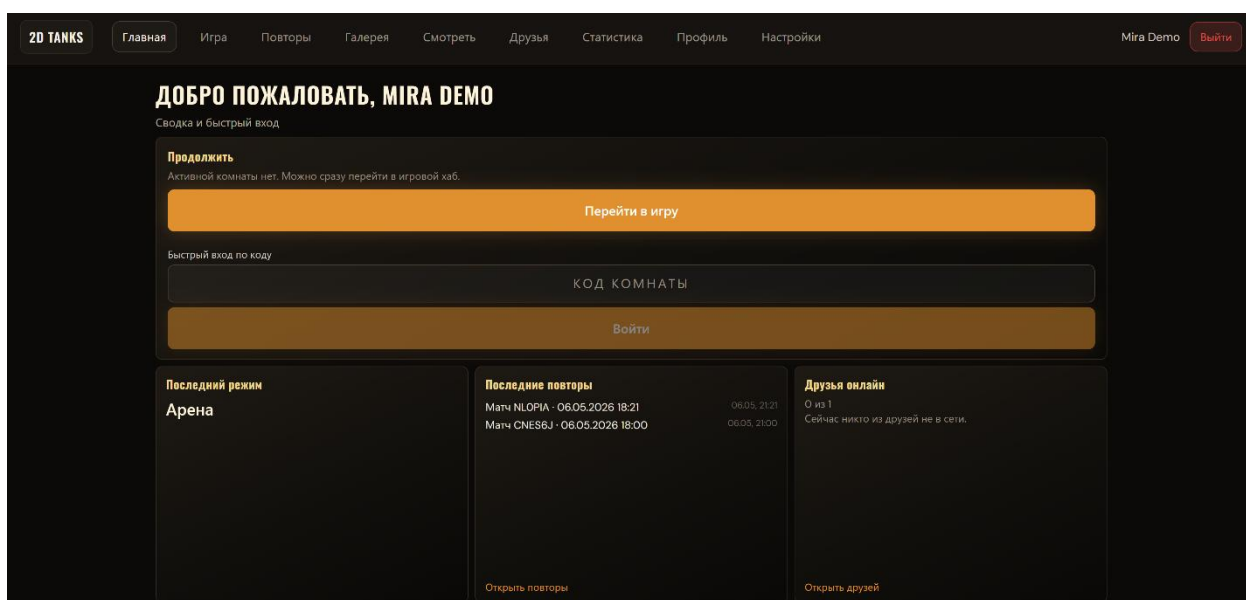


Рисунок 6.3 – Главная страница

В блоке «Продолжить» пользователь может вернуться в активную комнату, если такая комната существует, либо перейти к выбору режима игры. В блоке «Быстрый вход по коду» предусмотрено поле для ввода шестизначного кода комнаты. После ввода кода и нажатия кнопки «Войти» приложение открывает раздел игры и подготавливает подключение к указанной комнате.

В верхней части главной страницы и остальных основных разделов отображается общая шапка приложения. В ней расположены логотип-ссылка «2D Tanks», пункты меню «Главная», «Игра», «Повтор», «Галерея», «Смотреть», «Друзья», «Статистика», «Профиль», «Настройки», имя текущего пользователя и кнопка «Выйти». При выборе пункта меню открывается соответствующий раздел. Активный раздел выделяется в навигации, что позволяет пользователю определить текущее местоположение в приложении. Переход по логотипу «2D Tanks» возвращает пользователя на главную страницу.

6.4 Страница раздела «Игра»

Раздел «Игра» открывается по ссылке в шапке приложения. На начальном экране, представленном на рисунке 6.4, пользователь выбирает режим матча: классика, тренировка или арена. Также на странице доступны создание новой комнаты и подключение к существующей комнате по коду комнаты.

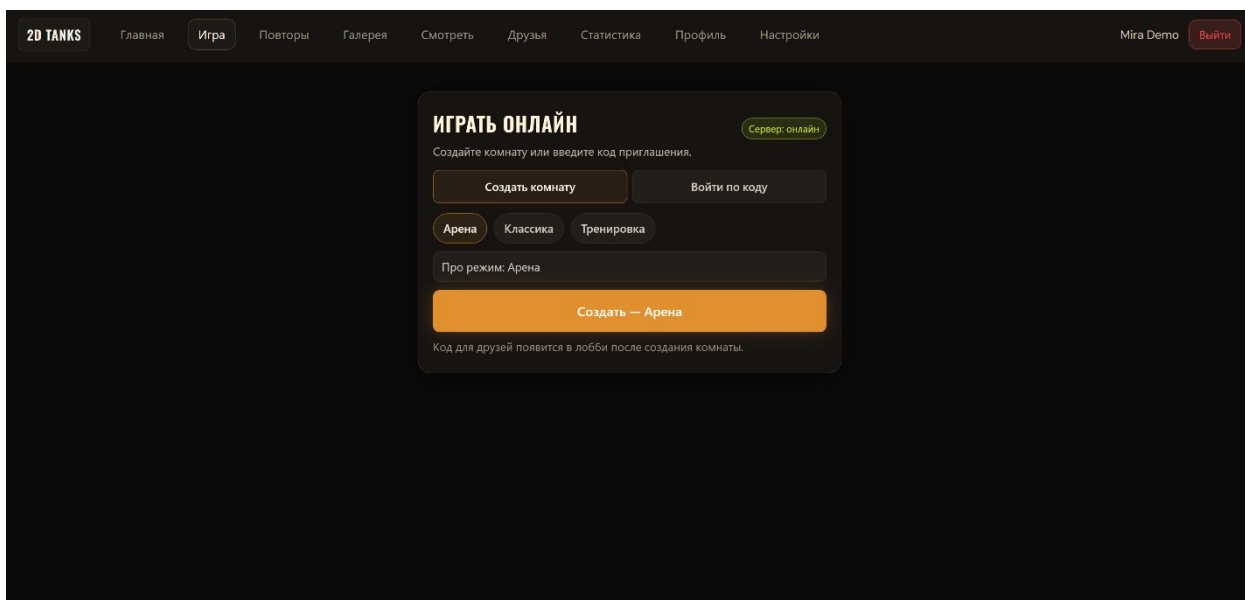


Рисунок 6.4 – Страница выбора режима игры

Для создания комнаты необходимо выбрать подходящий режим матча и нажать кнопку создания. После этого приложение формирует новую игровую комнату и переводит пользователя к следующему этапу подготовки. Для подключения к существующей комнате необходимо перейти на вкладку входа по коду, ввести код приглашения и нажать кнопку «Войти». Такой способ используется, если комнату уже создал другой участник.

Таким образом, раздел «Игра» объединяет два основных сценария начала матча: создание новой комнаты и подключение к уже созданной комнате по коду приглашения. После выбора одного из этих сценариев пользователь переходит к подготовке собственной игровой конфигурации.

6.5 Страница выбора танка

Перед входом в комнату пользователю предлагается настроить конфигурацию танка. На странице отображаются доступные модули танка и область предварительного просмотра. Пользователь может выбрать корпус, башню, гусеницы, оружие и цветовое оформление, оценивая изменения внешнего вида до подтверждения выбора. Каждый элемент конфигурации влияет на характеристики танка: корпус определяет запас прочности и устойчивость, башня — скорость поворота и сектор наведения, гусеницы — скорость движения и манёвренность, оружие — урон, скорострельность и тип снарядов. Это позволяет подобрать подходящий стиль игры. Страница выбора танка представлена на рисунке 6.5.

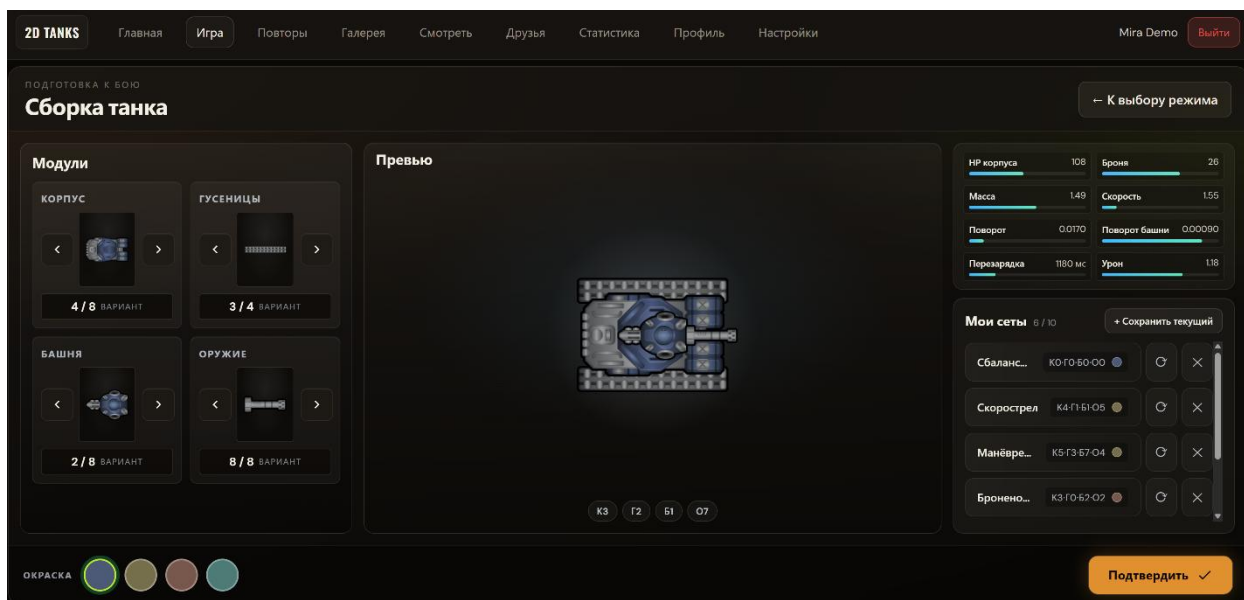


Рисунок 6.5 – Страница выбора танка

После настройки необходимо нажать кнопку «Подтвердить», после чего выбранная конфигурация передаётся в комнату ожидания.

6.6 Страница комнаты

После подтверждения конфигурации открывается страница комнаты, представленная на рисунке 6.6. На ней отображаются код комнаты, список участников, выбранный режим, состояние готовности игроков и чат.

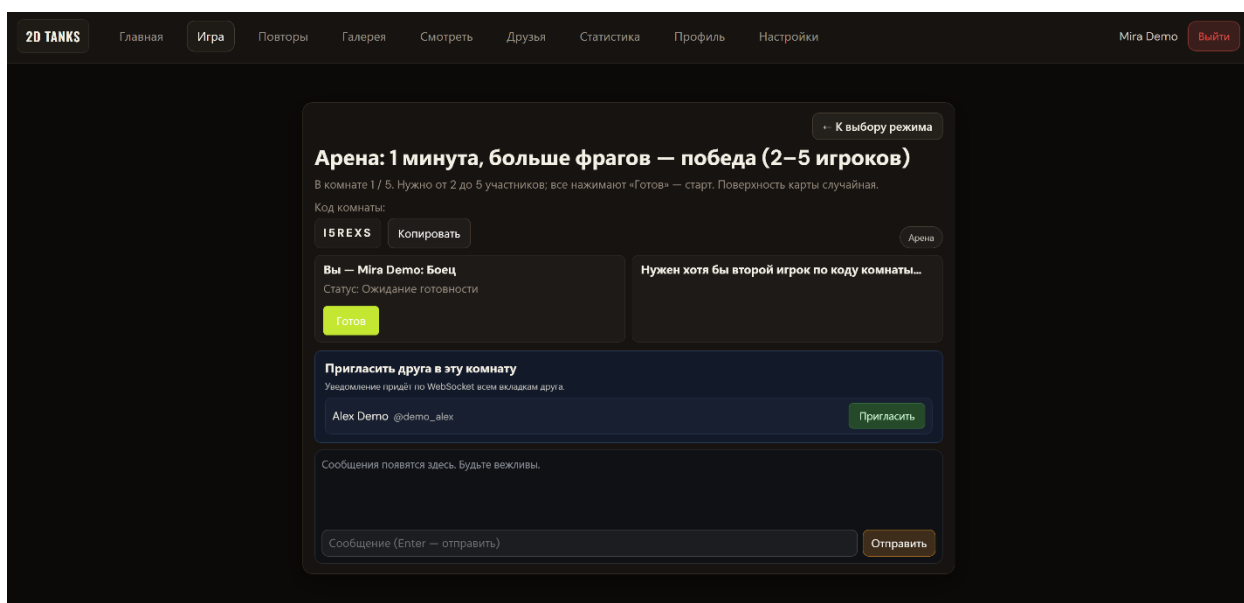


Рисунок 6.6 – Страница комнаты

Код комнаты может быть передан другому пользователю для подключения к матчу. Также пользователь может пригласить друга непосредственно из интерфейса комнаты с помощью соответствующей кнопки. Для начала игры участники должны подтвердить готовность. В зависимости от режима и количества игроков приложение ожидает второго участника либо позволяет начать матч после выполнения условий готовности.

6.7 Страница игры

После начала матча открывается игровая сцена, на которой выполняется основное взаимодействие пользователя с программным средством. Пользователь управляет танком, перемещается по карте, выбирает направление движения и ведёт огонь по противникам или объектам окружения. На экране отображаются танки участников, препятствия, подбираемые объекты, снаряды и элементы интерфейса, как показано на рисунке 6.7.

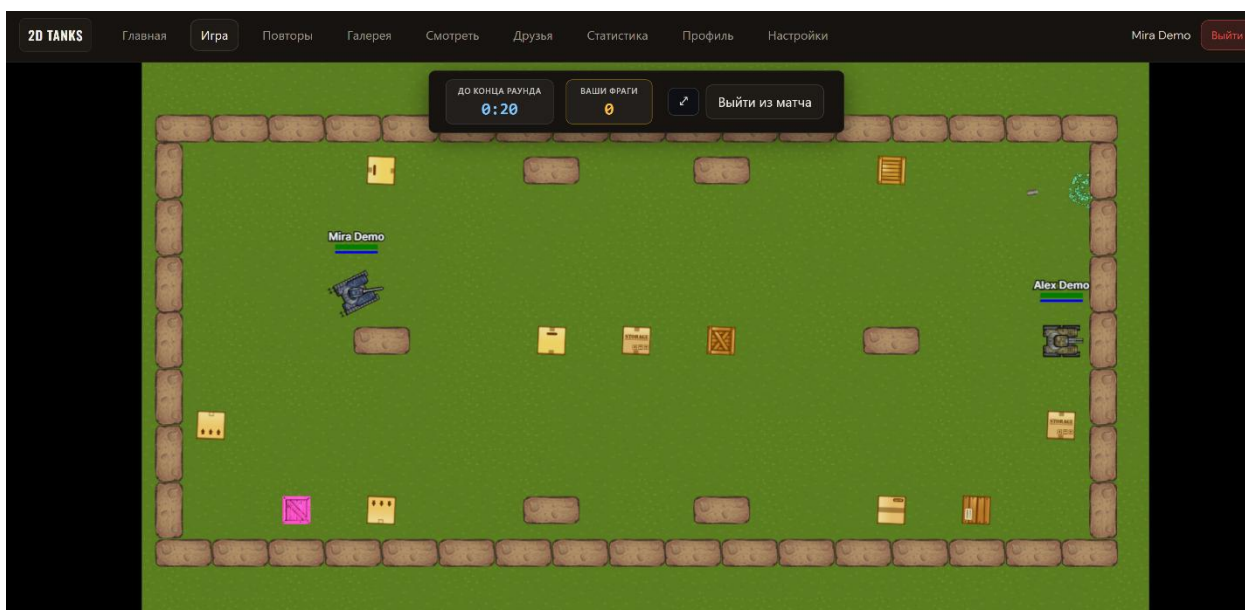


Рисунок 6.7 – Страница игры

В интерфейсе отображаются показатели состояния танка, информация о боеприпасах, событиях матча, текущем счёте, доступных действиях и состоянии подключения. Состояние матча обновляется в реальном времени, поэтому перемещения участников, попадания и изменения игровых объектов сразу отображаются на экране. Пользователь остаётся на игровой сцене до завершения боя или выхода из комнаты. При случайной перезагрузке

страницы или повторном открытии приложения пользователь может вернуться в активный матч, если комната ещё существует, а игра не завершена.

6.8 Страница окончания игры

После завершения матча открывается страница окончания игры. На ней отображается итоговый результат матча. Страница позволяет пользователю сразу оценить исход боя и перейти к просмотру основных результатов. На экране выводится сообщение, которое показывает состояние матча и результат участия пользователя, как показано на рисунке 6.8.

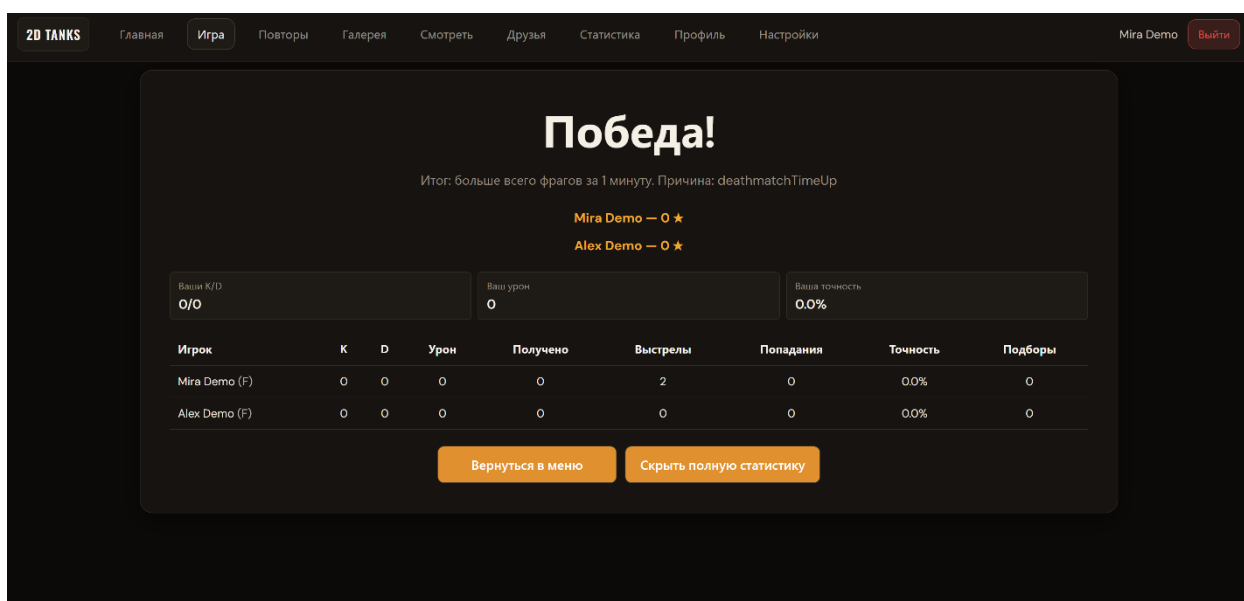


Рисунок 6.8 – Страница окончания игры

На странице результатов выводятся итоговые показатели участников: число уничтожений, смертей, нанесённый и полученный урон, количество выстрелов, попаданий и другие статистические значения. Эти данные позволяют сравнить действия. После просмотра результатов пользователь может вернуться в меню игры.

6.9 Страница раздела «Повтор»

Раздел «Повтор» предназначен для просмотра сохранённых записей матчей текущего пользователя и истории завершённых игр. Он используется для возврата к ранее сыгранным матчам, анализа хода игры и просмотра основных сведений о завершённых боях. На странице раздела отображаются

списки доступных записей и матчей, представленные на рисунке 6.9, с краткими результатами игр.

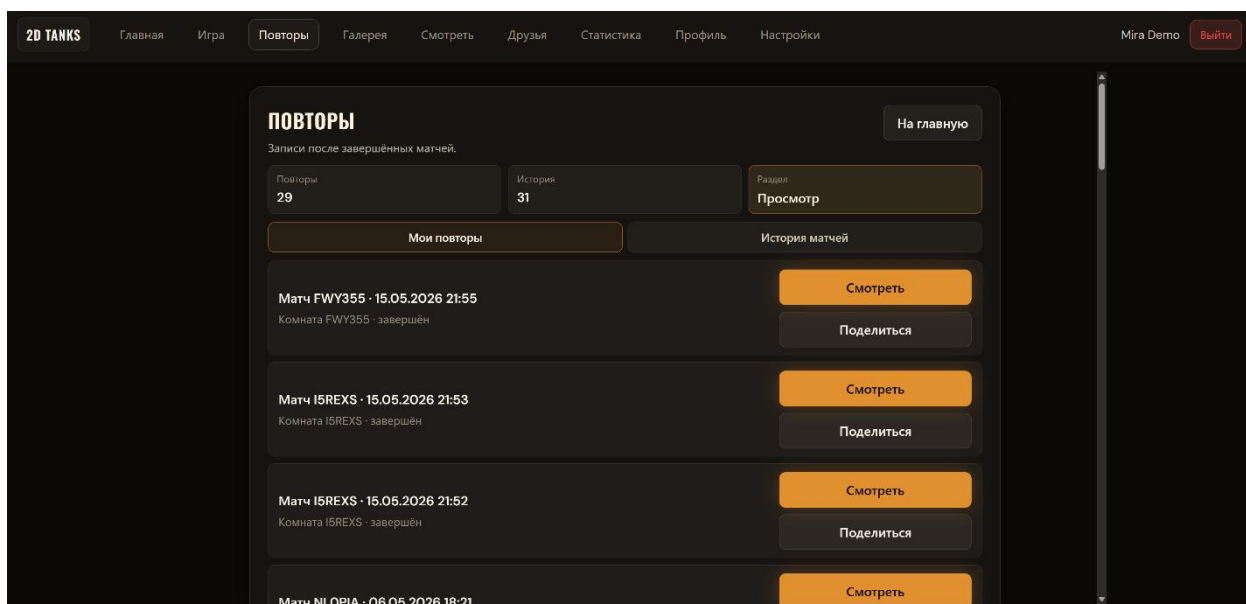


Рисунок 6.9 – Страница списка сохранённых повторов

В разделе доступны вкладки «Мои повторы» и «История матчей». На вкладке «Мои повторы» отображаются записи завершённых матчей, которые можно открыть для воспроизведения. Для каждой записи показываются краткие сведения о матче, что позволяет выбрать нужный повтор из списка. На вкладке «История матчей» отображаются сведения о сыгранных матчах и подробная статистика по участникам.

6.10 Страница просмотра сохранённого повтора

При выборе конкретной записи в разделе «Повтор» открывается страница просмотра сохранённого повтора. На ней отображается проигрыватель матча, позволяющий повторно просмотреть ход игры. Пользователь может запустить или приостановить воспроизведение, перемещаться по записи, менять положение камеры и просматривать события боя в нужном темпе. Повтор воспроизводит последовательность действий участников и изменение состояния игровых объектов, поэтому пользователь может проследить развитие матча от начала до завершения. На экране также отображаются элементы управления воспроизведением и область игровой сцены, представленная на рисунке 6.10.

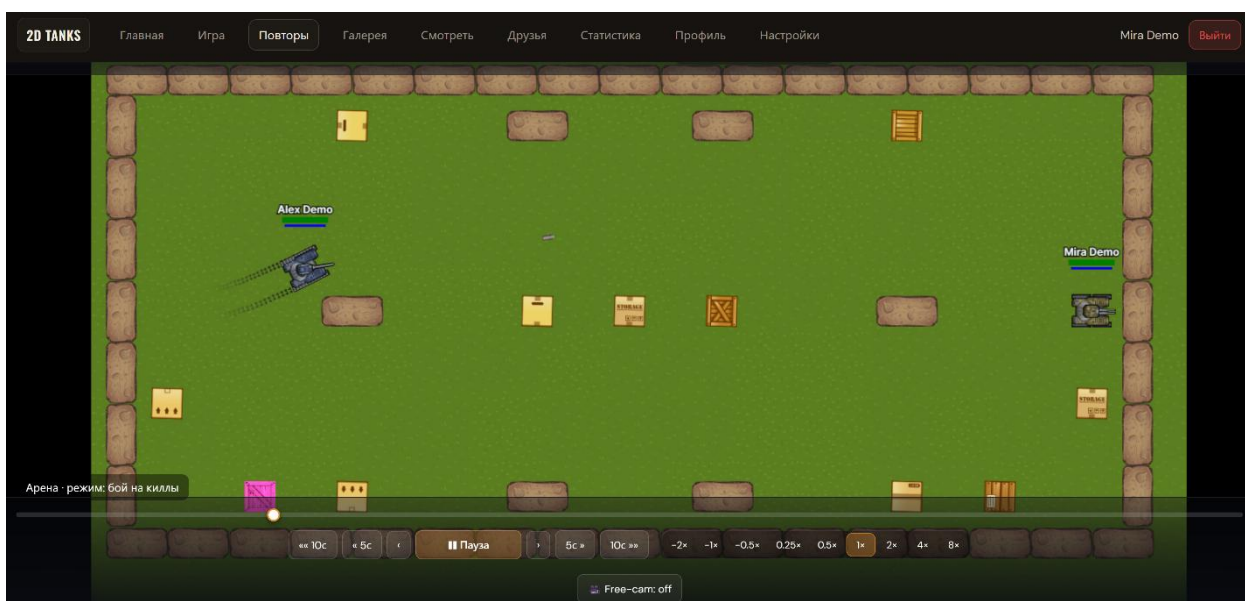


Рисунок 6.10 – Страница просмотра сохранённого повтора

После просмотра пользователь может вернуться к списку сохранённых повторов. Этот экран используется только для ознакомления с записью матча.

6.11 Страница раздела «Друзья»

Раздел «Друзья» используется для управления социальными связями между игроками. Страница раздела представлена на рисунке 6.11.

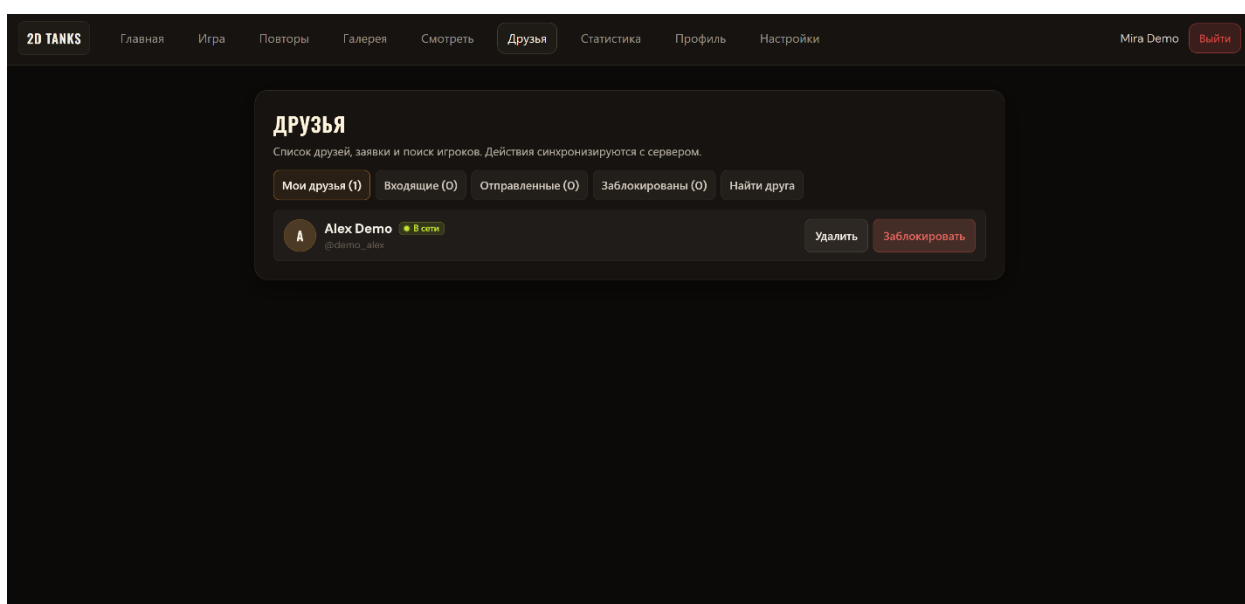


Рисунок 6.11 – Страница управления друзьями

На странице доступны вкладки «Мои друзья», «Входящие», «Отправленные», «Заблокированы» и «Найти друга». В названиях вкладок отображается количество записей в соответствующих списках. Для каждого пользователя показываются аватар, имя в игре, логин и текущий сетевой статус. Для добавления друга необходимо открыть вкладку поиска, ввести логин или имя игрока и нажать кнопку «Добавить». Входящие заявки можно принять или отклонить, отправленные заявки – отменить, а заблокированных пользователей – разблокировать. После выполнения действия на странице отображается краткое уведомление о результате операции.

6.12 Страница настроек приложения

Раздел «Настройки» содержит параметры внешнего вида и управления. Страница настроек представлена на рисунке 6.12.

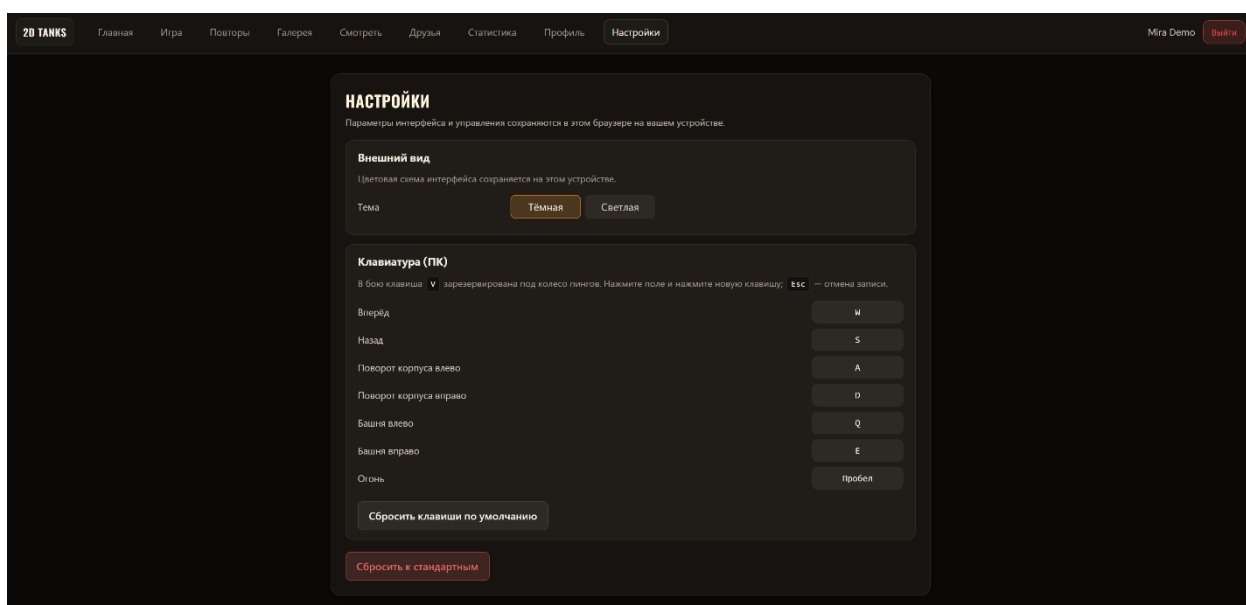


Рисунок 6.12 – Страница настроек

В блоке внешнего вида пользователь выбирает тему оформления приложения. В блоке управления можно изменить назначение клавиш для действий в игре или сбросить раскладку к значениям по умолчанию. Настройки сохраняются локально в браузере пользователя и применяются при последующих посещениях приложения с того же устройства.

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО СРЕДСТВА МНОГОПОЛЬЗОВАТЕЛЬСКОГО ВЗАИМОДЕЙСТВИЯ В ОНЛАЙН ИГРЕ С ФИЗИЧЕСКОЙ МОДЕЛЬЮ НА JAVASCRIPT-ПЛАТФОРМЕ

7.1 Характеристика разработанного программного средства

Разработанное программное средство представляет собой систему организации многопользовательского взаимодействия в онлайн игре с физической моделью. Система предназначена для обеспечения совместного участия пользователей в игровом процессе в режиме реального времени с синхронизацией действий участников и моделированием поведения игровых объектов.

Разработанное программное средство образует программную среду, в которой несколько пользователей могут одновременно взаимодействовать в едином виртуальном пространстве. Система обеспечивает обработку действий игроков, управление состоянием игрового мира и отображение результатов взаимодействия участников.

Разработанное программное средство предназначено для повышения качества пользовательского взаимодействия в сетевых игровых приложениях, а также для демонстрации принципов построения многопользовательских систем реального времени. Система обеспечивает согласованное состояние игрового пространства для всех участников и поддерживает динамическое изменение игровой ситуации.

Программное средство обладает следующими функциональными возможностями:

- подключение пользователей к игровой сессии;
- управление процессом взаимодействия игроков в виртуальном пространстве;
- обработка действий участников игрового процесса;
- моделирование движения и взаимодействия игровых объектов;
- обнаружение и обработка столкновений объектов игрового мира;
- синхронизация состояния игрового пространства между участниками;
- визуальное отображение игрового процесса;
- отображение результатов игровых действий и событий.

В составе разработанного программного средства реализованы следующие возможности:

- регистрация и аутентификация;

- создание и подключение к игровым комнатам;
- управление списком друзей и приглашениями в игровые комнаты;
- наблюдение за матчами;
- история игр и статистика;
- сохранение и воспроизведение повторов.

Среди преимуществ разработанного программного средства можно выделить:

- возможность организации многопользовательского взаимодействия в режиме реального времени;
- согласованность состояния игрового мира для всех участников;
- динамичное изменение игровой ситуации в процессе взаимодействия пользователей;
- возможность расширения функциональности и добавления новых игровых режимов;
- удобство использования и доступность для широкого круга пользователей.

На рынке существуют различные многопользовательские игровые платформы и системы разработки сетевых игр. Однако многие из них обладают высокой сложностью внедрения, требуют использования специализированных инструментов разработки или ориентированы на крупные игровые проекты.

Разработанное программное средство отличается более простой архитектурой, ориентированностью на демонстрацию принципов построения сетевых игровых систем и возможностью дальнейшего расширения функциональности.

Разработанная система может использоваться разработчиками программного обеспечения, студентами и исследователями, изучающими технологии многопользовательского взаимодействия, а также пользователями, заинтересованными в участии в сетевых игровых приложениях.

7.2 Расчет затрат на разработку программного средства

Затраты на основную заработную плату команды разработчиков определяются исходя из состава и численности команды, размеров месячной заработной платы каждого из участников команды, а также общей трудоемкости разработки программного обеспечения [29].

Расчет затрат на разработку программного средства следует делать в разрезе следующих статей:

- затраты на основную заработную плату разработчиков;
- затраты на дополнительную заработную плату разработчиков;
- отчисления на социальные нужды;
- прочие затраты (амортизационные отчисления, расходы на электроэнергию, командировочные расходы, арендная плата за офисные помещения и оборудование).

7.2.1 Расчет затрат на основную заработную плату разработчикам

Расчет затрат на основную заработную плату разработчиков осуществляется по формуле 7.1:

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_{\text{чи}}, \quad (7.1)$$

где n – количество исполнителей, занятых разработкой конкретного ПО;

$K_{\text{п}}$ – коэффициент, учитывающий процент премий ($K_{\text{пр}} = 1,5$);

$Z_{\text{чи}}$ – часовая заработная плата i -го исполнителя, р.;

$t_{\text{чи}}$ – трудоемкость работ, выполняемых i -м исполнителем, ч.

Расчет затрат на основную заработную плату разработчиков осуществляется в табличной форме (табл. 7.1).

Таблица 7.1 – Расчет затрат на основную заработную плату разработчиков

Наименование должности разработчика	Вид выполняемой работы	Месячная заработная плата, р	Часовая заработная плата, р	Трудоемкость работ, ч	Сумма, р
Инженер-программист первой категории	Проектирование архитектуры системы	3600	21,43	200	4286
Инженер-программист первой категории	Разработка серверной логики и игровой симуляции	3600	21,43	260	5572

Продолжение таблицы 7.1

Наименование должности разработчика	Вид выполняемой работы	Месячная заработная плата, р	Часовая заработная плата, р	Трудоемкость работ, ч	Сумма, р
Инженер-программист второй категории	Разработка пользовательского интерфейса	3400	20,24	220	4453
Тестирующий программного обеспечения	Обеспечение качества	2700	16,07	120	1928
Итого					16239
Премия (50%)					8119,5
Всего основная заработная плата					24358,5

7.2.2 Расчет затрат на дополнительную заработную плату

Затраты на дополнительную заработную плату разработчикам включают выплаты, предусмотренные законодательством, определяются по формуле 7.2:

$$З_d = \frac{З_o \cdot Н_d}{100}, \quad (7.2)$$

где $З_o$ – затраты на основную заработную плату, р.;

$Н_d$ – норматив дополнительной заработной платы ($Н_d = 15\%$).

Дополнительная заработная плата разработчиков составит:

$$З_d = \frac{24358,5 \cdot 15}{100} = 3653,78 \text{ р.}$$

7.2.3 Расчет отчисления на социальные нужды

Отчисления на социальные нужды включают предусмотренные законодательством отчисления в фонд социальной защиты и фонд обязательного страхования и определяются по формуле 7.3:

$$P_{\text{соц}} = \frac{(3_o + 3_d) \cdot H_{\text{соц}}}{100}, \quad (7.3)$$

где $H_{\text{соц}}$ – норматив отчислений от фонда оплаты труда ($H_{\text{соц}} = 34,6\%$).

Отчисления на социальные нужды составят:

$$P_{\text{соц}} = \frac{(24358,5 + 3653,78) \cdot 34,6}{100} = 9692,25 \text{ р.}$$

7.2.4 Расчет прочих затрат

Прочие затраты включают аренду помещения, амортизацию основных средств, затраты на инфраструктуру, хранение данных и пр. Прочие затраты определяются по формуле 7.4:

$$P_{\text{пз}} = \frac{3_o \cdot H_{\text{пз}}}{100}, \quad (7.4)$$

где $H_{\text{пз}}$ – норматив прочих затрат ($H_{\text{пз}} = 30\%$).

Прочие затраты составят:

$$P_{\text{пз}} = \frac{24358,5 \cdot 30}{100} = 7307,55 \text{ р.}$$

7.2.5 Расчет суммарных затрат

Общая сумма затрат на разработку представляет собой себестоимость программы, выраженная в денежной форме, вычисляется по формуле 7.5:

$$3_p = 3_o + 3_d + P_{\text{соц}} + P_{\text{пз}} \quad (7.5)$$

Полученные значения затрат на разработку по основным статьям представлены в таблице 7.2.

Таблица 7.2 – Затраты на разработку веб-приложения и отпускная цена

Статья затрат	Сумма, р
Основная заработная плата команды разработки	24358,5
Дополнительная заработная плата команды разработки	3653,78
Отчисления на социальные нужды	9692,25
Прочие затраты	7307,55
Общая сумма затрат на разработку	45012,08

7.3 Расчет результата от разработки и использования программного средства

Экономический эффект разрабатываемого программного средства определяется как прибыль от реализации лицензий на использование игровой системы.

Разрабатываемое программное средство предназначено для пользователей, заинтересованных в участии в многопользовательских онлайн играх, а также для разработчиков и студентов, изучающих принципы построения сетевых игровых приложений. Потенциальными потребителями являются пользователи персональных компьютеров и браузерных платформ, использующие веб-приложения для развлечения и обучения. Дополнительным фактором, влияющим на возможность коммерческого использования, является наличие серверной части, обеспечивающей синхронизацию игрового состояния, хранение пользовательских данных и поддержку многопользовательского взаимодействия в режиме реального времени.

С учётом специфики распространения веб-игр, прогнозируемый объём использования программного средства целесообразно оценивать как количество активных платных пользователей за год. Для расчёта экономического эффекта принимается, что часть пользователей использует бесплатную версию игры, а часть приобретает лицензию для получения расширенного функционала и дополнительных возможностей.

Обоснование прогнозируемого объёма реализации лицензий выполняется на основе оценки потенциальной аудитории и типовых коэффициентов конверсии для онлайн игр. Потенциальными пользователями являются лица, заинтересованные в простых многопользовательских браузерных играх. При этом для нового продукта характерны ограниченные показатели распространения вследствие отсутствия известности, наличия конкурирующих решений и необходимости времени на формирование пользовательской базы.

Для расчётной оценки принимаются следующие допущения, характерные для начального этапа распространения:

Охват аудитории на первом году ограничен за счёт отсутствия активного маркетинга и осуществляется преимущественно за счёт органического распространения (публикация в сети Интернет, демонстрация возможностей, ограниченные каналы продвижения). В таких условиях реалистичным является достижение порядка 40 000 пользователей за год.

Доля активных пользователей среди зарегистрированных, как правило, значительно ниже 100%, поскольку часть пользователей прекращает

использование приложения. Для расчёта принимается, что 25% пользователей остаются активными.

Конверсия в приобретение лицензии для игровых продуктов зависит от стоимости и доступности функционала. Поскольку разрабатываемое программное средство имеет невысокую стоимость и ориентировано на массового пользователя, ожидается повышенный уровень конверсии по сравнению с типовыми значениями. В расчётной модели принимается конверсия 20% от числа активных пользователей.

Таким образом, прогнозируемый объём реализации определяется по формуле 7.6:

$$N = N_d \cdot \frac{k_a}{100} \cdot \frac{k_l}{100}, \quad (7.6)$$

где N_d – количество установленных копий, шт.;
 k_a – коэффициент доли активных пользователей среди установивших приложение ($k_a = 25\%$);
 k_l – коэффициент конверсии активных пользователей в приобретение лицензии ($k_l = 20\%$).

С учётом принятых исходных коэффициентов прогнозируемое количество приобретенных лицензий составит:

$$N = 40000 \cdot \frac{25}{100} \cdot \frac{20}{100} = 2000 \text{ шт.}$$

Выбранное значение является реалистичным для начального этапа распространения, поскольку учитывает ограниченный охват аудитории и при этом отражает влияние низкой стоимости продукта на увеличение доли платных пользователей. При дальнейшем развитии программного средства и расширении функциональности объём реализации может быть увеличен.

Предполагаемая стоимость лицензии устанавливается на уровне, доступном для массового пользователя и достаточном для покрытия затрат на поддержку серверной инфраструктуры. Стоимость годовой лицензии принимается равной 100 рублей в год на одного пользователя.

Для обоснования выбранной стоимости лицензии был выполнен анализ ценовой политики аналогичных игровых решений.

Например, для аналогов характерна следующая ценовая политика:

– в игре *Agar.io* реализована платная подписка, стоимость которой составляет около 7,99 долл. США в неделю. В пересчёте на год это составляет около 415,48 долл. США или 1246,44 руб. в год;

– в игре *Krunker* предусмотрена подписка «Premium», стоимость которой составляет 4,99 долл. США в месяц. В пересчёте на год это составляет 59,88 долл. США или 179,64 руб. в год.

Таким образом, стоимость подписки на аналогичные игровые продукты находится в диапазоне от 179,64 до 1246,44 руб. в год.

Установленная стоимость лицензии на разрабатываемое программное средство в размере 100 рублей в год является значительно ниже по сравнению с аналогичными решениями. Это обеспечивает доступность продукта для широкой аудитории пользователей, повышает его конкурентоспособность и способствует увеличению вероятности успешного внедрения на начальном этапе распространения.

Налог на добавленную стоимость рассчитывается по формуле 7.7:

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{дс}}}{100 + \text{Н}_{\text{дс}}}, \quad (7.7)$$

где $\text{Н}_{\text{дс}}$ – ставка налога на добавленную стоимость согласно действующему законодательству ($\text{Н}_{\text{дс}} = 20\%$).

Тогда налог на добавленную стоимость составит:

$$\text{НДС} = \frac{100 \cdot 2000 \cdot 20}{100 + 20} = 33333,33 \text{ р.}$$

Прибыль от реализации ПО на рынке вычисляется по формуле 7.8:

$$\text{П} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} - \text{З}_{\text{р}}, \quad (7.8)$$

где $\text{Ц}_{\text{отп}}$ – отпускная цена копии (лицензии) программного средства, р.;

N – количество копий (лицензий) программного средства, реализуемое за год, шт.;

НДС – сумма налога на добавленную стоимость, р.;

$\text{Р}_{\text{пр}}$ – рентабельность продаж копий лицензий ($\text{Р}_{\text{пр}} = 35\%$).

Прибыль от реализации ПО на рынке составит:

$$\text{П} = \frac{(100 \cdot 2000 - 33333,33) \cdot 35}{100} - 45012,08 = 13321,25 \text{ р.}$$

Согласно действующему налоговому законодательству ставка налога на прибыль составляет 20 %. Если организация является плательщиком налога на прибыль, то экономический эффект рассчитывается по формуле 7.9:

$$\Pi_{\text{ч}} = \Pi \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (7.9)$$

где $H_{\text{п}}$ – ставка налога на прибыль согласно действующему законодательству ($H_{\text{п}} = 20\%$);

Чистая прибыль после уплаты налога на прибыль:

$$\Pi_{\text{ч}} = 13321,25 \cdot \left(1 - \frac{20}{100}\right) = 10657 \text{ р.}$$

Уровень рентабельности рассчитывается по формуле 8.0:

$$Y_{\text{р}} = \frac{\Pi_{\text{ч}}}{Z_{\text{р}}} \cdot 100. \quad (8.0)$$

Значение рентабельности равняется:

$$Y_{\text{р}} = \frac{10657}{45012,08} \cdot 100 = 23,68\%$$

Рентабельность затрат на разработку программного средства многопользовательского взаимодействия в онлайн игре с физической моделью превышает среднюю процентную ставку по банковским депозитным вкладам, равную 14,5%. Следовательно, разработка данного программного средства является экономически целесообразной.

В результате проведенного технико-экономического обоснования были получены следующие показатели:

- общая сумма инвестиций в разработку – 45012,08 р.;
- чистая прибыль – 10657 р.;
- уровень рентабельности – 23,68 % (при цене реализации лицензии 100 р. в год и количестве продаж 2000 лицензий).

Полученные результаты технико-экономического обоснования свидетельствуют об эффективности разработки и высоком уровне рентабельности.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была достигнута поставленная цель – обеспечение согласованного многопользовательского игрового процесса в режиме реального времени за счёт реализации клиент-серверной архитектуры, механизмов синхронизации состояния игрового мира и физической симуляции. Все задачи, сформулированные во введении, были решены в полном объёме.

Исследована предметная область многопользовательских онлайн игр и выполнен анализ архитектурных решений распределённых систем, алгоритмов пространственной оптимизации, методов детекции коллизий, сетевых технологий и существующих аналогов. Показано, что для интерактивных систем реального времени наиболее целесообразным является использование клиент-серверной архитектуры с авторитарной ролью сервера и постоянным двусторонним обменом данными по протоколу *WebSocket*.

Проведён анализ требований к программному средству и разработаны функциональные требования, определяющие поведение системы при организации игровых сессий, синхронизации состояния игрового мира, обработке пользовательских действий и взаимодействии пользователей. Разработаны функциональная модель программного средства на основе *UML*-диаграммы вариантов использования и информационная модель базы данных, отражающая основные сущности предметной области и связи между ними.

Разработана архитектура программного средства, включающая клиентскую и серверную подсистемы, механизмы сетевого взаимодействия и хранения данных. Предложено решение на основе *Node.js*, *React*, *TypeScript* и *WebSocket*, обеспечивающее централизованное управление игровыми сессиями и согласованное обновление игрового состояния между подключёнными клиентами.

Спроектирована и реализована структура реляционной базы данных на основе СУБД *PostgreSQL*. Разработанная модель обеспечивает хранение учётных записей пользователей, параметров игровых сессий, статистики матчей, повторов и социальных взаимодействий между пользователями. Использование ограничений целостности и внешних ключей позволило обеспечить согласованность данных и возможность дальнейшего расширения системы.

Выполнена программная реализация игрового движка, включающего подсистемы физической симуляции, обнаружения столкновений и игрового цикла. Для повышения производительности системы применён алгоритм пространственного разбиения *Quadtree*, позволяющий сократить количество

проверок взаимодействий между объектами. Реализован алгоритм обнаружения столкновений на основе теоремы о разделяющей оси, обеспечивающий корректную обработку пересечений игровых объектов.

Реализована серверная часть программного средства, обеспечивающая обработку пользовательских команд, выполнение игровой логики, управление игровыми сессиями и синхронизацию состояния игрового мира. Серверная логика разработана на платформе *Node.js* с использованием *TypeScript* и протокола *WebSocket*, что позволило организовать устойчивый обмен данными между участниками игры в режиме реального времени.

Реализована клиентская часть программного средства на базе *React* и *TypeScript*. Клиентское приложение обеспечивает визуализацию игрового состояния, обработку пользовательского ввода, отображение результатов матчей, работу с игровыми комнатами, историей матчей и повторами.

Проведено тестирование разработанного программного средства. Результаты тестирования подтвердили корректность синхронизации состояния игрового мира, устойчивость обработки пользовательских действий и работоспособность системы при многопользовательском взаимодействии. Проверена корректность функционирования игровых механик, сетевого взаимодействия и пользовательского интерфейса.

Выполнено технико-экономическое обоснование разработки. Рассчитаны показатели экономической эффективности, подтверждающие целесообразность внедрения разработанного программного средства. Полученные результаты показывают, что разработка является экономически оправданной и готовой к практической эксплуатации.

Перспективы развития проекта включают расширение игровых режимов, внедрение системы рейтингового подбора игроков, оптимизацию сетевой синхронизации при высоких нагрузках, а также разработку дополнительных механизмов физического взаимодействия и пользовательского контента.

Разработанное программное средство соответствует поставленным требованиям, обеспечивает согласованное многопользовательское взаимодействие в режиме реального времени и может быть использовано в качестве основы для дальнейшего развития многопользовательских игровых систем на *JavaScript*-платформе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] *MDN Web Docs. Introduction to web APIs* [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Client-side_APIs/Introduction. – Дата доступа: 11.05.2026.
- [2] *MDN Web Docs. 2D collision detection* [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/en-US/docs/Games/Techniques/2D_collision_detection. – Дата доступа: 11.05.2026.
- [3] *Node.js Documentation. Getting started guide* [Электронный ресурс]. – Режим доступа: <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>. – Дата доступа: 11.05.2026.
- [4] Куроуз, Дж. Компьютерные сети / Дж. Куроуз, К. Росс. – 6-е изд. – М. : Вильямс, 2017. – 111–114 с.
- [5] *Gaffer on Games. What every programmer needs to know about game networking* [Электронный ресурс]. – Режим доступа: https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/. – Дата доступа: 11.05.2026.
- [6] *Dordal, P. L. An introduction to computer networks. Chapter 12: TCP transport* [Электронный ресурс]. – Режим доступа: <https://intronetworks.cs.luc.edu/1/html/tcp.html>. – Дата доступа: 11.05.2026.
- [7] *Gambetta, G. Client-side prediction and server reconciliation* [Электронный ресурс]. – Режим доступа: <https://www.gabrielgambetta.com/client-side-prediction-server-reconciliation.html>. – Дата доступа: 11.05.2026.
- [8] *Samet, H. Foundations of multidimensional and metric data structures / H. Samet.* – Burlington : Morgan Kaufmann, 2006. – 28 с.
- [9] *Nystrom, R. Game programming patterns. Spatial partition* [Электронный ресурс]. – Режим доступа: <https://gameprogrammingpatterns.com/spatial-partition.html>. – Дата доступа: 11.05.2026.
- [10] Эриксон, К. Обнаружение столкновений в реальном времени / К. Эриксон. – М. : ДМК Пресс, 2009. – 101 с.
- [11] *Metanet Software. Tutorial A: separation axis theorem for collision detection* [Электронный ресурс]. – Режим доступа: <https://www.metanetsoftware.com/technique/tutorialA.html>. – Дата доступа: 11.05.2026.
- [12] *dyn4j. SAT* [Электронный ресурс]. – Режим доступа: <https://dyn4j.org/2010/01/sat/>. – Дата доступа: 11.05.2026.
- [13] *MDN Web Docs. An overview of HTTP* [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>. – Дата доступа: 11.05.2026.

- [14] *JavaScript.info. Long polling* [Электронный ресурс]. – Режим доступа: <https://javascript.info/long-polling>. – Дата доступа: 11.05.2026.
- [15] *MDN Web Docs. WebSocket API (WebSockets)* [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API. – Дата доступа: 11.05.2026.
- [16] *Diep.io* [Электронный ресурс]. – Режим доступа: <https://diep.io/>. – Дата доступа: 11.05.2026.
- [17] *Tank Trouble* [Электронный ресурс]. – Режим доступа: <https://tanktrouble.com/>. – Дата доступа: 11.05.2026.
- [18] *ShellShock Live* [Электронный ресурс]. – Режим доступа: https://store.steampowered.com/app/326460/ShellShock_Live/. – Дата доступа: 11.05.2026.
- [19] *Fowler, M. UML distilled : a brief guide to the standard object modeling language / M. Fowler. – 3rd ed. – Boston : Addison-Wesley, 2004. – 82 с.*
- [20] *Connolly, T. M. Database systems : a practical approach to design, implementation, and management / T. M. Connolly, C. E. Begg. – 4th ed. – Harlow : Addison-Wesley, 2005. – 491 с.*
- [21] *Sommerville, I. Software engineering / I. Sommerville. – 10th ed. – Boston : Pearson, 2016. – 105 с.*
- [22] *Gambetta, G. Client-server game architecture* [Электронный ресурс]. – Режим доступа: <https://www.gabrielgambetta.com/client-server-game-architecture.html>. – Дата доступа: 11.05.2026.
- [23] *React. Thinking in React* [Электронный ресурс]. – Режим доступа: <https://react.dev/learn/thinking-in-react>. – Дата доступа: 11.05.2026.
- [24] *TypeScript Documentation. The TypeScript handbook* [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/docs/handbook/intro.html>. – Дата доступа: 11.05.2026.
- [25] *Nginx. Beginner's guide* [Электронный ресурс]. – Режим доступа: https://nginx.org/en/docs/beginners_guide.html. – Дата доступа: 11.05.2026.
- [26] *Express.js. Routing* [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/guide/routing.html>. – Дата доступа: 11.05.2026.
- [27] *node-postgres Documentation* [Электронный ресурс]. – Режим доступа: <https://node-postgres.com/>. – Дата доступа: 11.05.2026.
- [28] *PostgreSQL. About* [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/about/>. – Дата доступа: 11.05.2026.
- [29] *Технико-экономическое обоснование дипломных проектов (работ) : метод. указания / А. А. Горюшкин, В. Г. Горовой, В. А. Пархименко. – Минск : БГУИР, 2025. – 107 с.*

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода серверной части

```
import 'dotenv/config';
import http from 'http';
import { WebSocketServer, WebSocket } from 'ws';
import { RoomManager } from './room/RoomManager';
import { parseWsUserFromRequest } from './ws/parseWsUser';
import { resolveListenPort } from './serverPort';
import { createHttpApp } from './createHttpApp';
import { setRoomManager } from './room/roomManagerRuntime';

const PORT = resolveListenPort();
const app = createHttpApp();
const server = http.createServer(app);
const wss = new WebSocketServer({ server });
const roomManager = new RoomManager();
setRoomManager(roomManager);

server.listen(PORT, () => {
  console.log(`HTTP + WebSocket on port ${PORT}`);
});

wss.on('connection', (ws: WebSocket, req) => {
  const wsUser = parseWsUserFromRequest(req);
  if (!wsUser) {
    ws.send(JSON.stringify({ type: 'error', message: 'Authentication required'
  }));
    ws.close(4401, 'Authentication required');
    return;
  }

  let playerId: string | null = null;
  let roomCode: string | null = null;
  const restored = roomManager.reconnectByUser(ws, wsUser);
  if (restored) {
    roomCode = restored.code;
    playerId = restored.playerId;
  }
});
```

```

    }

    ws.on('message', (message: Buffer) => {
      try {
        const data = JSON.parse(message.toString());

        if (data.type === 'createRoom') {
          const mode = typeof data.mode === 'string' ? data.mode : 'deathmatch';
          const result = roomManager.createRoom(
            mode === 'solo',
            wsUser,
            mode === 'practice',
            mode === 'deathmatch'
          );
          roomCode = result.code;
          playerId = result.playerId;
          roomManager.getRoom(roomCode)?.updatePlayerWebSocket(playerId, ws);
        } else if (data.type === 'joinRoom') {
          const code = String(data.code || '').trim().toUpperCase();
          const result = roomManager.joinRoom(code, ws, wsUser);
          if (!result) {
            ws.send(JSON.stringify({ type: 'error', message: 'Room is full or
does not exist' }));
            return;
          }
          roomCode = code;
          playerId = result.playerId;
        } else if (data.type === 'tankConfig' && roomCode && playerId) {
          const result = roomManager.setTankConfig(roomCode, playerId,
data.data);
          if (!result.success) ws.send(JSON.stringify({ type: 'error', message:
result.message }));
        } else if (data.type === 'ready' && roomCode && playerId) {
          const result = roomManager.setReady(roomCode, playerId, data.ready);
          if (!result.success) ws.send(JSON.stringify({ type: 'error', message:
result.message }));
        } else if (data.type === 'action' && roomCode && playerId) {
          roomManager.handlePlayerAction(roomCode, playerId, data.action ||
data);

```

```

        } else if (data.type === 'ping:send' || data.type === 'chat:send' ||
data.type === 'spectateRoom') {
            }
        } catch (error) {
            ws.send(JSON.stringify({ type: 'error', message: 'Invalid message format'
})));
        }
    });

    ws.on('close', () => {
        if (roomCode && playerId) roomManager.handleDisconnect(roomCode, playerId);
    });
});

```

```

import { WebSocket } from 'ws';
import { GameWorld } from '../game/world/GameWorld';
import { TankConfig } from '../utils/types';
import type { WsAuthUser } from '../auth/types';
import * as matchRepo from '../repos/matchRepo';
import * as replayRepo from '../repos/replayRepo';

```

```

type PlayerRole = 'attacker' | 'defender' | 'fighter';

```

```

interface Player {
    id: string;
    ws: WebSocket | null;
    role: PlayerRole | null;
    tankConfig: TankConfig | null;
    ready: boolean;
    userId: string | null;
    displayName: string | null;
}

```

```

export type RoomCreateOptions = {
    singlePlayerTest?: boolean;
    practiceMode?: boolean;
    deathmatchMode?: boolean;
};

```

```

export class Room {
  private players: Map<string, Player> = new Map();
  private gameWorld: GameWorld | null = null;
  private gameLoopInterval: NodeJS.Timeout | null = null;
  private lastGameLoopTime = 0;
  private readonly TICK_RATE = 60;
  private readonly TICK_INTERVAL = 1000 / this.TICK_RATE;
  private readonly FIXED_DELTA_MS = 1000 / 60;
  private readonly singlePlayerTest: boolean;
  private readonly practiceMode: boolean;
  private readonly deathmatchMode: boolean;
  private readonly maxPlayers: number;
  private matchId: string | null = null;
  private replayEvents: replayRepo.ReplayEvent[] = [];
  private replayStartMeta: replayRepo.ReplayStartMeta | null = null;
  private static readonly REPLAY_MAX_ACTIONS = 500000;

  constructor(private code: string, options?: RoomCreateOptions) {
    this.singlePlayerTest = options?.singlePlayerTest === true;
    this.deathmatchMode = options?.deathmatchMode === true &&
!this.singlePlayerTest;
    this.practiceMode = options?.practiceMode === true && !this.singlePlayerTest
&& !this.deathmatchMode;
    this.maxPlayers = this.singlePlayerTest ? 1 : this.deathmatchMode ? 5 : 2;
  }

  addPlayer(ws: WebSocket | null, auth: WsAuthUser | null = null): string | null {
    if (this.players.size >= this.maxPlayers) return null;
    const playerId = `player_${Date.now()}_${Math.random()}`;
    const role: PlayerRole = this.deathmatchMode ? 'fighter' : this.players.size
=== 0 ? 'attacker' : 'defender';
    this.players.set(playerId, {
      id: playerId,
      ws,
      role,
      tankConfig: null,
      ready: false,
      userId: auth?.userId ?? null,

```

```

        displayName: auth?.displayName ?? null
    });
    ws?.send(JSON.stringify({ type: 'joined', roomId: this.code, playerId, role
})));
    this.broadcastRoomUpdate();
    return playerId;
}

```

```

setTankConfig(playerId: string, config: TankConfig): { success: boolean; message?:
string } {
    const player = this.players.get(playerId);
    if (!player) return { success: false, message: 'Player not found' };
    for (const other of this.players.values()) {
        if (other.id !== playerId && other.tankConfig?.color === config.color) {
            return { success: false, message: `Color ${config.color} is already
selected` };
        }
    }
    player.tankConfig = config;
    this.broadcastRoomUpdate();
    return { success: true };
}

```

```

setReady(playerId: string, ready: boolean): { success: boolean; message?: string
} {
    const player = this.players.get(playerId);
    if (!player) return { success: false, message: 'Player not found' };
    if (ready && !player.tankConfig) return { success: false, message: 'Cannot
become ready without tank' };
    player.ready = ready;
    this.broadcastRoomUpdate();
    if (ready && this.areAllPlayersReady()) void this.startGameAsync();
    return { success: true };
}

```

```

private areAllPlayersReady(): boolean {
    if (this.deathmatchMode && (this.players.size < 2 || this.players.size > 5))
return false;
}

```

```

        if (!this.deathmatchMode && this.players.size !== this.maxPlayers) return
false;
        return [...this.players.values()].every((p) => p.tankConfig && p.ready);
    }

    private async startGameAsync(): Promise<void> {
        const configs = [...this.players.values()].map((p) => ({ playerId: p.id, role:
p.role!, tankConfig: p.tankConfig! }));
        this.gameWorld = new GameWorld(configs, {
            practiceMode: this.practiceMode,
            deathmatchMode: this.deathmatchMode,
            replaySeed: Date.now()
        });

        if (!this.practiceMode && !this.singlePlayerTest) {
            this.matchId = await matchRepo.createMatchWithParticipants({
                roomCode: this.code,
                deathmatchMode: this.deathmatchMode,
                participants: configs
            });
            this.replayStartMeta = this.gameWorld.getReplayStartMeta();
            this.gameWorld.setReplayEventSink((event) => {
                if (this.replayEvents.length < Room.REPLAY_MAX_ACTIONS)
this.replayEvents.push(event);
            });
            this.gameWorld.pushReplayWorldInitEvent();
        }

        this.broadcast(JSON.stringify({ type: 'gameStart' }));
        this.broadcastSnapshot();
        this.lastGameLoopTime = Date.now();
        let accumulator = 0;
        this.gameLoopInterval = setInterval(() => {
            if (!this.gameWorld) return;
            const currentTime = Date.now();
            accumulator += Math.min(currentTime - this.lastGameLoopTime, 250);
            this.lastGameLoopTime = currentTime;

            let ticks = 0;

```

```

        while (accumulator >= this.FIXED_DELTA_MS && ticks < 5) {
            this.gameWorld.update(this.FIXED_DELTA_MS);
            accumulator -= this.FIXED_DELTA_MS;
            ticks++;
            const gameEnd = this.gameWorld.checkGameEnd();
            if (gameEnd) {
                this.endGame(gameEnd);
                return;
            }
        }
        if (ticks > 0) this.broadcastSnapshot();
    }, this.TICK_INTERVAL);
}

handlePlayerAction(playerId: string, action: any): void {
    if (!this.gameWorld) return;
    if (this.matchId && this.replayStartMeta && this.replayEvents.length <
Room.REPLAY_MAX_ACTIONS) {
        this.replayEvents.push({
            kind: 'player_input',
            tick:
this.gameWorld.getTick(), playerId, action });
    }
    this.gameWorld.handlePlayerAction(playerId, action, this.FIXED_DELTA_MS);
}

private broadcastSnapshot(): void {
    if (!this.gameWorld) return;
    this.broadcast(JSON.stringify({
        type: 'snapshot',
        tick: this.gameWorld.getTick(),
        world: this.gameWorld.getSnapshot()
    }));
}

public broadcast(message: string | Record<string, unknown>): void {
    const payload = typeof message === 'string' ? message :
JSON.stringify(message);
    for (const player of this.players.values()) {
        if (player.ws?.readyState === WebSocket.OPEN) player.ws.send(payload);
    }
}

```

```

    }

    private async endGame(result: any): Promise<void> {
        if (this.gameLoopInterval) clearInterval(this.gameLoopInterval);
        this.broadcast({ type: 'gameEnd', result });
        if (this.matchId && this.replayStartMeta && this.gameWorld) {
            await matchRepo.finalizeMatch(this.matchId, result);
            await replayRepo.saveMatchReplay({
                matchId: this.matchId,
                startMeta: this.replayStartMeta,
                events: this.replayEvents
            });
        }
    }
}

private broadcastRoomUpdate(): void {
    this.broadcast({
        type: 'roomUpdate',
        players: [...this.players.values()].map((p) => ({
            playerId: p.id,
            role: p.role,
            ready: p.ready,
            tankConfig: p.tankConfig,
            displayName: p.displayName
        })))
    });
}

}

import { Quadtree } from '../..//polygon/ICollisionSystem';
import { CollisionResolver } from '../..//geometry/CollisionResolver';
import { ITankModel } from '../..//model/tank/ITankModel';
import { BulletModelCreator } from '../..//model/bullet/BulletModelCreator';
import type { ReplayEvent, ReplayStartMeta } from '../..//repos/replayRepo';

export class GameWorld {
    private tanks = new Map<string, any>();
    private bullets = new Map<number, any>();

```

```

private walls = new Map<number, any>();
private crates = new Map<number, any>();
private items = new Map<number, any>();
private tick = 0;
private elapsedMs = 0;
private replayEventSink: ((event: ReplayEvent) => void) | null = null;

constructor(private players: any[], private options: any) {
    this.initializeLevel(1);
    this.spawnTanks();
}

private initializeLevel(level: number): void {
    this.walls.clear();
    this.crates.clear();
    this.items.clear();
}

private spawnTanks(): void {
    for (const player of this.players) {
        const tank = ITankModel.createFromConfig(player.tankConfig);
        tank.playerId = player.playerId;
        tank.role = player.role;
        this.tanks.set(player.playerId, tank);
    }
}

public handlePlayerAction(playerId: string, action: any, deltaMs: number): void {
    const tank = this.tanks.get(playerId);
    if (!tank || tank.destroyed) return;

    this.applyTankMovement(tank, action, deltaMs);
    if (action.turretLeft) tank.rotateTurret(-deltaMs);
    if (action.turretRight) tank.rotateTurret(deltaMs);

    if (action.shoot && tank.canShoot(this.elapsedMs)) {
        const bullet = BulletModelCreator.createFromTank(tank);
        this.bullets.set(bullet.id, bullet);
        tank.markShot(this.elapsedMs);
    }
}

```

```

    }
}

private applyTankMovement(tank: any, action: any, deltaMs: number): void {
    if (action.forward) tank.applyThrottle(1, deltaMs);
    if (action.backward) tank.applyThrottle(-1, deltaMs);
    if (action.turnLeft) tank.rotateHull(-deltaMs);
    if (action.turnRight) tank.rotateHull(deltaMs);
    tank.integrate(deltaMs);

    const collisionSystem = new Quadtree<any>(0, 0, 1920, 1080);
    for (const wall of this.walls.values()) collisionSystem.insert(wall);
    for (const crate of this.crates.values()) collisionSystem.insert(crate);

    for (const obstacle of collisionSystem.getCollisions(tank)) {
        CollisionResolver.resolveCollision(tank, obstacle);
    }
}

public update(deltaMs: number): void {
    this.tick++;
    this.elapsedMs += deltaMs;

    for (const tank of this.tanks.values()) {
        tank.applyPassiveForces(deltaMs);
        tank.integrateResidualMotion(deltaMs);
    }

    for (const bullet of this.bullets.values()) {
        bullet.update(deltaMs);
        this.checkBulletCollisions(bullet);
        if (bullet.isExpired(this.elapsedMs)) this.bullets.delete(bullet.id);
    }

    this.checkItemCollisions();
    this.updateBonusBoxSpawning(deltaMs);
}

private checkBulletCollisions(bullet: any): void {

```

```

        const candidates = [...this.walls.values(), ...this.crates.values(),
...this.tanks.values()];
        for (const target of candidates) {
            const collision = CollisionResolver.resolveCollision(bullet, target);
            if (!collision) continue;
            if (target.applyDamage) target.applyDamage(bullet.damage,
bullet.penetration);
            this.bullets.delete(bullet.id);
            return;
        }
    }

    private checkItemCollisions(): void {
        for (const tank of this.tanks.values()) {
            for (const item of this.items.values()) {
                if (!tank.intersects(item)) continue;
                tank.pickup(item);
                this.items.delete(item.id);
            }
        }
    }

    public setReplayEventSink(callback: (event: ReplayEvent) => void): void {
        this.replayEventSink = callback;
    }

    public pushReplayWorldInitEvent(): void {
        this.replayEventSink?.({
            kind: 'world_init',
            tick: this.tick,
            snapshot: this.getSnapshot()
        } as ReplayEvent);
    }

    public getReplayStartMeta(): ReplayStartMeta {
        return {
            tickRate: 60,
            mode: this.options.deathmatchMode ? 'deathmatch' : 'standard',
            seed: this.options.replaySeed,

```

```

        players: this.players
    };
}

public getSnapshot(): any {
    return {
        tanks: [...this.tanks.values()].map((t) => t.toSnapshot()),
        bullets: [...this.bullets.values()].map((b) => b.toSnapshot()),
        walls: [...this.walls.values()].map((w) => w.toSnapshot()),
        crates: [...this.crates.values()].map((c) => c.toSnapshot()),
        items: [...this.items.values()].map((i) => i.toSnapshot()),
        currentTimeMs: this.elapsedMs
    };
}

public getTick(): number {
    return this.tick;
}

public checkGameEnd(): any | null {
    return null;
}
}

import { Axis, Point, Vector } from './Point';
import { IPolygon } from '../polygon/IPolygon';

type Projection = { min: number; max: number };

export class CollisionDetector {
    private constructor() {}

    public static hasCollision(
        polygon1: IPolygon,
        polygon2: IPolygon,
        axes1: Axis[] = CollisionDetector.getAxes(polygon1),
        axes2: Axis[] = CollisionDetector.getAxes(polygon2)
    ): boolean {

```

```

    for (const axis of [...axes1, ...axes2]) {
        const projection1 = this.getProjection(polygon1, axis);
        const projection2 = this.getProjection(polygon2, axis);
        if (!this.areProjectionsOverlap(projection1, projection2)) return false;
    }
    return true;
}

public static getAxes(polygon: IPolygon): Axis[] {
    const axes: Axis[] = [];
    for (let i = 0; i < polygon.points.length; i++) {
        const next = (i + 1) % polygon.points.length;
        axes.push(Axis.create(polygon.points[i], polygon.points[next]));
    }
    return axes;
}

public static getCollisionResult(
    polygon1: IPolygon,
    polygon2: IPolygon
): { normal: Vector; overlap: number; collisionPoint: Point } | null {
    let minOverlap = Number.POSITIVE_INFINITY;
    let smallestAxis: Axis | null = null;

    for (const axis of [...this.getAxes(polygon1), ...this.getAxes(polygon2)]) {
        const p1 = this.getProjection(polygon1, axis);
        const p2 = this.getProjection(polygon2, axis);
        if (!this.areProjectionsOverlap(p1, p2)) return null;
        const overlap = Math.min(p1.max, p2.max) - Math.max(p1.min, p2.min);
        if (overlap < minOverlap) {
            minOverlap = overlap;
            smallestAxis = axis;
        }
    }

    const center1 = polygon1.calcCenter();
    const center2 = polygon2.calcCenter();
    const direction = new Vector(center2.x - center1.x, center2.y - center1.y);
    const normal = smallestAxis!.clone();

```

```

    if (normal.x * direction.x + normal.y * direction.y < 0) normal.scale(-1);

    return {
        normal,
        overlap: minOverlap,
        collisionPoint: this.findClosestVertex(polygon1, polygon2)
    };
}

private static areProjectionsOverlap(a: Projection, b: Projection): boolean {
    return a.max >= b.min && b.max >= a.min;
}

private static getProjection(polygon: IPolygon, axis: Axis): Projection {
    let min = Number.POSITIVE_INFINITY;
    let max = Number.NEGATIVE_INFINITY;
    for (const point of polygon.points) {
        const projection = point.x * axis.x + point.y * axis.y;
        min = Math.min(min, projection);
        max = Math.max(max, projection);
    }
    return { min, max };
}

private static findClosestVertex(polygon1: IPolygon, polygon2: IPolygon): Point {
    const center = polygon2.calcCenter();
    let best = polygon1.points[0];
    let bestDistance = Number.POSITIVE_INFINITY;
    for (const point of polygon1.points) {
        const dx = point.x - center.x;
        const dy = point.y - center.y;
        const d = dx * dx + dy * dy;
        if (d < bestDistance) {
            best = point;
            bestDistance = d;
        }
    }
    return best.clone();
}

```

```

}

import { CollisionDetector } from '../geometry/CollisionDetector';
import { Axis, Point } from '../geometry/Point';
import { IPolygon } from './IPolygon';

export interface ICollisionDetection<T extends IPolygon> {
    getCollisions(polygon: IPolygon): Iterable<T>;
}

class Boundary implements IPolygon {
    public readonly points: Point[];
    public readonly axes: Axis[];
    public readonly angle = 0;

    constructor(xStart: number, yStart: number, xLast: number, yLast: number, public
readonly id: number) {
        this.points = [
            new Point(xStart, yStart),
            new Point(xLast, yStart),
            new Point(xLast, yLast),
            new Point(xStart, yLast)
        ];
        this.axes = CollisionDetector.getAxes(this);
    }

    public calcCenter(): Point {
        return new Point((this.points[0].x + this.points[2].x) / 2, (this.points[0].y
+ this.points[2].y) / 2);
    }
}

export class Quadtree<T extends IPolygon> implements ICollisionDetection<T> {
    private root: QuadtreeNode<T>;
    private readonly boundary: Boundary;

    constructor(xStart: number, yStart: number, xLast: number, yLast: number) {
        this.boundary = new Boundary(xStart, yStart, xLast, yLast, -1);
    }
}

```

```

        this.root = new QuadtreeNode(this.boundary);
    }

    public insert(entity: T): void {
        this.root.insert(entity, CollisionDetector.getAxes(entity));
    }

    public remove(entity: T): void {
        this.root.remove(entity, CollisionDetector.getAxes(entity));
    }

    public getCollisions(polygon: IPolygon): Iterable<T> {
        const result = new Map<number, T>();
        this.root.getCollisions(polygon, CollisionDetector.getAxes(polygon), result);
        return result.values();
    }

    public clear(): void {
        this.root = new QuadtreeNode(this.boundary);
    }
}

class QuadtreeNode<T extends IPolygon> {
    private static readonly CAPACITY = 8;
    private totalPolygons = 0;
    private polygons: Map<number, T> | null = new Map();
    private children: QuadtreeNode<T>[] | null = null;

    constructor(private readonly boundary: Boundary) {}

    private isSubdivided(): boolean {
        return this.polygons === null;
    }

    public insert(entity: T, axes: Axis[]): void {
        if (!CollisionDetector.hasCollision(this.boundary, entity,
this.boundary.axes, axes)) return;
        if (this.isSubdivided()) {
            this.totalPolygons = 0;

```

```

        for (const child of this.children!) {
            child.insert(entity, axes);
            this.totalPolygons += child.totalPolygons;
        }
        return;
    }
    this.totalPolygons++;
    this.polygons!.set(entity.id, entity);
    if (this.polygons!.size > QuadtreeNode.CAPACITY) this.subdivide();
}

public remove(entity: T, axes: Axis[]): void {
    if (!CollisionDetector.hasCollision(this.boundary, entity,
this.boundary.axes, axes)) return;
    if (this.isSubdivided()) {
        this.totalPolygons = 0;
        for (const child of this.children!) {
            child.remove(entity, axes);
            this.totalPolygons += child.totalPolygons;
        }
        if (this.totalPolygons <= (QuadtreeNode.CAPACITY >> 1))
this.mergeWithChildren();
    } else {
        this.totalPolygons--;
        this.polygons!.delete(entity.id);
    }
}

public getCollisions(polygon: IPolygon, axes: Axis[], result: Map<number, T>):
void {
    if (this.isSubdivided()) {
        for (const child of this.children!) {
            if (CollisionDetector.hasCollision(child.boundary, polygon,
child.boundary.axes, axes)) {
                child.getCollisions(polygon, axes, result);
            }
        }
    }
    return;
}

```

```

        for (const entity of this.polygons!.values()) {
            if (CollisionDetector.hasCollision(polygon, entity, axes,
CollisionDetector.getAxes(entity))) {
                result.set(entity.id, entity);
            }
        }
    }
}

```

```

private subdivide(): void {
    const [p0, , p2] = this.boundary.points;
    const width = (p2.x - p0.x) / 2;
    const height = (p2.y - p0.y) / 2;
    this.children = [
        new QuadtreeNode(new Boundary(p0.x, p0.y, p0.x + width, p0.y + height, -
1)),
        new QuadtreeNode(new Boundary(p0.x + width, p0.y, p2.x, p0.y + height, -
1)),
        new QuadtreeNode(new Boundary(p0.x, p0.y + height, p0.x + width, p2.y, -
1)),
        new QuadtreeNode(new Boundary(p0.x + width, p0.y + height, p2.x, p2.y, -
1))
    ];
    this.redistribute();
}

```

```

private redistribute(): void {
    const old = this.polygons!;
    this.polygons = null;
    this.totalPolygons = 0;
    for (const entity of old.values()) {
        const axes = CollisionDetector.getAxes(entity);
        for (const child of this.children!) child.insert(entity, axes);
    }
    for (const child of this.children!) this.totalPolygons += child.totalPolygons;
}

```

```

private mergeWithChildren(): void {
    this.polygons = new Map();
    for (const child of this.children!) {

```

```

        if (child.isSubdivided()) child.mergeWithChildren();
        for (const entity of child.polygons!.values())
this.polygons.set(entity.id, entity);
    }
    this.totalPolygons = this.polygons.size;
    this.children = null;
}
}

```

```

import { GameWorld } from './GameWorld';
import type { ReplayEvent, ReplayPlayerInputEvent, ReplayStartMeta } from
'../../repos/replayRepo';

```

```

export type ReplayFrame = { tick: number; world: any };

```

```

const FALLBACK_TICK_RATE = 60;

```

```

function groupActionsByTick(actions: ReplayPlayerInputEvent[]): Map<number,
ReplayPlayerInputEvent[]> {
    const byTick = new Map<number, ReplayPlayerInputEvent[]>();
    for (const action of actions) {
        const list = byTick.get(action.tick) ?? [];
        list.push(action);
        byTick.set(action.tick, list);
    }
    return byTick;
}

```

```

function createWorldFromReplayMeta(meta: ReplayStartMeta): GameWorld {
    return new GameWorld(meta.players, {
        deathmatchMode: meta.mode === 'deathmatch',
        practiceMode: false,
        replaySeed: meta.seed
    });
}

```

```

export function buildReplayFramesFromEvents(meta: ReplayStartMeta, events:
ReplayEvent[]): ReplayFrame[] {

```

```

const tickRate = meta.tickRate || FALLBACK_TICK_RATE;
const stepMs = 1000 / tickRate;
const world = createWorldFromReplayMeta(meta);
const actions = events.filter((e): e is ReplayPlayerInputEvent => e.kind ===
'player_input');
const actionsByTick = groupActionsByTick(actions);
const lastTick = Math.max(0, ...events.map((e) => e.tick));
const frames: ReplayFrame[] = [];

for (let tick = 0; tick <= lastTick; tick++) {
  for (const action of actionsByTick.get(tick) ?? []) {
    world.handlePlayerAction(action.playerId, action.action, stepMs);
  }
  world.update(stepMs);
  frames.push({ tick, world: world.getSnapshot() });
  if (world.checkGameEnd()) break;
}
return frames;
}

```

```

import { getPool } from '../db/pool';

```

```

export async function createMatchWithParticipants(input: any): Promise<string> {
  const pool = getPool();
  if (!pool) throw new Error('Database is not configured');
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const typeCode = input.deathmatchMode ? 'kill_time' : 'standard';
    const type = await client.query('SELECT match_type_id FROM match_types WHERE
code=$1', [typeCode]);
    const match = await client.query(
      `INSERT INTO matches(match_type_id, room_code, match_status, started_at)
VALUES($1, $2, 'in_progress', NOW())
RETURNING match_id`,
      [type.rows[0].match_type_id, input.roomCode]
    );
    const matchId = match.rows[0].match_id;

```

```

        await client.query('COMMIT');
        return matchId;
    } catch (error) {
        await client.query('ROLLBACK');
        throw error;
    } finally {
        client.release();
    }
}

export async function finalizeMatch(matchId: string, result: any): Promise<void> {
    const pool = getPool();
    if (!pool) return;
    await pool.query(
        `UPDATE matches
        SET match_status='completed', winner_role=$2, end_reason=$3,
            duration_ticks=$4, ended_at=NOW(), updated_at=NOW()
        WHERE match_id=$1`,
        [matchId, result.winnerRole ?? null, result.reason, result.durationTicks]
    );
}

import { getPool } from '../db/pool';

export type ReplayWorldInitEvent = { kind: 'world_init'; tick: number; snapshot: any };
export type ReplayItemSpawnEvent = { kind: 'item_spawn'; tick: number; item: any };
export type ReplayPlayerInputEvent = { kind: 'player_input'; tick: number; playerId: string; action: any };
export type ReplayEvent = ReplayWorldInitEvent | ReplayItemSpawnEvent | ReplayPlayerInputEvent;

export type ReplayStartMeta = {
    tickRate: number;
    mode: 'standard' | 'deathmatch';
    seed: number;
    players: any[];
};

```

```

export async function saveMatchReplay(input: {
  matchId: string;
  startMeta: ReplayStartMeta;
  events: ReplayEvent[];
}): Promise<void> {
  const pool = getPool();
  if (!pool) return;
  await pool.query(
    `INSERT INTO match_replay_actions(match_id, start_meta, events)
    VALUES($1, $2::jsonb, $3::jsonb)
    ON CONFLICT (match_id)
    DO UPDATE SET start_meta=$2::jsonb, events=$3::jsonb`,
    [input.matchId,
      JSON.stringify(input.startMeta),
      JSON.stringify(input.events)]
  );
}

export async function getReplayActions(matchId: string): Promise<{ meta:
ReplayStartMeta; events: ReplayEvent[] } | null> {
  const pool = getPool();
  if (!pool) return null;
  const result = await pool.query(
    `SELECT start_meta, events FROM match_replay_actions WHERE match_id=$1`,
    [matchId]
  );
  if (!result.rowCount) return null;
  return {
    meta: result.rows[0].start_meta,
    events: result.rows[0].events ?? []
  };
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Листинг исходного кода клиентской части

```
import React from 'react';
import { Navigate, Route, Routes } from 'react-router-dom';
import AppShell from './layout/AppShell';
import { AuthProvider } from './context/AuthContext';
import { GameSocketProvider } from './context/GameSocketContext';
import { SettingsProvider } from './context/SettingsContext';
import GuestRoute from './routes/GuestRoute';
import ProtectedRoute from './routes/ProtectedRoute';
import RootRedirect from './routes/RootRedirect';
import HomePage from './pages/HomePage';
import LoginPage from './pages/LoginPage';
import PlayPage from './pages/PlayPage';
import RegisterPage from './pages/RegisterPage';
import ReplaysPage from './pages/ReplaysPage';
import ReplayWatchPage from './pages/ReplayWatchPage';
import SharedReplayPage from './pages/SharedReplayPage';
import './styles/index.css';

const AppRoutes: React.FC = () => (
  <Routes>
    <Route path="/" element={ <RootRedirect /> } />
    <Route path="/s/:slug" element={ <SharedReplayPage /> } />
    <Route element={ <GuestRoute /> }>
      <Route path="/login" element={ <LoginPage /> } />
      <Route path="/register" element={ <RegisterPage /> } />
    </Route>
    <Route element={ <ProtectedRoute /> }>
      <Route element={ <AppShell /> }>
        <Route path="/home" element={ <HomePage /> } />
        <Route path="/play" element={ <PlayPage /> } />
        <Route path="/replays" element={ <ReplaysPage /> } />
        <Route path="/replays/watch/:replayId" element={ <ReplayWatchPage /> } />
      </Route>
    </Route>
  </Routes>
)
```

```

        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    );

const App: React.FC = () => (
  <SettingsProvider>
    <AuthProvider>
      <GameSocketProvider>
        <AppRoutes />
      </GameSocketProvider>
    </AuthProvider>
  </SettingsProvider>
);

export default App;

export interface ServerMessage {
  type: string;
  [key: string]: any;
}

export interface ClientMessage {
  type: string;
  [key: string]: any;
}

type MessageHandler = (message: ServerMessage) => void;
const DEFAULT_GAME_WS_URL = 'ws://localhost:3001';
const WS_PATH = '/ws';

export class WebSocketClient {
  private socket: WebSocket | null = null;
  private listeners = new Map<string, Set<MessageHandler>>>();
  private pendingMessages: ClientMessage[] = [];
  private reconnectAttempts = 0;
  private reconnectTimer: number | null = null;
  private manuallyClosed = false;

```

```

    constructor(private readonly token: string, private readonly baseUrl =
DEFAULT_GAME_WS_URL) {}

    private buildConnectionUrl(): string {
        const url = new URL(WS_PATH, this.baseUrl);
        url.searchParams.set('token', this.token);
        return url.toString().replace(/^http/, 'ws');
    }

    public connect(): void {
        if (this.socket && this.socket.readyState <= WebSocket.OPEN) return;
        this.manuallyClosed = false;
        this.socket = new WebSocket(this.buildConnectionUrl());

        this.socket.onopen = () => {
            this.reconnectAttempts = 0;
            this.flushPendingMessages();
            this.emit('open', { type: 'open' });
        };

        this.socket.onmessage = (event) => {
            try {
                const message = JSON.parse(event.data);
                this.emit(message.type, message);
                this.emit('*', message);
            } catch {
                this.emit('error', { type: 'error', message: 'Invalid server message'
});
            }
        };

        this.socket.onclose = () => {
            this.emit('close', { type: 'close' });
            if (!this.manuallyClosed) this.attemptReconnect();
        };

        this.socket.onerror = () => {
            this.emit('error', { type: 'error', message: 'WebSocket error' });
        };
    }

```

```

    }

    public send(message: ClientMessage): void {
        if (this.socket?.readyState === WebSocket.OPEN) {
            this.socket.send(JSON.stringify(message));
            return;
        }
        this.pendingMessages.push(message);
        this.connect();
    }

    private attemptReconnect(): void {
        if (this.reconnectAttempts >= 5) return;
        const delay = Math.min(1000 * 2 ** this.reconnectAttempts, 10000);
        this.reconnectAttempts++;
        this.reconnectTimer = window.setTimeout(() => this.connect(), delay);
    }

    private flushPendingMessages(): void {
        const messages = this.pendingMessages.splice(0);
        for (const message of messages) this.send(message);
    }

    public on(event: string, handler: MessageHandler): void {
        const handlers = this.listeners.get(event) ?? new Set();
        handlers.add(handler);
        this.listeners.set(event, handlers);
    }

    private emit(event: string, message: ServerMessage): void {
        for (const handler of this.listeners.get(event) ?? []) handler(message);
    }
}

import React, { useCallback, useEffect, useState } from 'react';
import { useGameSocket } from '../context/GameSocketContext';
import PlayHubScreen from '../components/ui/PlayHubScreen';
import OnlineTankCustomizer from '../components/ui/OnlineTankCustomizer';

```

```

import LobbyScreen from '../components/ui/LobbyScreen';
import GameScreen from '../components/ui/GameScreen';
import GameEndScreen from '../components/ui/GameEndScreen';

type PlayScreen = 'hub' | 'customizer' | 'lobby' | 'game' | 'end';

const PlayPage: React.FC = () => {
  const { client, send } = useGameSocket();
  const [screen, setScreen] = useState<PlayScreen>('hub');
  const [roomCode, setRoomCode] = useState<string | null>(null);
  const [playerId, setPlayerId] = useState<string | null>(null);
  const [role, setRole] = useState<string | null>(null);
  const [players, setPlayers] = useState<any[]>([]);
  const [lastSnapshot, setLastSnapshot] = useState<any | null>(null);
  const [gameResult, setGameResult] = useState<any | null>(null);

  useEffect(() => {
    if (!client) return;
    const handleMessage = (message: any) => {
      if (message.type === 'joined') {
        setRoomCode(message.roomId);
        setPlayerId(message.playerId);
        setRole(message.role);
        setScreen('customizer');
      } else if (message.type === 'roomUpdate') {
        setPlayers(message.players ?? []);
        setScreen((current) => current === 'game' ? current : 'lobby');
      } else if (message.type === 'gameStart') {
        setScreen('game');
      } else if (message.type === 'snapshot') {
        setLastSnapshot(message.world);
      } else if (message.type === 'gameEnd') {
        setGameResult(message.result);
        setScreen('end');
      }
    };
    client.on('*', handleMessage);
    return () => client.off('*', handleMessage);
  }, [client]);

```

```

    const createRoom = useCallback((mode: string) => send({ type: 'createRoom', mode
    })), [send]);
    const joinRoom = useCallback((code: string) => send({ type: 'joinRoom', code })),
    [send]);
    const submitTank = useCallback((config: any) => {
        send({ type: 'tankConfig', data: config });
        setScreen('lobby');
    }, [send]);
    const setReady = useCallback((ready: boolean) => send({ type: 'ready', ready })),
    [send]);

    if (screen === 'customizer') return <OnlineTankCustomizer onSubmit={submitTank}
    />;
    if (screen === 'lobby') return <LobbyScreen players={players} roomCode={roomCode}
    onReady={setReady} />;
    if (screen === 'game') return <GameScreen playerId={playerId} role={role}
    snapshot={lastSnapshot} send={send} />;
    if (screen === 'end') return <GameEndScreen result={gameResult} onExit={() =>
    setScreen('hub')} />;
    return <PlayHubScreen onCreate={createRoom} onJoin={joinRoom} />;
};

```

```

export default PlayPage;

```

```

import React, { useEffect, useRef } from 'react';
import { OnlineGameRenderer } from '../../online/OnlineGameRenderer';
import { ICanvas } from '../../game/processors/ICanvas';

```

```

const REQUIRED_KEYS = ['forward', 'backward', 'turnLeft', 'turnRight', 'turretLeft',
'turretRight', 'shoot'];

```

```

function buildAction(keys: Set<string>): any {
    return {
        forward: keys.has('KeyW'),
        backward: keys.has('KeyS'),
        turnLeft: keys.has('KeyA'),
        turnRight: keys.has('KeyD'),
    }
}

```

```

    turretLeft: keys.has('ArrowLeft'),
    turretRight: keys.has('ArrowRight'),
    shoot: keys.has('Space')
  };
}

const GameScreen: React.FC<any> = ({ playerId, snapshot, send }) => {
  const canvasRef = useRef<HTMLCanvasElement | null>(null);
  const rendererRef = useRef<OnlineGameRenderer | null>(null);
  const pressedKeysRef = useRef(new Set<string>());

  useEffect(() => {
    if (!canvasRef.current) return;
    const canvas = new ICanvas(canvasRef.current);
    rendererRef.current = new OnlineGameRenderer(canvas);
    return () => rendererRef.current?.clear();
  }, []);

  useEffect(() => {
    if (!snapshot) return;
    rendererRef.current?.updateFromSnapshot(snapshot);
    rendererRef.current?.render();
  }, [snapshot]);

  useEffect(() => {
    const onKeyDown = (event: KeyboardEvent) =>
pressedKeysRef.current.add(event.code);
    const onKeyUp = (event: KeyboardEvent) =>
pressedKeysRef.current.delete(event.code);
    window.addEventListener('keydown', onKeyDown);
    window.addEventListener('keyup', onKeyUp);
    const timer = window.setInterval(() => {
      const action = buildAction(pressedKeysRef.current);
      if (REQUIRED_KEYS.some((key) => action[key])) {
        send({ type: 'action', action });
      }
    }, 1000 / 60);
    return () => {
      window.removeEventListener('keydown', onKeyDown);

```

```

        window.removeEventListener('keyup', onKeyUp);
        window.clearInterval(timer);
    };
}, [send]));

return (
    <section className="game-screen">
        <canvas ref={canvasRef} className="game-canvas" />
        <div className="game-hud">P PiC PsPe: {playerId}</div>
    </section>
);
};

export default GameScreen;

import { ICanvas } from '../game/processors/ICanvas';
import { TankSprite } from '../sprite/tank/TankSprite';
import { TankPartsCreator } from '../components/tank parts/TankPartsCreator';

type RenderableTank = { sprite: TankSprite; config: any };

export class OnlineGameRenderer {
    private tanks = new Map<string, RenderableTank>();
    private bullets = new Map<number, any>();

    constructor(private readonly canvas: ICanvas) {}

    public updateFromSnapshot(snapshot: any): void {
        this.updateTanks(snapshot.tanks ?? []);
        this.updateBullets(snapshot.bullets ?? []);
    }

    private updateTanks(serverTanks: any[]): void {
        const currentIds = new Set(serverTanks.map((tank) => tank.id));
        for (const [id, tank] of this.tanks.entries()) {
            if (!currentIds.has(id)) {
                this.canvas.removeById(tank.sprite);
                this.tanks.delete(id);
            }
        }
    }
}

```

```

    }
  }
  for (const serverTank of serverTanks) {
    let tank = this.tanks.get(serverTank.id);
    if (!tank) {
      const parts = TankPartsCreator.create(
        serverTank.hullNum,
        serverTank.trackNum,
        serverTank.turretNum,
        serverTank.weaponNum,
        serverTank.color
      );
      tank = { sprite: new TankSprite(parts), config: serverTank };
      this.tanks.set(serverTank.id, tank);
      this.canvas.add(tank.sprite);
    }
    tank.sprite.setPosition(serverTank.x, serverTank.y);
    tank.sprite.setHullAngle(serverTank.angle);
    tank.sprite.setTurretAngle(serverTank.turretAngle);
    tank.sprite.setHealth(serverTank.health, serverTank.maxHealth);
  }
}

private updateBullets(serverBullets: any[]): void {
}

public render(): void {
  this.canvas.render();
}

public clear(): void {
  this.canvas.clear();
  this.tanks.clear();
  this.bullets.clear();
}
}

```

```
import React, { useEffect, useRef, useState } from 'react';
```

```

import { OnlineGameRenderer } from '../..online/OnlineGameRenderer';
import { ICanvas } from '../..game/processors/ICanvas';
import { getReplayPlaybackData } from '../..auth/gameApi';

const ReplayPlaybackScreen: React.FC<{ replayId: string; token: string }> = ({
  replayId, token }) => {
  const canvasRef = useRef<HTMLCanvasElement | null>(null);
  const rendererRef = useRef<OnlineGameRenderer | null>(null);
  const [frames, setFrames] = useState<any[]>([]);
  const [frameIndex, setFrameIndex] = useState(0);

  useEffect(() => {
    void getReplayPlaybackData(token, replayId).then((data) =>
      setFrames(data.frames ?? []));
  }, [token, replayId]);

  useEffect(() => {
    if (!canvasRef.current) return;
    rendererRef.current = new OnlineGameRenderer(new ICanvas(canvasRef.current));
    return () => rendererRef.current?.clear();
  }, []);

  useEffect(() => {
    const frame = frames[frameIndex];
    if (!frame) return;
    rendererRef.current?.updateFromSnapshot(frame.world);
    rendererRef.current?.render();
  }, [frames, frameIndex]);

  return <canvas ref={canvasRef} className="game-canvas" />;
};

export default ReplayPlaybackScreen;

```

ПРИЛОЖЕНИЕ В

(обязательное)

Результат проверки на заимствования



Отчет предоставлен сервисом
«Антиплагиат» - <http://my.antiplagiat.ru>



Отчет о проверке

Автор: Панкратьев Егор

Проверяющий: Панкратьев Егор

Название документа: Диплом_Панкратьев

РЕЗУЛЬТАТЫ ПРОВЕРКИ

Тариф: FULL



Совпадения:
5,17%

Оригинальность:
94,83%

ИИ-контент:
15,14%

Цитирования:
0%

Самоцитирования:
0%

«Совпадения», «Цитирования», «Самоцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.

Есть подозрения на следующие группы маскировки заимствований: Сгенерированный текст на страницах: 5, 6, 10, 11, 13, 16, 18, 19, 28, 29... еще на 14 стр.

Проверено: 95,73% текста документа, исключено из проверки: 4,27% текста документа. Разделы и элементы, отключенные пользователем: Библиография, Таблицы

- Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самоцитированию. Показатель «Совпадения» — это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- Самоцитирования** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самоцитирования» — это доля фрагментов текста, отнесенных к самоцитированию, в общем объеме текста.
- Цитирования** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы; библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» — это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» — это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 2

Тип документа: Не указано

Дата проверки: 17.05.2026 21:18:58

Дата корректировки: Нет

Количество страниц: 94

Символов в тексте: 150991

Слов в тексте: 17673

Число предложений: 3960

Рисунок В.1 - Результат проверки на заимствование

Обозначение					Наименование		Дополнительные сведения					
					Текстовые документы							
БГУИР ДП 1-40 01 01 01 072 ПЗ					Пояснительная записка		127 с.					
					Отзыв руководителя							
					Рецензия							
					Графические документы							
ГУИР.251003.001 СП					Программное средство		Формат А1					
					многопользовательского взаимодействия							
					в онлайн игре с физической моделью на							
					JavaScript-платформе.							
					Схема программы							
ГУИР.251003.001 СА					Алгоритм создания лабиринта.		Формат А1					
					Схема алгоритма							
ГУИР.251003.002 СА					Алгоритм обработки действия игрока.		Формат А1					
					Схема алгоритма							
ГУИР.251003.001 ПЛ					Диаграмма вариантов использования.		Формат А1					
					Плакат							
ГУИР.251003.002 ПЛ					Инфологическая модель базы данных.		Формат А1					
					Плакат							
ГУИР.251003.003 ПЛ					Диаграмма развертывания.		Формат А1					
					Плакат							
					Программные документы							
					Электронный носитель информации с про-		USB-накопитель					
					граммным обеспечением и материалами							
					БГУИР ДП 1-40 01 01 01 072 Д1							
Изм	Лист	№ докум.	Подпись	Дата	Программное средство многопользовательского взаимодействия в онлайн игре с физической моделью на JavaScript-платформе Ведомость документов		Литера	Лист	Листов			
Разраб.		Панкратьев Е.С.					T	127	127			
Проверил		Хмелев А.Г.					Кафедра ПОИТ гр. 251003					
Т.контр.		Хмелев А.Г.										
Н.контр.		Болтак С.В.										
Утвердил												

